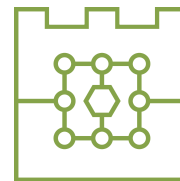




POLITECHNIKA KRAKOWSKA
im. T. Kościuszki
Wydział Informatyki i Telekomunikacji
Katedra Informatyki



Kierunek studiów: Informatyka
Specjalność: Cyberbezpieczeństwo

STUDIA STACJONARNE

PRACA DYPLOMOWA

MAGISTERSKA

Wojciech Drózd

Zastosowanie fine-tuning'u dużych modeli językowych do
wykrywania niechcianej poczty elektronicznej

Application of Large Language Model Fine-Tuning in Unwanted
Email Detection

Promotor:
dr inż. Dariusz Żelasko

Kraków, rok akad. 2025/2026

Spis treści

1. Wstęp	1
1.1. Motywacja	1
1.2. Cel pracy	1
1.3. Zakres i metoda badawcza	2
1.4. Pytania badawcze	2
1.5. Struktura pracy	2
2. Podstawy teoretyczne	4
2.1. Niechciana poczta elektroniczna	4
2.1.1. Pojęcie spamu	4
2.1.2. Typy niepożądanych wiadomości	4
2.1.3. Klasyfikacja wiadomości jako problem binarny	5
2.2. Klasyfikacja tekstu	5
2.2.1. Reprezentacja dokumentu tekstowego	5
2.2.2. Tokenizacja i długość kontekstu	6
2.2.3. Metryki oceny klasyfikatora	8
2.3. Modele językowe	10
2.3.1. Zadanie modelowania języka	10
2.3.2. Modele przyczynowe i przewidywanie następnego tokenu	10
2.3.3. Modele instrukcyjne i format czatu	11
2.4. Architektura Transformer	13
2.4.1. Mechanizm self-attention	13
2.4.2. Okno kontekstu i koszt przetwarzania sekwencji	15
2.4.3. Sieć feed-forward w bloku Transformer	16
2.5. Dostrajanie modeli językowych	17
2.5.1. Fine-tuning	17
2.5.2. Instruction tuning	18
2.5.3. Parameter-efficient fine-tuning	18
2.5.4. LoRA	19
2.5.5. Punkty kontrolne i przeuczenie	20
2.6. Modele językowe jako klasyfikatory	20
2.7. Modele z otwartymi wagami	21
2.7.1. Otwarte wagi i możliwość uruchomienia lokalnego	21
2.7.2. Rozmiar modelu, koszt treningu i inferencji	21
3. Przegląd metod wykrywania niechcianej poczty	23
3.1. Wielowarstwowe systemy filtrowania poczty	23
3.2. Metody regułowe, reputacyjne i uwierzytelnianie nadawcy	23
3.3. Klasyczne metody uczenia maszynowego	24
3.4. Szybkie modele tekstowe i reprezentacje semantyczne	25

3.5. Modele transformerowe i językowe	25
3.6. Zakres porównania eksperymentalnego	26
4. Część badawcza	28
4.1. Wybór modelu językowego z otwartymi wagami	28
4.2. Dane i metodologia badawcza	30
4.2.1. Analiza wykorzystanych zbiorów danych	30
4.2.2. Przygotowanie danych	35
4.2.3. Testowane sposoby dostrajania modelu	43
4.2.4. Ocena modelu bazowego bez dostrajania	44
4.3. Dobór parametrów treningu	46
4.3.1. Przestrzeń hiperparametrów	46
4.3.2. Ewaluacja pośrednich punktów kontrolnych	46
4.3.3. Zbiory użyte do ewaluacji	47
4.3.4. Metryki oceny	47
4.3.5. Statystyki przebiegu treningu	48
4.3.6. Ranking konfiguracji	50
4.3.7. Analiza przeuczenia	52
4.3.8. Wybór najlepszej konfiguracji	55
4.3.9. Profil błędów wybranej konfiguracji	56
4.3.10. Wnioski z eksperymentu	57
4.4. Porównanie metod dostrajania	57
4.5. Porównanie z metodami bazowymi	62
4.5.1. Dobór parametrów metod bazowych	62
4.5.2. Skuteczność klasyfikacji	63
4.5.3. Koszt inferencji	65
5. Podsumowanie i wnioski	67
5.1. Synteza przeprowadzonych badań	67
5.2. Odpowiedzi na pytania badawcze	67
5.3. Ograniczenia pracy	68
5.4. Możliwość wdrożenia	69
5.5. Kierunki dalszych badań	70
Bibliografia	71
Summary	76
Spis rysunków	78
Spis tabel	80
Lista kodów źródłowych	81

1. Wstęp

1.1. Motywacja

Poczta elektroniczna pozostaje jednym z podstawowych kanałów komunikacji prywatnej i zawodowej. Jej powszechność pozwala niewielkim kosztem rozsyłać wiadomości reklamowe, phishingowe i oszukańcze do dużej liczby odbiorców. Filtr musi rozpoznawać zmieniające się sposoby formułowania wiadomości i nie blokować poprawnej korespondencji. Fałszywy alarm może oznaczać przeoczenie ważnej wiadomości, natomiast przepuszczenie ataku naraża odbiorcę na utratę danych lub pieniędzy.

Jednym z etapów wielowarstwowego filtrowania poczty jest klasyfikacja tekstu. W klasycznych metodach treść reprezentuje się między innymi za pomocą TF-IDF, a następnie przekazuje do klasyfikatora. Modele językowe wykorzystują natomiast reprezentacje zależne od kontekstu oraz wiedzę zdobytą podczas treningu wstępnego. Ich zastosowanie wiąże się jednak z wyższym kosztem treningu i inferencji niż w przypadku prostszych klasyfikatorów. Filtr pocztowy może przetwarzać duży strumień wiadomości, dlatego oprócz skuteczności znaczenie mają również czas odpowiedzi, przepustowość i zużycie zasobów. Powstaje zatem pytanie, czy modele językowe uzyskują lepsze wyniki w klasyfikacji oraz jaki jest ich koszt w porównaniu z klasycznymi klasyfikatorami.

1.2. Cel pracy

Celem pracy jest sprawdzenie, czy mały model językowy dostrojony za pomocą adapterów LoRA może skutecznie klasyfikować wiadomości elektroniczne jako **ham** albo **spam**. Badanie ma określić wpływ parametrów treningu i sposobu sformułowania zadania na wyniki uzyskiwane na kilku korpusach wiadomości.

Drugim celem jest porównanie dostrojonego modelu z modelem bez dostrajania oraz prostszymi klasyfikatorami tekstu. Porównanie obejmuje skuteczność, profil błędów i koszt inferencji.

1.3. Zakres i metoda badawcza

Badania dotyczą klasyfikacji wiadomości na podstawie tematu i treści. Nie analizowano załączników, reputacji adresów i domen, pełnych nagłówków transportowych ani wyników SPF, DKIM i DMARC. Przyjęta klasa **spam** obejmuje niechciane reklamy, phishing oraz wiadomości oszukańcze. Model rozstrzyga, czy wiadomość jest pożądana, ale nie określa rodzaju wykrytego nadużycia.

Jako model bazowy wybrano **Qwen3-0.6B**, który dostrajano metodą LoRA. Dla klasyfikacji przez ocenę następnego tokenu porównano 16 konfiguracji różniących się długością kontekstu, współczynnikiem uczenia i parametrami adaptera. Oceniono również pośrednie punkty kontrolne. Wybrane ustawienia wykorzystano następnie do porównania czterech metod: klasyfikacji sekwencji, generowania etykiety, oceny następnego tokenu oraz generowania odpowiedzi ustrukturyzowanej.

Wyniki odniesiono do modelu bez dostrajania. Porównanie objęło Naive Bayes, regresję logistyczną i SVM korzystające z reprezentacji TF-IDF, a także **fastText**, MiniLM z regresją logistyczną oraz DistilBERT. Dla metod bazowych dobrano parametry i progi klasyfikacji na wydzielonym zbiorze walidacyjnym. Koszt inferencji zmierzono na wspólnej próbce 1000 wiadomości.

1.4. Pytania badawcze

Zakres pracy opisują następujące pytania badawcze:

1. Jakie wyniki osiąga mały model językowy dostrojony za pomocą LoRA na próbce treningowej i w zewnętrznej walidacji obejmującej źródła niewykorzystane do aktualizacji wag?
2. Który z badanych sposobów użycia modelu językowego daje najkorzystniejsze połączenie skuteczności, niezależności decyzji od formatu generowanej odpowiedzi i prostoty inferencji?
3. Jak parametry treningu oraz jego czas trwania wpływają na wynik modelu na korpusach niewykorzystanych w treningu?
4. Jaki kompromis między skutecznością a kosztem inferencji wynika z porównania dostrojonego modelu językowego z prostszymi klasyfikatorami?

1.5. Struktura pracy

Po wstępie przedstawiono podstawy teoretyczne klasyfikacji wiadomości oraz działania modeli językowych. Rozdział obejmuje tokenizację, architekturę Transformer, dostrajanie

adapterowe i sposoby wykorzystania modelu językowego do klasyfikacji. Następnie omówiono mechanizmy stosowane w filtrowaniu poczty, od reguł i reputacji po klasyczne algorytmy uczenia maszynowego oraz klasyfikatory transformerowe.

Część badawcza opisuje wybór modelu, przygotowanie zbiorów danych i przebieg eksperymentów. Kolejne sekcje dotyczą doboru parametrów LoRA, analizy punktów kontrolnych, porównania sposobów dostrajania oraz zestawienia najlepszego wariantu z metodami bazowymi. Rozdział końcowy odpowiada na pytania badawcze, omawia ograniczenia, możliwość wdrożenia klasyfikatora i dalsze kierunki badań.

2. Podstawy teoretyczne

2.1. Niechciana poczta elektroniczna

2.1.1. Pojęcie spamu

W odniesieniu do poczty elektronicznej spam oznacza niechcianą wiadomość wysyłaną masowo przez nadużycie systemów komunikacji elektronicznej [1]. Pod tą nazwą mieszczą się jednak wiadomości o różnym celu i poziomie ryzyka. W korpusach spamowych pojawiają się między innymi reklamy, próby oszustwa, phishing oraz fałszywe informacje [2]. W zadaniu klasyfikacji binarnej takie wiadomości są zwykle sprowadzane do jednej klasy niepożądaney.

2.1.2. Typy niepożądanych wiadomości

Spam reklamowy

Spam reklamowy służy przede wszystkim promowaniu produktu, usługi albo strony internetowej. Wiadomości zazwyczaj są uciążliwe i prowadzą do zaśmiecania skrzynki pocztowej. Reklamowany produkt bywa niskiej jakości, niezgodny z opisem, fałszywy albo szkodliwy dla odbiorcy. Dotyczy to zwłaszcza ofert leków, suplementów, usług finansowych, inwestycji lub produktów obiecujących szybki efekt.

Phishing

Phishing polega na pozyskaniu wrażliwych danych, na przykład danych logowania lub numerów kart płatniczych, przez podszycie się pod zaufaną firmę albo instytucję [3]. W poczcie elektronicznej atakujący najczęściej próbuje skłonić odbiorcę do kliknięcia linku, otwarcia załącznika albo wpisania danych na fałszywej stronie. Adres takiej strony może różnić się od prawdziwej witryny tylko pojedynczym znakiem, a jej wygląd może naśladować oryginalny serwis. Po przejęciu danych atakujący może przekierować użytkownika na prawdziwą stronę, aby zasymulować prawidłowe logowanie i nie wzbudzić podejrzeń. Dla zwiększenia skuteczności wiadomości phishingowe często wykorzystują elementy psychologiczne, takie jak presja czasu. Przykładem jest wiadomość podszywająca się pod InPost, opisana przez Bitdefender w 2024 roku [4]:

INPOST: Masz paczkę INPOST, która dotarła do stacji wysyłkowej, ale nie może zostać dostarczona ze względu na nieprawidłowe dane adresowe. Aby ułatwić dostawę, zaktualizuj informacje o dostawie tak szybko, jak to możliwe. Aby wprowadzić zmiany, otwórz link do pakietu: *[złośliwy link]*

Wiadomości oszukańcze

Wiadomości oszukańcze (ang. *scam*) opierają się przede wszystkim na inżynierii społecznej. Atakujący przyjmuje fałszywą tożsamość i próbuje doprowadzić do sytuacji, w której odbiorca sam poda wrażliwe dane albo przekaże pieniądze. Początkowo atak przypomina phishing, ale potem przechodzi w bezpośrednią rozmowę z oszustem. Manipulacja nie musi nastąpić od razu. W niektórych wariantach kontakt trwa wiele dni lub tygodni, a kolejne wiadomości stopniowo budują zaufanie [5].

W niniejszej pracy analizowane są wiadomości tekstowe, choć sam tekst może być tylko jednym elementem oszustwa. Upowszechnienie narzędzi AI umożliwia atakującym generowanie fałszywych zdjęć, nagrań głosu, a nawet przeprowadzanie wideorozmów ze zmianą wyglądu w czasie rzeczywistym. Takie techniki wzmacniają wiarygodność atakującego i przyspieszają zdobywanie zaufania ofiary [6].

2.1.3. Klasyfikacja wiadomości jako problem binarny

W tej pracy filtrowanie poczty traktowane jest jako klasyfikacja binarna: wiadomość może należeć do klasy korespondencji pożądanej albo do klasy *spam*, obejmującej spam reklamowy, phishing i wiadomości oszukańcze. Taki zapis odpowiada decyzji filtra poczty: przepuścić wiadomość albo oznaczyć ją jako niepożądaną. Przy ocenie modelu ważny jest więc nie tylko odsetek poprawnych decyzji, lecz także typ błędu. Błąd fałszywie dodatni (ang. *false positive*, FP) oznacza przeniesienie poprawnej wiadomości do spamu. Błąd fałszywie ujemny (ang. *false negative*, FN) oznacza przepuszczenie wiadomości niepożądanej do skrzynki odbiorczej.

2.2. Klasyfikacja tekstu

2.2.1. Reprezentacja dokumentu tekstowego

W klasyfikacji tekstu dokument otrzymuje etykietę określającą jego klasę. Dla wiadomości e-mail takim dokumentem może być sam temat, sama treść albo połączenie obu pól. Metadane, nagłówki i informacje techniczne są szczególnie użyteczne, ale w publicznych korpusach

rzadko mają jednolity format. Celem pracy jest również przeanalizowanie skuteczności wykrywania spamu na podstawie samego tekstu.

Model nie przetwarza jednak tekstu w takiej postaci, w jakiej widzi go użytkownik. W klasycznych metodach uczenia maszynowego wiadomość zamienia się zwykle na wektor cech, na przykład zliczenia słów albo wagi TF-IDF [2]. W modelach językowych wejściem jest sekwencja identyfikatorów tokenów.

Poczta elektroniczna jest pod tym względem mniej regularna niż typowy akapit tekstu. Wiadomość może zawierać cytowaną korespondencję, podpis, linki, fragmenty HTML, automatyczne stopki albo artefakty kodowania. Część tych elementów pomaga w klasyfikacji, a część wprowadza szum. Przygotowanie danych wymaga więc decyzji, które fragmenty wiadomości tworzą wejście modelu i jak traktować elementy występujące tylko w niektórych zbiorach.

2.2.2. Tokenizacja i długość kontekstu

Tokeny

Token jest podstawową jednostką tekstu przetwarzaną przez model językowy. W zależności od użytego tokenizatora może odpowiadać całemu słowu, fragmentowi słowa, znakowi interpunkcyjnemu, części adresu URL albo tokenowi specjalnemu. Dlatego długość wiadomości mierzona w tokenach różni się od długości mierzonej w słowach. Link, nazwa domeny, nietypowy zapis kwoty albo celowo zniekształcone słowo mogą zostać podzielone na wiele elementów.

Ma to znaczenie szczególnie dla spamu i phishingu. Takie wiadomości często zawierają adresy URL, losowe ciągi znaków, nazwy domen, kwoty, daty i nietypowe formatowanie. Dwie wiadomości o podobnej liczbie słów mogą więc po tokenizacji mieć różną długość. Różnica ta wpływa później na koszt przetwarzania i na to, czy cała treść mieści się w wejściu modelu.

Tokenizacja BPE

Byte Pair Encoding (BPE) jest algorytmem tworzenia tokenów przez łączenie często występujących obok siebie fragmentów tekstu. Na początku tekst jest reprezentowany jako krótkie jednostki, na przykład znaki albo bajty. Następnie algorytm znajduje najczęstsze pary sąsiednich jednostek i dodaje ich połączenia do słownika. Po wielu takich krokach częste fragmenty tekstu stają się pojedynczymi tokenami, a rzadsze pozostają zapisane jako kilka krótszych jednostek [7].

Przykład tokenizacji byte-level BPE na podstawie GPT-2 Jest to przykład dydaktyczny; w eksperymentach użyto tokenizatora Qwen3, który ma własny słownik i reguły tokenizacji. Tokenizer GPT-2 wybrano do opisu, ponieważ jego implementacja jest stosunkowo prosta

i publicznie dostępna. Nowsze modele często stosują bardziej rozbudowane wyrażenia regularne, dodatkowe tokeny specjalne oraz implementacje, których pełny przebieg trudniej odtworzyć wyłącznie z opisu modelu.

GPT-2 wykorzystywał wariant bajtowy BPE [8]. Tokenizer działa w trzech głównych krokach:

1. **Podział tekstu na fragmenty.** Wejściowy tekst jest dzielony za pomocą wyrażenia regularnego. W implementacji GPT-2 użyto następującego wzorca [9]:

```
's|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\\s\\p{L}\\p{N}]+|\\s+
+(?!\\S)|\\s+
```

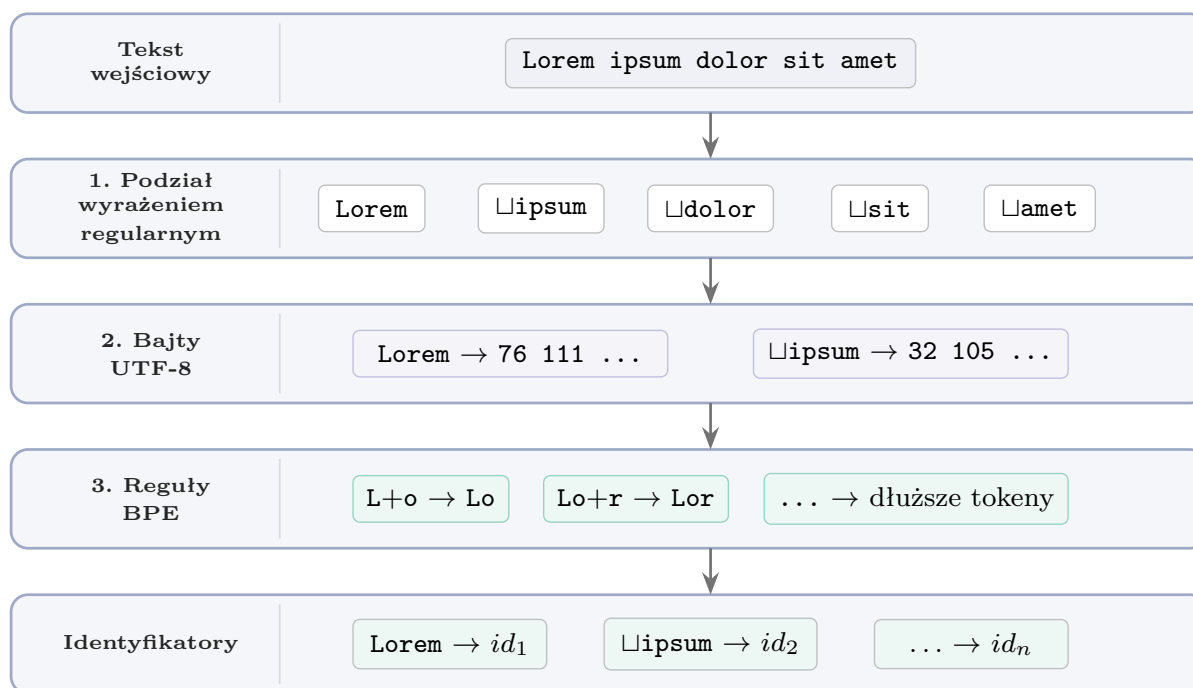
Dla tekstu `I've made a calculation! 3+3=6!!` taki podział daje fragmenty:

```
['I', "'ve", ' made', ' a', ' calculation', '!', ' 3', '+', '3', '=', '6', '!!!']
```

Zastosowanie takiego wyrażenia nie jest optymalne pod względem kompresji tekstu, ale ogranicza tworzenie tokenów łączących przypadkowe kombinacje liter, cyfr i interpunkcji. W opisie GPT-2 wskazano, że bez takiego ograniczenia BPE tworzyło wiele wariantów tych samych częstych słów, przez co gorzej wykorzystywało ograniczony słownik i pojemność modelu [8].

2. **Konwersja na bajty.** Każdy fragment jest zapisywany jako sekwencja bajtów UTF-8. Zbiór 256 wartości bajtowych tworzy bazowy słownik tokenizatora. Reprezentacja bajtowa pozwala zakodować każdy tekst w UTF-8 bez potrzeby wprowadzania osobnego tokenu dla każdego znaku Unicode i bez tokenu nieznanego dla znaków spoza słownika, na przykład `<unk>`. Unicode przypisuje znakom abstrakcyjne punkty kodowe, natomiast UTF-8 zapisuje je jako ciągi bajtów. Przykładowo znak `U+1F40E` jest reprezentowany w UTF-8 przez cztery bajty: `[240, 159, 144, 142]`. Tokenizer nie potrzebuje więc osobnego wpisu dla każdego znaku Unicode, ponieważ może zapisać go za pomocą elementów bazowego słownika bajtowego.
3. **Zastosowanie reguł BPE.** Na reprezentacji bajtowej stosowane są reguły BPE wyuczone wcześniej na korpusie tekstowym. Częste sąsiednie ciągi bajtów są łączone w dłuższe tokeny, a rzadsze fragmenty pozostają rozbite na krótsze jednostki. Podczas inferencji i treningu tokenizer stosuje gotowy słownik i listę połączeń.

Kolejne etapy tokenizacji zestawiono na rysunku [2.1](#).



␣ oznacza spację poprzedzającą fragment tekstu.

Rysunek 2.1.: Przykład tokenizacji byte-level BPE na podstawie GPT-2. Jest to przykład dydaktyczny przedstawiający trzy etapy tokenizacji tekstu Lorem ipsum dolor sit amet. W eksperymentach użyto tokenizatora Qwen3, który ma własny słownik i reguły tokenizacji.

Źródło: opracowanie własne.

Długość kontekstu

Maksymalna długość wejścia modelu nazywana jest oknem kontekstu. Gdy wiadomość po tokenizacji przekracza ten limit, konieczne jest jej obcięcie. Skracanie może usunąć ważny fragment, na przykład link umieszczony pod koniec treści. Dłuższe okno kontekstu zachowuje więcej informacji, ale zwiększa koszt treningu i inferencji. W praktyce długość wejścia jest więc kompromisem między ilością zachowanego tekstu a kosztem obliczeniowym.

2.2.3. Metryki oceny klasyfikatora

W problemach binarnych, takich jak klasyfikacja korespondencji, podstawowym elementem ewaluacji modelu jest macierz pomyłek. Przedstawia ona cztery możliwe kombinacje klasyfikacji. W tej pracy klasą pozytywną jest spam, phishing lub inna wiadomość niepo-

żądana, a klasą negatywną pożądana korespondencja. Znaczenie jej pól przedstawiono w tabeli 2.1.

Tabela 2.1.: Macierz pomyłek dla binarnej klasyfikacji wiadomości

	Rzeczywista wiadomość niepożądana	Rzeczywista wiadomość pożądana
Wiadomość przewidziana jako niepożądana	TP <i>True positive</i> Poprawnie wykryty spam	FP <i>False positive</i> Pożądana wiadomość oznaczona jako spam
Wiadomość przewidziana jako pożądana	FN <i>False negative</i> Spam przepuszczony jako wiadomość pożądana	TN <i>True negative</i> Poprawnie rozpoznana wiadomość pożądana

Accuracy podaje udział poprawnych decyzji w całym zbiorze:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Przy nierównych klasach samo *accuracy* może ukrywać problemy z rozpoznawaniem klasy rzadszej. Model wybierający głównie klasę większościową nadal może osiągnąć wysoki wynik, mimo że przepuszcza wiele przykładów drugiej klasy. Dlatego obok tej metryki stosuje się miary opisujące osobno decyzje dla spamu i wiadomości poświadanych.

Precision określa, jaki odsetek wiadomości oznaczonych przez model jako spam rzeczywiście należał do tej klasy:

$$\text{precision} = \frac{TP}{TP + FP}.$$

Recall pokazuje, jaki odsetek rzeczywistych wiadomości spamowych został wykryty:

$$\text{recall} = \frac{TP}{TP + FN}.$$

Specificity pełni podobną rolę dla klasy negatywnej. Mierzy odsetek wiadomości poświadanych, których model nie oznaczył jako spam:

$$\text{specificity} = \frac{TN}{TN + FP}.$$

F1 jest średnią harmoniczną *precision* i *recall*:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

F1 łączy *precision* i *recall* klasy pozytywnej. Niska wartość jednej z tych składowych mocno obniża wynik, dlatego metryka nie premiuje modeli dobrych tylko w jednym z tych aspektów. Nie uwzględnia jednak liczby poprawnie rozpoznanych przykładów klasy negatywnej, dlatego powinna być interpretowana razem z *specificity*, FPR oraz rozkładem klas.

Balanced accuracy uśrednia *recall* i *specificity*, więc w równym stopniu uwzględnia klasę pozytywną i negatywną:

$$\text{balanced accuracy} = \frac{\text{recall} + \text{specificity}}{2}.$$

2.3. Modele językowe

2.3.1. Zadanie modelowania języka

Model językowy opisuje prawdopodobieństwo sekwencji tokenów. Dla tekstu zapisanego jako $x_1, x_2, \dots, x_n$ oznacza to ocenę, jak prawdopodobna jest dana kolejność elementów języka. Taka definicja jest użyteczna także wtedy, gdy interesuje nas nie prawdopodobieństwo całego tekstu, lecz brakujący albo kolejny fragment. Model uczy się bowiem zależności między tokenami występującymi w korpusie [10].

Współczesny model językowy przechodzi najpierw trening na dużym zbiorze tekstu, bez etykiet przygotowanych dla jednego zadania. Uczy się wtedy składni, typowych związków między słowami, sposobów zapisu informacji oraz części regularności obecnych w danych treningowych. Dopiero później taki model można dostosować do konkretnego problemu, na przykład do rozróżniania wiadomości **spam** i **ham**. Nie oznacza to trenowania modelu od początku. Zwykle wystarcza dalsze uczenie na mniejszym korpusie zadaniowym.

W tej pracy model językowy zastępuje ręcznie zaprojektowany wektor cech opisany we wcześniejszej części rozdziału. Wiadomość jest zamieniana na sekwencję tokenów, a model wykorzystuje reprezentacje wyuczone podczas pretrainingu. Część badawcza sprawdza, jak skutecznie takie reprezentacje da się dostroić do wykrywania niepożądanego korespondencji.

2.3.2. Modele przyczynowe i przewidywanie następnego tokenu

W modelowaniu przyczynowym (ang. *causal language modeling*) model otrzymuje wcześniejsze tokeny i na ich podstawie przewiduje następny. Nie korzysta z informacji po prawej stronie przewidywanej pozycji, więc zależność odpowiada generowaniu tekstu od lewej do prawej. Dla sekwencji $x_1, \dots, x_n$ prawdopodobieństwo tekstu zapisuje się jako iloczyn prawdopodobieństw kolejnych tokenów:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid x_1, \dots, x_{i-1}).$$

Taki sposób uczenia wykorzystano między innymi w rodzinie modeli GPT [8, 11]. Podczas generowania model zwraca rozkład prawdopodobieństwa nad słownikiem tokenizatora. Z tego rozkładu można wybrać najbardziej prawdopodobny token albo próbować tokeny z uwzględnieniem parametrów dekodowania. Ten sam rozkład może posłużyć do klasyfikacji.

W zadaniu binarnym prompt można ułożyć tak, aby następnym oczekiwanym elementem była etykieta klasy. Model nie generuje wtedy pełnej odpowiedzi. Wystarczy porównać prawdopodobieństwa tokenów odpowiadających etykietom **spam** i **ham**. Decyzja klasyfikatora wynika z rozkładu prawdopodobieństwa w jednej pozycji, a nie z późniejszego parsowania tekstu zwróconego przez model. Takie podejście zależy jednak od tokenizacji etykiet oraz od sformułowania promptu. Jeżeli etykieta składa się z wielu tokenów albo prompt kieruje model w stronę innej odpowiedzi, porównanie staje się mniej jednoznaczne.

2.3.3. Modele instrukcyjne i format czatu

Sam pretraining uczy model przewidywania tekstu, ale nie gwarantuje, że model będzie wykonywał polecenia użytkownika w przewidywalnym formacie. Modele, z którymi zazwyczaj mamy kontakt, przechodzą więc dodatkowe dostrajanie instrukcyjne (ang. *instruction tuning*). Dane treningowe mają wtedy postać instrukcji i oczekiwanej odpowiedzi, a celem jest dopasowanie zachowania modelu do poleceń podawanych wprost przez użytkownika. W praktyce etap ten może być łączony z uczeniem z informacji zwrotnej od ludzi [12].

Modele instrukcyjne często pracują w formacie czatu. Zamiast pojedynczego ciągu tekstu wejście składa się z komunikatów o rolach. Najczęściej spotykane role to **system**, **user** i **assistant**. Komunikat systemowy określa ogólne zachowanie modelu, komunikat użytkownika zawiera polecenie, a odpowiedź asystenta jest tekstem generowanym przez model. W treści komunikatu asystenta mogą występować dodatkowe bloki, na przykład **think** służący do zapisu rozumowania lub **tool_call** opisujący wywołanie narzędzia. Przed przekazaniem danych do modelu komunikaty są zamieniane na sekwencję tokenów według szablonu czatu, który dodaje znaczniki ról i tokeny specjalne.

Schemat poglądowy takiej rozmowy, oparty na składni ChatML, pokazuje listing 2.1 [13].

Kod źródłowy 2.1: Schemat poglądowy rozmowy w stylu ChatML z blokiem analizy i wywołaniem narzędzia.

```
<|im_start|>system
You are a weather assistant. Answer weather questions briefly.
# Tools
<tools>
{"type": "function", "function": {"name": "get_weather", "description": "
    Get weather for a city", "parameters": {"type": "object", "properties":
```

```
    {"city": {"type": "string"}, "unit": {"type": "string"}}, "required":  
    ["city"]}}}  
</tools>  
Return each function call as JSON inside <tool_call></tool_call>.  
<|im_end|>  
<|im_start|>user  
What is the weather in Tokyo?<|im_end|>  
<|im_start|>assistant  
<think>  
I need to check the weather for Tokyo.  
</think>  
I will check the weather in Tokyo.  
<tool_call>  
{ "name": "get_weather", "arguments": { "city": "Tokyo", "unit": "celsius" } }  
</tool_call><|im_end|>  
<|im_start|>user  
<tool_response>  
{ "temperature": 22, "condition": "partly cloudy" }  
</tool_response><|im_end|>  
<|im_start|>assistant  
Tokyo is 22 degrees Celsius and partly cloudy.<|im_end|>
```

W klasyfikacji wiadomości taki format sprowadza zadanie do instrukcji: oceń treść wiadomości i zwróć jedną z dwóch etykiet. Pozostaje jednak problem kontroli odpowiedzi. Zamiast samego słowa **spam** model może wygenerować pełne zdanie, uzasadnienie albo odpowiedź w niepełnym formacie. Metody oparte na generowaniu wymagają więc parsera, który zamienia tekst odpowiedzi na etykietę. Metoda przewidywania następnego tokenu ogranicza ten problem, ponieważ porównuje prawdopodobieństwa etykiet przed wygenerowaniem dłuższej odpowiedzi.

Model instrukcyjny nadal trzeba dopasować do zadania, ale daje praktyczny punkt startowy. Rozpoznaje strukturę polecenia, pracuje w formacie czatu i może zostać dalej dostrojony na przykładach zawierających wiadomości e-mail oraz oczekiwane etykiety. Ta własność jest później wykorzystywana przy porównaniu testowanych wariantów klasyfikacji: klasyfikacji sekwencji, generowania etykiety, przewidywania następnego tokenu oraz generowania odpowiedzi strukturyzowanej.

2.4. Architektura Transformer

Architektura Transformer [14] składa się z powtarzanych bloków, które przetwarzają reprezentacje kolejnych pozycji sekwencji. Po tokenizacji identyfikatory tokenów są zamieniane na wektory. Do wektorów dodaje się informację o położeniu tokenu w tekście, a następnie cała sekwencja przechodzi przez kolejne bloki Transformera. Każdy blok aktualizuje opis danej pozycji na podstawie jej dotychczasowej postaci oraz kontekstu pozostałych tokenów.

Wariant decoder-only, używany w wielu generatywnych modelach językowych, działa przyczynowo: na podstawie wcześniejszych tokenów oblicza rozkład prawdopodobieństwa następnego tokenu. Pojedynczy blok łączy mechanizm self-attention z siecią feed-forward. Pierwszy element wiąże informacje z różnych pozycji sekwencji, a drugi przekształca reprezentację każdej pozycji osobno. W praktycznych implementacjach stosuje się także normalizację warstw i połączenia rezydualne, które stabilizują przepływ informacji przez wiele warstw modelu [14].

Ogólny przebieg danych przez model decoder-only przedstawiono na rysunku 2.2.

2.4.1. Mechanizm self-attention

Mechanizm self-attention jest jednym z podstawowych elementów architektury Transformer. Wyznacza wagi relacji między tokenami w tej samej sekwencji, a następnie wykorzystuje je do obliczenia nowej reprezentacji każdego tokenu. W klasycznych modelach sekwencyjnych informacja przechodziła zwykle krok po kroku przez kolejne pozycje. Transformer pozwala natomiast każdej pozycji porównać własną reprezentację z reprezentacjami innych pozycji w oknie kontekstu. Dzięki temu w jednej warstwie można powiązać fragmenty wiadomości oddalone od siebie, na przykład nazwę nadawcy, link w treści i późniejszą prośbę o wykonanie płatności.

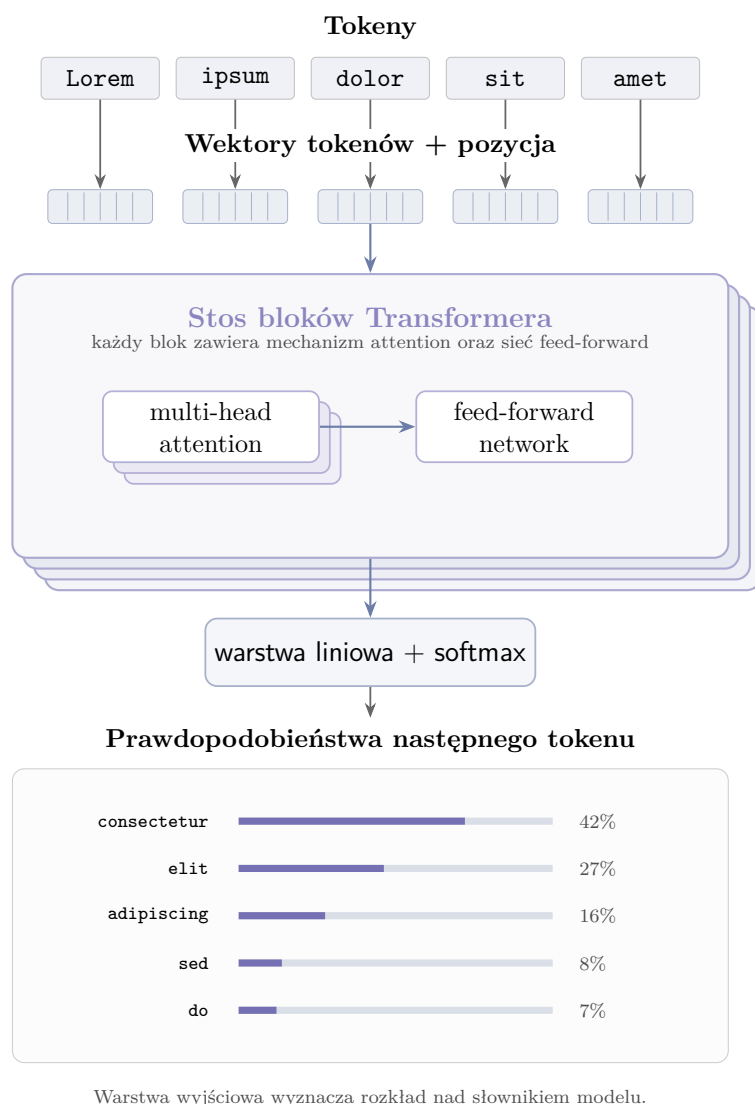
Dla macierzy reprezentacji wejściowych X warstwa self-attention tworzy trzy rzutowania liniowe: zapytania Q , klucze K oraz wartości V :

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Zapytania określają, czego dana pozycja szuka w sekwencji. Klucze służą do obliczenia dopasowania z innymi pozycjami, a wartości są wektorami, z których powstaje nowa reprezentacja. Wagi uwagi wyznacza się przez porównanie zapytań z kluczami. W wersji scaled dot-product attention ma to postać:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

gdzie d_k oznacza wymiar wektorów kluczy. Iloczyn QK^T tworzy macierz podobieństw między pozycjami, a funkcja softmax zamienia te wartości na wagi sumujące się do jedności. Warstwa zwraca ważoną sumę wektorów V , osobną dla każdej pozycji w sekwencji.



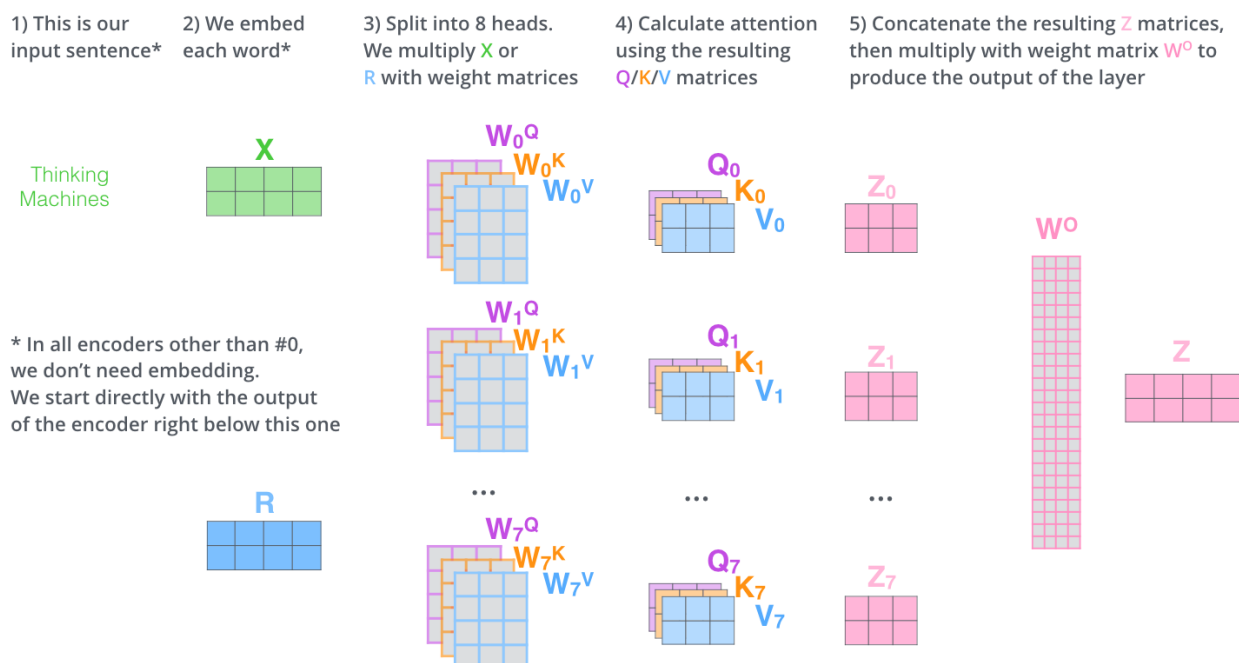
Rysunek 2.2.: Uproszczony przepływ informacji w modelu językowym opartym na architekturze Transformer dla sekwencji Lorem ipsum dolor sit amet.

Źródło: opracowanie własne.

W modelach językowych generujących tekst self-attention jest uzupełniany maską przyczynową. Maskę blokuje dostęp do tokenów znajdujących się po prawej stronie aktualnej pozycji. Model korzysta więc z wcześniejszego kontekstu, ale podczas przewidywania następnego tokenu nie widzi przyszłych elementów sekwencji. Jest to ten sam warunek, który wcześniej opisano przy modelowaniu przyczynowym.

Warstwa uwagi zwykle działa w wersji wielogłowej (ang. *multi-head attention*). Każda głowa ma własne rzutowania Q , K i V , a uzyskane wyniki są później łączone. Wielogłowy mechanizm uwagi umożliwia modelowanie różnych relacji; w tej pracy nie analizowano jednak funkcji poszczególnych głow.

Przykładowy przepływ danych między głowami uwagi pokazano na rysunku 2.3.



Rysunek 2.3.: Schemat działania mechanizmu multi-head self-attention.

Źródło: Jay Alammar, *The Illustrated Transformer* [15].

2.4.2. Okno kontekstu i koszt przetwarzania sekwencji

Okno kontekstu określa maksymalną liczbę tokenów, które model przetwarza w jednym przebiegu.

Do reprezentacji tokenów dodaje się informację o pozycji w sekwencji. W oryginalnym Transformerze zastosowano sinusoidalne reprezentacje pozycyjne [14]. Nowsze modele mogą

używać innych sposobów kodowania pozycji. Informacja o pozycji pozwala modelowi rozróżnić te same tokeny występujące w różnych miejscach tekstu.

W standardowym mechanizmie self-attention iloczyn QK^T dla sekwencji o długości n tworzy macierz $n \times n$, ponieważ warstwa uwagi porównuje każdą pozycję z każdą inną pozycją w sekwencji. Przejście z 512 do 1024 tokenów zwiększa rozmiar tej macierzy czterokrotnie. Część kosztu związana z macierzą uwagi rośnie jak $O(n^2)$; całkowity koszt zależy również od wymiaru ukrytego i implementacji.

Koszt obsługi długiego kontekstu jest jedną z głównych barier w rozwoju modeli językowych. Ograniczają go między innymi uwaga liniowa (ang. *linear attention*), uwaga rzadka (ang. *sparse attention*), uwaga o stałym wzorcu połączeń, warianty blokowe i blokowo-rzadkie oraz metody grupowania tokenów [16].

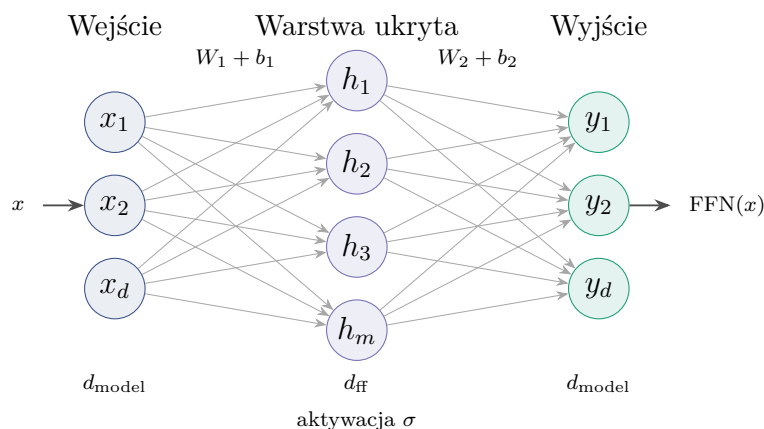
2.4.3. Sieć feed-forward w bloku Transformer

Po warstwie attention w każdym bloku Transformera znajduje się sieć feed-forward (ang. *feed-forward neural network*, FFN). Jej wejściem jest reprezentacja tokenu po uwzględnieniu kontekstu przez attention. Ta sama para przekształceń liniowych jest stosowana osobno dla każdej pozycji sekwencji:

$$\text{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2.$$

W oryginalnej architekturze Transformer funkcją σ była ReLU [14]; w nowszych modelach spotyka się także inne aktywacje i warianty bramkowane. Układ pozostaje podobny: pierwsze rzutowanie rozszerza wymiar wewnętrzny, aktywacja wprowadza nieliniowość, a drugie rzutowanie sprowadza wynik z powrotem do wymiaru używanego przez resztę modelu.

FFN przetwarza każdą pozycję osobno, ale korzysta z reprezentacji wzbogaconej wcześniej przez attention. W bloku Transformer attention zbiera więc informacje z sekwencji, a FFN wykonuje nieliniowe przekształcenie tej reprezentacji. Schemat dwóch przekształceń liniowych przedstawiono na rysunku 2.4.



Rysunek 2.4.: Uproszczony schemat sieci feed-forward typu MLP w bloku Transformer.

Źródło: opracowanie własne.

2.5. Dostrajanie modeli językowych

2.5.1. Fine-tuning

Dostrajanie modelu (ang. *fine-tuning*) polega na dalszym trenowaniu modelu wstępnie trenowanego na danych związanych z konkretnym zadaniem. Model nie jest wtedy uczony od początku. Punkt wyjścia stanowią parametry uzyskane podczas pretrenowania, a trening zadaniowy przesuwa zachowanie modelu w stronę danych, formatu odpowiedzi i kryterium oceny potrzebnych w danym zastosowaniu. W przypadku modeli instrukcyjnych takim treningiem mogą być pary złożone z polecenia oraz oczekiwanej odpowiedzi [12].

Fine-tuning różni się zarówno od pretrenowania, jak i od samej inferencji. Pretrenowanie buduje ogólną reprezentację języka na dużym, zróżnicowanym korpusie. Dostrajanie wykorzystuje mniejszy zbiór przykładów, zwykle przygotowany pod określoną funkcję modelu. Inferencja jest natomiast etapem użycia gotowego modelu: parametry pozostają stałe, a model oblicza odpowiedź dla nowego wejścia.

Dostrajanie stosuje się wtedy, gdy samo sformułowanie zapytania dla modelu nie daje wystarczająco stabilnego zachowania albo gdy model musi konsekwentnie używać określonego sposobu odpowiedzi. W zadaniach klasyfikacyjnych oznacza to powtarzalne przypisywanie tekstu do etykiet, a nie jedynie jednorazowe poproszenie modelu o ocenę przykładu. Ograniczeniem takiego treningu jest ryzyko, że przy małym albo wąskim tematycznie zbiorze model dopasuje się do cech danych treningowych zamiast do ogólnej reguły zadania [17, rozdz. 5.2]. Gdy problem wynika z artefaktów konkretnego korpusu, taki mechanizm można opisać jako uczenie skrótów, czyli wykorzystanie łatwych, przypadkowych cech danych zamiast właściwej zależności między tekstem i etykietą [18].

2.5.2. Instruction tuning

Dostrajanie instrukcyjne (ang. *instruction tuning*) jest formą nadzorowanego dostrajania na parach: polecenie zapisane w języku naturalnym i oczekiwana odpowiedź. Przykład ma strukturę zadania, dlatego trening uczy model rozpoznawania intencji polecenia oraz generowania odpowiedzi w przyjętej konwencji. W pracach nad rodziną FLAN pokazano ten schemat jako dostrajanie na zbiorze wielu zadań opisanych instrukcjami [19].

Celem dostrajania instrukcyjnego jest zamiana modelu kontynuującego tekst w model reagujący na polecenie użytkownika. Po samym pretrenowaniu model zna wiele wzorców językowych, ale odpowiedź na instrukcję może traktować jak zwykłą kontynuację dokumentu. Instruction tuning przesuwaa zachowanie modelu w stronę odpowiedzi powiązanych z instrukcją, utrzymanych w określonej roli i bardziej przewidywalnych dla użytkownika. Gotowe modele udostępniane jako warianty *instruct*, *chat* albo *assistant* zwykle są już po takim etapie. Dalsze dostrajanie nie uczy ich więc prowadzenia rozmowy od początku, lecz dopasowuje istniejące zachowanie instrukcyjne do konkretnego zadania.

W modelach czatowych ten etap zwykle realizuje się jako *supervised fine-tuning* (SFT) na rozmowach z rolami, takimi jak **system**, **user** i **assistant**. Komunikaty użytkownika oraz komunikat systemowy tworzą kontekst, a trenowana odpowiedź znajduje się po stronie asystenta. W podejściu takim jak InstructGPT punktem uczenia są przykłady oczekiwanego zachowania dla podanych promptów [12]. W praktycznych implementacjach SFT przekłada się to zwykle na trenowanie odpowiedzi asystenta w kontekście wcześniejszych komunikatów, a nie na bezwarunkowe kontynuowanie całego zapisu rozmowy.

Kontrola formatu pozostaje osobnym problemem. Model instrukcyjny może rozumieć polecenie, a mimo to odpowiedzieć pełnym zdaniem, dodać uzasadnienie albo użyć formatu innego niż oczekiwany. Dlatego w zadaniach wymagających krótkiej etykiety, obiektu JSON albo innej struktury odpowiedzi potrzebne bywa dalsze dostrajanie zadaniowe oraz osobna ewaluacja sposobu odczytywania wyniku.

2.5.3. Parameter-efficient fine-tuning

Pełne dostrajanie aktualizuje wszystkie parametry modelu. Przy małych modelach może być to akceptowalne, ale wraz ze wzrostem liczby parametrów rosną wymagania pamięciowe, czas treningu oraz koszt przechowywania kolejnych wersji modelu. W praktyce problemem jest nie tylko sam trening. Każdy wariant dostrojenia może wymagać osobnej kopii wag, co szybko utrudnia porównywanie konfiguracji.

Parameter-efficient fine-tuning (PEFT) ogranicza ten koszt przez trenowanie tylko niewielkiej części parametrów albo przez dodanie małych modułów adaptacyjnych do zamrożonego modelu bazowego. Taki kierunek rozwijano między innymi w metodach adapterowych [20]. Przeglądy PEFT opisują wiele wariantów tego podejścia. Różnią się konstrukcją, ale sprwadzają adaptację modelu do trenowania małego fragmentu architektury [21].

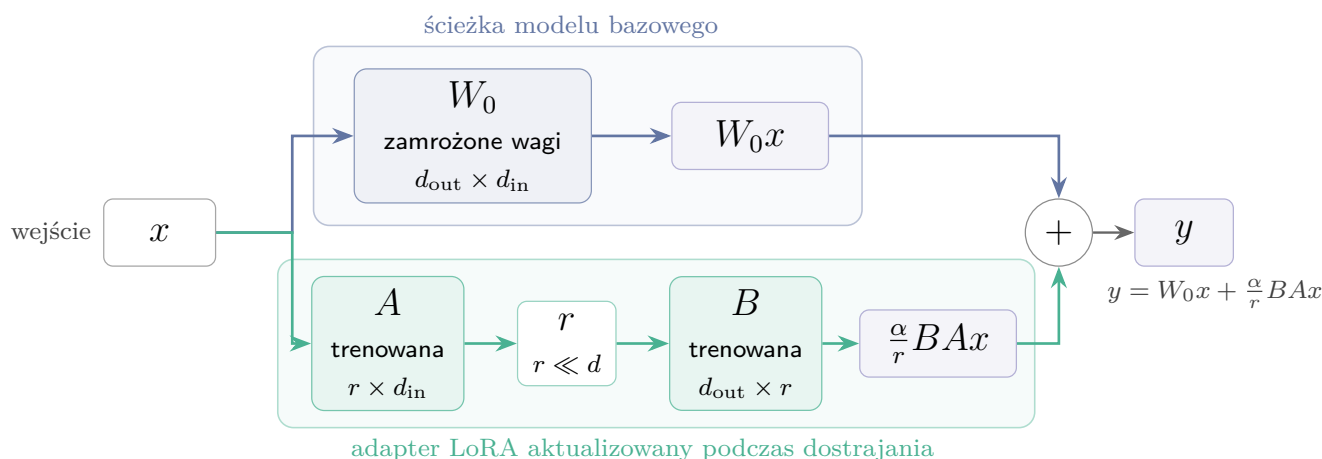
Przy porównywaniu wielu wariantów PEFT ma prostą zaletę organizacyjną. Model bazowy pozostaje jeden, a kolejne wersje można zapisywać jako małe zestawy parametrów. Ułatwia to powtarzanie treningów i przechowywanie punktów kontrolnych.

2.5.4. LoRA

LoRA (ang. *Low-Rank Adaptation*) należy do metod PEFT. Wagi modelu bazowego pozostają zamrożone, a trening obejmuje dodatkową, niskorzędową poprawkę dla wybranych warstw liniowych [22]. Zamiast aktualizować macierz wag W , metoda uczy zmianę ΔW , którą można zapisać jako iloczyn dwóch mniejszych macierzy:

$$W' = W + \Delta W, \quad \Delta W = \frac{\alpha}{r} B A.$$

Sposób dołączenia trenowanej poprawki do zamrożonej macierzy bazowej pokazano na rysunku 2.5.



Rysunek 2.5.: Uproszczony schemat adaptera LoRA. Macierz bazowa pozostaje zamrożona, a podczas dostrajania aktualizowane są tylko dwie małe macierze niskiego rzędu.

Źródło: opracowanie własne.

Parametr r określa rząd adaptacji, czyli rozmiar wewnętrznego wymiaru dodatkowych macierzy. Większa wartość daje adapterowi więcej swobody, ale zwiększa liczbę trenowanych parametrów. Parametr α skaluje wpływ adaptera na wynik warstwy. W praktycznych konfiguracjach pojawia się także dropout LoRA, który losowo wygasa część aktywacji adaptera podczas treningu i działa jako forma regularyzacji.

Po zakończeniu treningu adapter LoRA można przechowywać oddzielnie od modelu bazowego. Dzięki temu wiele wariantów treningu nie wymaga zapisywania pełnych kopii modelu. W razie potrzeby adapter może zostać załadowany razem z modelem bazowym albo scalony z jego wagami.

2.5.5. Punkty kontrolne i przeuczenie

Punkt kontrolny (ang. *checkpoint*) jest zapisanym stanem treningu z określonego momentu. W zależności od konfiguracji może obejmować wagi modelu lub adaptera, stan optymalizatora i dodatkowe metadane potrzebne do wznowienia pracy. W przypadku metod adapterowych najważniejszy jest zwykle stan adaptera, ponieważ model bazowy pozostaje niezmieniony.

Analiza punktów kontrolnych pomaga odróżnić samo dopasowanie do zbioru treningowego od generalizacji. Model może poprawiać wynik na przykładach treningowych, a jednocześnie tracić jakość na danych walidacyjnych lub na zbiorach o innej charakterystyce. W takim przypadku końcowy stan treningu nie musi być najlepszym wyborem. Lepszy może okazać się wcześniejszy punkt kontrolny, zapisany przed nasileniem przeuczenia.

Dlatego przy dostrajaniu modeli językowych porównuje się wyniki w czasie, a nie tylko ostatnią zapisaną wersję. Dotyczy to zwłaszcza małych zbiorów danych i zadań, w których źródła przykładów różnią się stylem, formatem lub rozkładem klas.

2.6. Modele językowe jako klasyfikatory

Ten sam model językowy można wykorzystać do klasyfikacji na kilka sposobów. W schemacie najbliższym klasycznej klasyfikacji tekstu do reprezentacji sekwencji dodaje się głowicę, która zwraca etykietę. Wynik ma wtedy postać decyzji jednej z klas. Model nie generuje odpowiedzi tekstowej, ale nadal korzysta z reprezentacji wyuczonych podczas wcześniejszego treningu językowego.

W modelach generatywnych klasyfikację można zapisać jako krótką instrukcję. Model otrzymuje treść wiadomości i dopisuje etykietę, na przykład **spam** albo **ham**. Taki zapis pasuje do modeli instrukcyjnych, jednak wynik trzeba później odczytać z wygenerowanego tekstu. Jeżeli model dopisze komentarz, wybierze inny format albo zwróci dłuższą odpowiedź, o wyniku zaczyna decydować również parser.

Można też zrezygnować z generowania odpowiedzi i porównać prawdopodobieństwa tokenów odpowiadających etykietom. Decyzja wynika wtedy z rozkładu następnego tokenu dla możliwych klas. Ten wariant ogranicza problem parsowania, ale jest wrażliwy na zapis etykiet i sposób tokenizacji. Bardziej kontrolowaną odmianą generowania jest odpowiedź ustrukturyzowana, na przykład obiekt z polem przechowującym etykietę. Ułatwia to późniejsze przetwarzanie, choć nadal wymaga sprawdzenia, czy model zachował oczekiwany format.

Te podejścia różnią się sposobem treningu, kosztem inferencji i miejscem, w którym może pojawić się błąd: w głowicy klasyfikacyjnej, w generowaniu tekstu albo w parserze. Dlatego w części badawczej potraktowano je jako osobne metody użycia modelu językowego do klasyfikacji.

2.7. Modele z otwartymi wagami

2.7.1. Otwarte wagi i możliwość uruchomienia lokalnego

Sposób udostępnienia modelu wpływa na to, czy eksperyment da się później odtworzyć. W przypadku modeli zamkniętych użytkownik korzysta zwykle z gotowej usługi, na przykład przez API. Taki tryb jest wygodny w integracjach, ale ogranicza kontrolę nad wersją modelu i środowiskiem uruchomieniowym. W badaniu wynik może wtedy zależeć od limitów usługi, zmian po stronie dostawcy albo wersji modelu, której nie da się samodzielnie zamrozić w repozytorium.

W modelach z otwartymi wagami publicznie dostępne są pliki parametrów. Model można uruchomić lokalnie albo na własnym serwerze, jeśli pozwala na to licencja. To ona określa zakres swobody, dlatego samo opublikowanie plików nie oznacza jeszcze dowolnego użycia modelu. W tej pracy otwarte wagi są potrzebne głównie po to, aby kontrolować wersję modelu, środowisko, sposób dostrajania i zapis adapterów. Zmniejsza to zależność od zewnętrznej usługi.

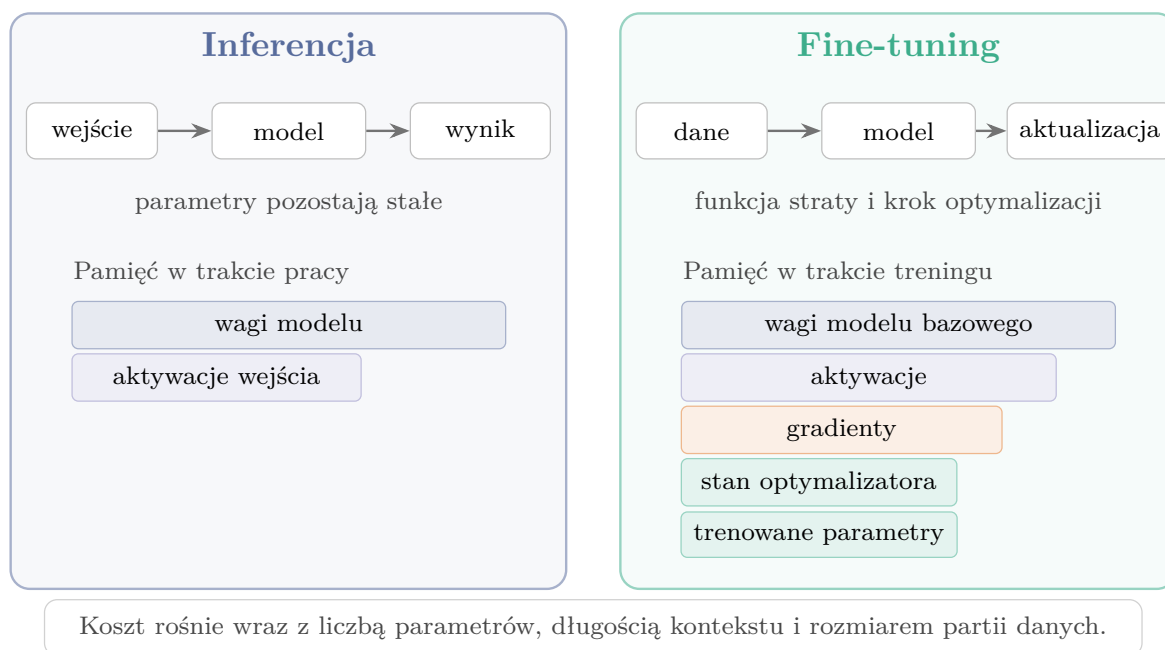
2.7.2. Rozmiar modelu, koszt treningu i inferencji

Liczba parametrów przekłada się na wymagania pamięciowe. Większy model potrzebuje więcej pamięci na wagi, a podczas treningu dochodzą aktywacje, gradienty i stan optymalizatora. Dostrajanie adapterowe ogranicza część aktualizowanych parametrów, lecz model bazowy nadal musi być załadowany w pamięci urządzenia. Znaczenie ma także długość kontekstu i rozmiar partii danych (ang. *batch size*), czyli liczba przykładów przetwarzanych w jednym kroku.

Różnica jest widoczna już w prostym rachunku pamięci. Same wagi zapisane w FP16 albo BF16 zajmują około 2 bajty na parametr, więc model o 0,6 mld parametrów potrzebuje około 1,2 GB pamięci na same parametry. Przy pełnym treningu z optymalizatorem Adam lub AdamW dochodzą gradienty, kopia wag w FP32 oraz dwa stany optymalizatora. W typowym treningu mieszanej precyzji daje to około 16 bajtów na parametr, czyli w przybliżeniu osiem razy więcej niż same wagi w FP16/BF16 [23, 24]. Jest to przybliżenie dla wariantu optymalizatora przechowującego dwie wartości stanu i kopię parametrów w wyższej precyzji; dokładny koszt zależy od implementacji, precyzji oraz zastosowanych technik oszczędzania pamięci. Dla modelu 0,6B jest to rząd wielkości od 9 do 10 GB jeszcze przed doliczeniem

aktywacji, narzutu biblioteki i pamięci zależnej od długości sekwencji oraz rozmiaru partii danych. LoRA zmniejsza ten koszt, ponieważ gradienty i stan optymalizatora dotyczą głównie adapterów, ale model bazowy nadal musi być obecny w pamięci [22].

Rysunek 2.6 zestawia te elementy dla inferencji i dostrajania.



Rysunek 2.6.: Porównanie składników kosztu inferencji i dostrajania modelu. Schemat jakościowy.

Źródło: opracowanie własne.

Podczas inferencji nie przechowuje się gradientów ani stanu optymalizatora. Pozostaje jednak pamięć na wagi i obliczenia wykonywane dla każdego wejścia. Z tego powodu mały model może lepiej pasować do praktycznego klasyfikatora nawet wtedy, gdy w ogólnych benchmarkach wypada słabiej: łatwiej uruchomić go na tańszym serwerze, szybciej powtarzać eksperymenty i przechowywać kolejne adaptory. W tej pracy rozmiar modelu jest traktowany głównie jako ograniczenie praktyczne.

3. Przegląd metod wykrywania niechcianej poczty

Praktyczny filtr pocztowy korzysta nie tylko z tematu i treści wiadomości, ale też z nagłówek SMTP, adresów IP, domen, historii nadawcy, wyników uwierzytelniania i informacji o odsyłaczach. Analiza tekstu jest jedną z jego warstw. Pozostałe mechanizmy oceniają nadawcę, wykrywają znane kampanie albo sprawdzają podejrzane adresy URL. W przeglądzie uwzględniono zarówno te rozwiązania, jak i metody możliwe do uruchomienia na zbiorach tekstowych użytych w części badawczej.

3.1. Wielowarstwowe systemy filtrowania poczty

W produkcyjnych skrzynkach pocztowych pierwsza decyzja często zapada jeszcze przed analizą pełnej treści wiadomości. Dostawca poczty może sprawdzić, czy nadawca spełnia wymagania techniczne, czy wiadomość przechodzi uwierzytelnianie SPF, DKIM i DMARC, czy adres IP ma dobrą reputację oraz czy domena nie występuje w kampaniach nadużyć. Wymagania publikowane dla nadawców poczty Gmail wskazują wprost na znaczenie SPF lub DKIM, polityki DMARC, poprawnego formatu wiadomości i utrzymywania niskiego odsetka zgłoszeń spamu [25]. Podobny obraz widać w ustawieniach Microsoft Defender for Office 365, gdzie osobno konfigurowane są między innymi polityki antyspamowe, antyphishingowe, obsługa podszywania się pod nadawcę i reakcje na DMARC [26].

SpamAssassin opiera decyzję na zestawie testów punktowanych, regułach treści, listach dozwolonych i blokowanych, testach sieciowych oraz mechanizmach uczących się [27]. Rspamd udostępnia osobne moduły dla SPF, DKIM, DMARC, reputacji, list DNS, fuzzy hashy, phishingu, sieci neuronowych i wielu innych sygnałów [28]. Oba filtry działają jako systemy punktowe, które składają wiele sygnałów w jedną decyzję.

3.2. Metody regułowe, reputacyjne i uwierzytelnianie nadawcy

W filtrze pocztowym reguła jest jawnie zapisanym warunkiem sprawdzanym na wiadomości lub jej metadanych. Po spełnieniu warunku filtr może dodać określoną liczbę punktów

do wyniku spamu, zablokować wiadomość albo przekazać ją do dalszej analizy. Reguły są nadal używane, ponieważ działają szybko, są interpretowalne i pozwalają reagować na znane schematy nadużyć bez ponownego trenowania modelu. Reguła może dotyczyć fragmentu nagłówka, nietypowego adresu URL, słowa często występującego w danej kampanii albo kombinacji kilku prostszych testów. Administrator może też podnieść lub obniżyć wagę konkretnego testu, co ułatwia bezpośrednią kontrolę nad polityką filtrowania.

Słabą stroną reguł jest zależność od wcześniej zauważonych wzorców. Nadawcy spamu mogą zmieniać pisownię słów, przenosić treść do obrazów, rotować domeny lub modyfikować szablony wiadomości. Dlatego reguły zwykle nie występują same, lecz są łączone z reputacją nadawcy, listami domen, uwierzytelnianiem i klasyfikatorami treści. Przeglądy metod antyphishingowych również pokazują, że obrona przed phishingiem obejmuje kilka rodzin podejść: listy znanych adresów, heurystyki, analizę treści, analizę wizualną oraz metody uczenia maszynowego [29].

Uwierzytelnianie nadawcy rozwiązuje inny problem niż klasyfikacja tekstu. Mechanizmy SPF, DKIM i DMARC pomagają ustalić, czy wiadomość mogła zostać wysłana w imieniu deklarowanej domeny. Poprawnie uwierzytelniona wiadomość może być niechcianą reklamą albo pochodzić z przejętego konta, dlatego analiza treści pozostaje potrzebna.

3.3. Klasyczne metody uczenia maszynowego

Klasyczne modele uczenia maszynowego oparte na cechach leksykalnych są najprostszym porównaniem dla klasyfikacji treści. Wiadomość zamienia się wtedy na wektor cech, na przykład przez zliczanie słów, n-gramy albo TF-IDF, a następnie trenuje klasyfikator taki jak Naive Bayes, regresja logistyczna lub SVM. Jest to stosunkowo proste podejście, które zapewnia tani trening i szybką inferencję, a przy odpowiednio dobranych danych może osiągać dobre wyniki.

Bayesowskie filtrowanie spamu było badane już we wczesnych pracach nad automatycznym filtrowaniem poczty. Androutsopoulos i współautorzy porównali Naive Bayes z podejściem pamięciowym i pokazali, że automatycznie uczone filtry mogą wyraźnie przewyższać ręcznie konstruowane reguły słów kluczowych [30]. Nowsze prace nadal porównują z nimi nowe rozwiązania. Janez-Martino i współautorzy porównali między innymi reprezentacje TF-IDF i bag-of-words z klasyfikatorami Naive Bayes, drzewami decyzyjnymi oraz SVM w zadaniu klasyfikacji wiadomości spamowych [31].

Porównanie z klasycznymi metodami pozwala ocenić, ile z wyniku można uzyskać dzięki dopasowaniu do słownictwa zbioru treningowego. Wynik TF-IDF bliski wynikowi modelu językowego podważałby zasadność użycia droższego rozwiązania. Przewaga modelu językowego na danych z innych źródeł wskazywałaby natomiast na lepszą generalizację.

Reprezentacja leksykalna dobrze wychwytuje powtarzalne zwroty, nazwy produktów, typowe frazy oszustw i często występujące domeny. Gorzej radzi sobie z parafrazą, długim

kontekstem i wiadomościami, w których znaczenie wynika z relacji między kilkoma zdaniami, ponieważ warianty typu bag-of-words tracą kolejność słów i słabiej opisują zależności semantyczne [32]. Rzadkie cechy oraz krótka treść wiadomości mogą dodatkowo prowadzić do przeuczenia i słabszej generalizacji [33]. W danych e-mailowych oznacza to ryzyko dopasowania się do cech konkretnego korpusu, na przykład charakterystycznego formatowania albo powtarzalnych fragmentów stopki.

3.4. Szybkie modele tekstowe i reprezentacje semantyczne

Pomiędzy klasycznym TF-IDF a pełnym dostrajaniem transformera znajdują się metody pośrednie. Jedną z nich jest **fastText**, który uczy reprezentacje słów z n-gramów znakowych i stosuje liniowy klasyfikator. Reprezentacje słów występujących w dokumencie są łączone przed wykonaniem klasyfikacji. Autorzy **fastText** pokazali, że taki model może być konkurencyjny wobec głębszych klasyfikatorów tekstu, a jednocześnie trenuje się i działa bardzo szybko [34]. W eksperymentach użyto klasyfikatora nadzorowanego z n-gramami znakowymi długości 3–6, unigramami słów, wektorami o wymiarze 100, współczynnikiem uczenia 0,25 i 20 epokami treningu.

Drugim wariantem pośrednim jest użycie gotowego modelu embeddingowego do zamiany wiadomości na wektor semantyczny, a następnie wytrenowanie prostego klasyfikatora liniowego. Sentence-BERT został zaproponowany właśnie po to, aby uzyskiwać reprezentacje zdań nadające się do porównań semantycznych i dalszych zadań bez kosztownego porównywania każdej pary tekstów przez pełny model BERT [35]. W eksperymentach sprawdzono ten wariant za pomocą MiniLM i regresji logistycznej.

Słabszym punktem takiego wariantu jest brak treningu pod konkretny typ nadużyć. Gotowy model embeddingowy może dobrze oddawać ogólne znaczenie tekstu, ale słabiej reagować na drobne sygnały istotne dla filtra pocztowego: nietypowe domeny, zapis kwot, celowe literówki albo powtarzalne elementy szablonów kampanii.

3.5. Modele transformerowe i językowe

Modele transformerowe budują reprezentację zależną od kontekstu. W wariantcie klasyfikacyjnym model, taki jak BERT lub DistilBERT, otrzymuje tekst wiadomości i zwraca etykietę przez głowicę klasyfikacyjną. DistilBERT jest mniejszą wersją BERT-a uzyskaną przez destylację wiedzy; według autorów zachowuje dużą część jakości modelu bazowego, przy mniejszym rozmiarze i szybszym działaniu [36].

Modele językowe wymagają więcej pamięci niż metody TF-IDF, a ich inferencja jest wolniejsza i zależy od długości kontekstu. Mogą jednak lepiej wykorzystywać kontekst i uogólniać poza dosłowne słownictwo zbioru treningowego. Dlatego w części badawczej oceniono

również ich profil błędów na danych spoza zbioru treningowego.

Nowsze prace sprawdzają także modele językowe bez dostrajania. Si i współautorzy porównali klasyfikację wykonywaną przez ChatGPT z Naive Bayes, SVM, regresją logistyczną, siecią feed-forward i BERT-em. Na dużym zbiorze anglojęzycznym metody nadzorowane uzyskały lepsze wyniki, natomiast ChatGPT był bardziej konkurencyjny dla mniejszego zbioru chińskojęzycznego [37]. Rojas-Galeano ocenił ChatGPT, GPT-4 i Flan-T5 na korpusie SpamAssassin. Flan-T5 uzyskał F1 równe 90% przy klasyfikacji skróconej treści, a GPT-4 95% po wcześniejszym streszczeniu wiadomości. Autor zaznaczył jednocześnie ograniczenie badania do jednego korpusu oraz koszt inferencji większych modeli [38]. Wyniki te pokazują, że sam prompt może wystarczyć w części warunków, lecz nie zastępuje oceny na kilku źródłach ani analizy kosztu działania.

Przywołane badania nie porównują kilku sposobów dostrojenia tego samego małego modelu z otwartymi wagami. W tej pracy sprawdzono cztery sformułowania zadania klasyfikacji dla Qwen3-0.6B, a uzyskane wyniki zestawiono ze zużyciem zasobów i przepustowością prostszych klasyfikatorów.

3.6. Zakres porównania eksperymentalnego

Bezpośrednie porównanie objęło metody korzystające z treści wiadomości. Wiadomości przypisywano do klasy `ham` albo `spam`. Wykorzystane zbiory nie zawierają spójnych danych o reputacji nadawcy, DNS, pełnych nagłówkach ani historii ruchu. Tabela 3.1 wskazuje, które metody omówiono jako kontekst literaturowy, a które uruchomiono w eksperymentach.

3. Przegląd metod wykrywania niechcianej poczty

Grupa metod	Główne źródło sygnału	Wymaga metadanych poczty	Ujęcie w eksperymentach
Uwierzytelnianie i reputacja nadawcy	Nagłówki SMTP, DNS, SPF, DKIM, DMARC, reputacja adresów IP i domen	Tak	Nie
Systemy regułowe i reputacyjne	Reguły treści, adresy URL, listy reputacyjne, scoring, czasem filtr bayesowski	Częściowo	Nie
TF-IDF + klasyfikator	Treść wiadomości reprezentowana przez cechy leksykalne	Nie	Tak
fastText	N-gramy słów i szybki model liniowy z embeddingami	Nie	Tak
Embeddingi zdań + klasyfikator	Wektor semantyczny wiadomości i klasyfikator liniowy	Nie	Tak
Transformer klasyfikacyjny	Dostrajany enkoder tekstu z głowicą klasyfikacyjną	Nie	Tak
Mały model językowy z LoRA	Dostrojony model generatywny, etykieta lub prawdopodobieństwo etykiety	Nie	Główna metoda opisywana w pracy

Tabela 3.1.: Zakres metod omawianych w przeglądzie oraz ich wykorzystanie w części eksperymentalnej.

4. Część badawcza

Badania obejmowały wybór modelu bazowego, przygotowanie danych, strojenie parametrów i porównanie metod dostrajania. Najlepszy wariant porównano następnie z innymi klasyfikatorami trenowanymi i testowanymi na tych samych danych. Oprócz skuteczności sprawdzono też czas i zasoby potrzebne podczas inferencji.

4.1. Wybór modelu językowego z otwartymi wagami

Model bazowy dobrano z myślą o eksperymentach, które trzeba było wielokrotnie powtarzać. Zbyt duży model podnosiłby koszt każdego treningu, wydłużał ewaluację i zwiększał rozmiar zapisywanych punktów kontrolnych. Z tego powodu przegląd ograniczono do małych rodzin modeli dostępnych jako otwarte wagi, zgodnych z narzędziami używanymi później w pracy, takimi jak `transformers`, `peft`, `vLLM` albo `llama.cpp`. Przyjęto też praktyczne założenie, że klasyfikator powinien działać na serwerze z jedną kartą GPU, a po kwantyzacji również w słabszym środowisku.

Porównanie obejmuje pięć rodzin: Qwen3, Gemma 3, Llama 3.2, SmolLM2 oraz Phi-3. Tabela 4.1 zestawia rozważane warianty, warunki licencyjne i cechy istotne dla tej pracy. Oznaczenia M i B odnoszą się odpowiednio do milionów oraz miliardów parametrów modelu.

Rodzina	Parametry wariantów	Licencja / dostęp	Cechy techniczne	Znaczenie dla tej pracy
Qwen3	0,6B	Apache 2.0	32k tokenów kontekstu, wsparcie ponad 100 języków, tryb odpowiedzi szybkiej i tryb rozumowania [39, 13].	Mały wariant o przyjętej w pracy pojemności, z wygodnym wsparciem standardowych narzędzi.
Gemma 3	270M / 1B	otwarte wagi, licencja Gemma	Bardzo małe warianty, kontekst 32k tokenów dla modeli 270M i 1B, integracja z ekosystemem Google [40].	Niski koszt inferencji, ale mniej neutralne warunki licencyjne niż w przypadku Apache 2.0.
Llama 3.2	1B / 3B	Llama 3.2 Community License	Warianty tekstowe 1B i 3B, kontekst 128k tokenów, szerokie wsparcie narzędziowe rodziny Llama [41].	Dobrze wspierana rodzina modeli, lecz najmniejszy wariant jest większy od Qwen3-0.6B, a licencja jest niestandardowa.
SmolLM2	135M / 360M / 1,7B	Apache 2.0	Rodzina projektowana z myślą o małych modelach i zastosowaniach lokalnych; wariant 1,7B trenowany na 11 bilionach tokenów [42].	Wariant 1,7B daje wygodną bazę lokalną, ale jest prawie trzykrotnie większy od Qwen3-0.6B.
Phi-3	3,8B	otwarte wagi, licencja MIT	Model Phi-3-mini ma 3,8B parametrów i był projektowany jako mały model o wysokiej jakości ogólnej [43, 44].	Wysoka jakość ogólna okupiona większym kosztem treningu, inferencji i przechowywania punktów kontrolnych.

Tabela 4.1.: Porównanie małych rodzin modeli językowych rozważanych jako baza dla klasyfikatora.

Do eksperymentów wybrano model Qwen/Qwen3-0.6B. Był mniejszy od rozważanych wariantów Llama 3.2, Phi-3 oraz SmolLM2 1,7B, a zarazem większy od najmniejszych modeli Gemma 3 i SmolLM2. O wyborze zdecydowały przede wszystkim koszt wielokrotnego dostrajania, licencja Apache 2.0, kontekst 32k tokenów, obsługa formatu czatu oraz zgodność z bibliotekami używanymi w eksperymentach. Najmniejszych wariantów nie wybrano ze względu na przyjęte ryzyko ograniczonej pojemności i chęć zachowania marginesu jakości; hipotezy tej nie zweryfikowano eksperymentalnie.

4.2. Dane i metodologia badawcza

4.2.1. Analiza wykorzystanych zbiorów danych

Wybór i przygotowanie danych może mieć równie duży wpływ na końcowy wynik jak architektura modelu lub parametry treningu.

Pierwszym etapem eksperymentów było utworzenie zbioru wiadomości elektronicznych przeznaczonego do zadania binarnej klasyfikacji. Wiadomości oznaczano jako pożądane (*ham*, klasa negatywna) albo niepożądane (*spam*, klasa pozytywna). Zachowano etykiety nadane w zbiorach źródłowych i nie stosowano własnego kryterium oceny wiadomości marketingowych. Wiadomość marketingowa trafiała do klasy *spam* wyłącznie wtedy, gdy tak oznaczono ją w źródle. Szczegółowe mapowanie etykiet opisano w podsekcji dotyczącej normalizacji schematu danych.

Publiczne zbiory danych różnią się liczbą rekordów, sposobem zapisu wiadomości, semantyką etykiet, obecnością metadanych oraz formatem treści. Analiza obejmuje więc pochodzenie danych, rozkład klas, długość wiadomości, obecność duplikatów oraz potencjalne ryzyko przecieku informacji między częścią treningową i testową.

W pracy przyjęto, że wszystkie zbiory są sprowadzane do wspólnego schematu rekordu: `subject`, `body`, `label` oraz `source`. Pole `subject` przechowuje temat wiadomości, `body` jej treść, `label` wartość liczbową klasy, a `source` nazwę zbioru źródłowego. Taki zapis pozwala porównywać zbiory o różnej strukturze bez opierania się na metadanych, które nie występują we wszystkich źródłach.

Charakterystyka poszczególnych źródeł

ENRON

Zbiór *enron* jest największym z wybranych źródeł danych. Zawiera 517 401 wiadomości, którym w użytym źródle przypisano klasę negatywną. Jego zaletą jest duża liczność i naturalny charakter korespondencji, jednak jednocześnie jest to zbiór specyficzny domenowo. Pochodzi z archiwum organizacyjnego, dlatego może zawierać wiele powtarzalnych struktur, tematów, podpisów, cytowań oraz wątków wewnętrznych. Włączenie go w całości do

zbioru treningowego nieproporcjonalnie zwiększałoby liczbę przykładów korespondencji pożądaney, ale niekoniecznie poprawiłoby zdolność modelu do ogólnego rozpoznawania takich wiadomości. Dlatego zbiór pominięto w treningu i wykorzystano jako zewnętrzny materiał do oceny fałszywych alarmów. Zbiór został potraktowany jako przybliżenie klasy **ham**: archiwum może zawierać reklamy, wiadomości automatyczne lub inne przykłady niepożądane. Ręczny przegląd 100 wiadomości wylosowanych z ziarnem 67 ujawnił dwie wyraźne reklamy oraz kilka wiadomości automatycznych i łańcuskowych. Wynik na tym zbiorze należy więc interpretować z uwzględnieniem możliwego szumu etykiet.

LING

Zbiór **ling** obejmuje 2 893 rekordy: 2 412 wiadomości poświadanych oraz 481 przykładów klasy pozytywnej. Jest jednym z mniejszych zbiorów, posiada format z osobnym tematem, treścią i etykietą.

NAZARIO

Zbiór **nazario** obejmuje 3 065 rekordów i jest prawie zbalansowany: 1 500 wiadomości poświadanych oraz 1 565 przykładów klasy pozytywnej. Posiada pola tematu, treści oraz dodatkowych metadanych, takich jak nadawca, odbiorca, data i adresy URL.

TREC_2007

Zbiór **trec_2007** zawiera 75 419 wiadomości, z czego 50 199 należy do klasy pozytywnej, a 25 220 do klasy negatywnej. Rozkład klas jest więc przesunięty w kierunku spamu, który stanowi około 66,6% rekordów.

CEAS_2008

Zbiór **ceas_2008** zawiera 39 154 wiadomości i również ma przewagę klasy pozytywnej, ale jest ona mniejsza niż w **trec_2007**. W zbiorze znajduje się 21 842 wiadomości spamowych oraz 17 312 poświadanych. Pola dostępne dla tego zbioru to temat, treść, nadawca, odbiorca, data i adresy URL.

FRAUDULENT_EMAIL_CORPUS

Zbiór **fraudulent_email_corpus** obejmuje wiadomości oszukańcze, w tym oszustwa typu *advance-fee fraud*, nazywane też oszustwami nigeryjskimi. Jest jednoklasowy i zawiera 3 978 wiadomości traktowanych jako klasa pozytywna. Dane te są wartościowe z punktu widzenia różnorodności spamu, ponieważ wiadomości oszukańcze mogą różnić się od typowych reklam farmaceutycznych, newsletterów czy masowego phishingu.

SPAM_ASSASSIN I SPAM_HAM

Zbiory **spam_assassin** i **spam_ham** są mniejsze, ale dobrze pasują do klasycznego zadania klasyfikacji spam/ham. **spam_assassin** zawiera 5 796 rekordów, a **spam_ham** 5 171. W obu przypadkach klasa pozytywna jest mniejszością. Reprezentują inny format i historię przygotowania danych niż zbiory CEAS i TREC użyte w treningu.

Porównanie zbiorów

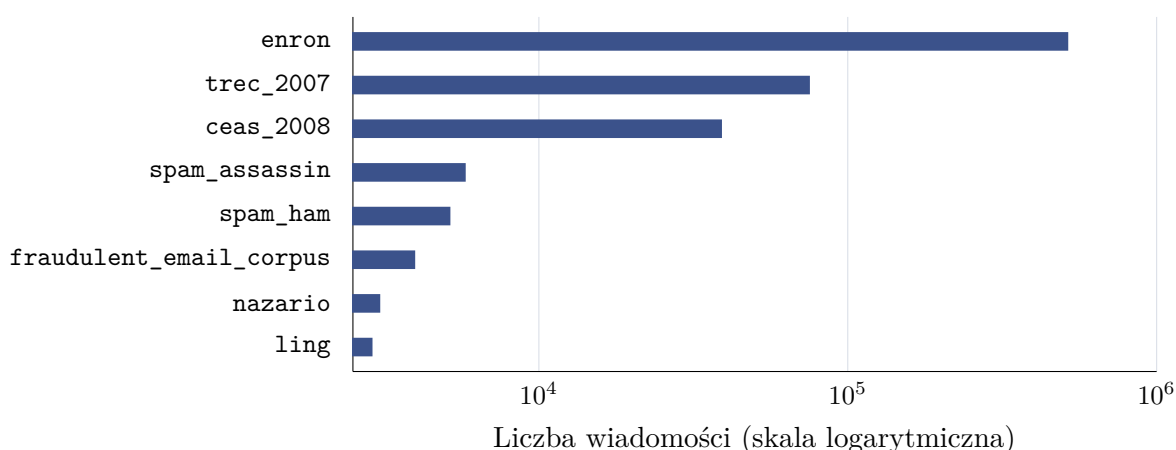
Do eksperymentów wykorzystano zbiory wymienione w tabeli 4.2. W tabeli podano licznosc każdego źródła, rozkład klas oraz krótką informację o charakterze danych. Ten podział jest istotny, ponieważ część zbiorów służyła do treningu, a część pozostawiono do zewnętrznej walidacji.

Tabela 4.2.: Charakterystyka zbiorów danych

Zbiór	Wiersze	Ham	Spam	Spam [%]	Uwagi
enron [45]	517 401	517 401	0	0,0	zbiór wiadomości pożądaných, traktowany jako klasa negatywna
trec_2007 [46]	75 419	25 220	50 199	66,6	zbiór TREC 2007
ceas_2008 [47]	39 154	17 312	21 842	55,8	dane CEAS 2008
spam_assassin [48]	5 796	3 900	1 896	32,7	
spam_ham [49]	5 171	3 672	1 499	29,0	
fraudulent_email [50] _corpus	3 978	0	3 978	100,0	zbiór wiadomości oszukańczych, w tym oszustw typu <i>advance-fee fraud</i> [51]
nazario [52]	3 065	1 500	1 565	51,1	zbiór wiadomości phishingowych
ling [53]	2 893	2 412	481	16,6	

Łącznie analizowane zbiory surowe obejmują 652 877 wiadomości. Nie wszystkie źródła zostały jednak bezpośrednio połączone w jeden zbiór treningowy. Największy zbiór, **enron**, zawiera wyłącznie wiadomości klasy negatywnej. Z kolei **fraudulent_email_corpus** zawiera wyłącznie przykłady klasy pozytywnej. Bezpośrednie wykorzystanie wszystkich danych mogłoby oznaczać, że model zamiast uczyć się różnic między treścią spamu i poczty pożądaney nauczyłby się rozpoznawać pochodzenie przykładu. Przykładowo, jeśli większość wiadomości pożądaných pochodziłaby z jednego archiwum firmowego, a większość wiadomości niepożądaných z innego publicznego zbioru, klasyfikator mógłby wykorzystywać artefakty konkretnego źródła danych, a nie ogólne cechy spamu. Taki problem jest szczególnie niebezpieczny w eksperymentach z dużymi modelami językowymi, ponieważ modele te potrafią wychwytywać subtelne i nieintencjonalne wzorce formatowania [18].

Rysunek 4.1 pokazuje skalę różnic między zbiorami. Połączenie wszystkich danych bez próbkowania prowadziłoby do dominacji zbioru **enron**. Taka dominacja byłaby niepożądana nie tylko ze względu na nierównowagę klas, lecz także ze względu na pochodzenie większości danych *ham* z jednego źródła.



Rysunek 4.1.: Porównanie liczności dostępnych surowych zbiorów danych. Skala pozioma jest logarytmiczna.

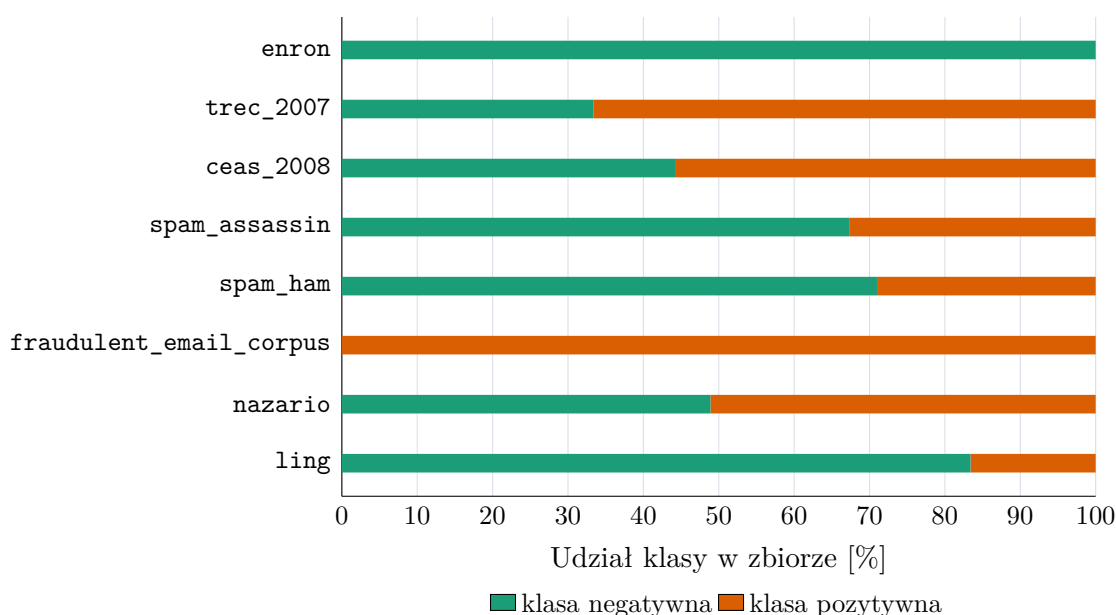
Źródło: opracowanie własne.

Rysunek 4.2 przedstawia rozkład klas w źródłowych zbiorach. Widoczne są trzy typy zbiorów. Pierwszą grupę tworzą zbiory względnie zbalansowane, takie jak **nazario** oraz **ceas_2008**. Drugą grupę tworzą zbiory o wyraźnej przewadze jednej z klas, na przykład **trec_2007**, **ling**, **spam_assassin** i **spam_ham**. Trzecią grupę stanowią zbiory jednoklasowe: **enron** oraz **fraudulent_email_corpus**. Dla modelowania klasyfikacyjnego najbezpieczniejsze są zbiory, które zawierają obie klasy i w których rozkład klas nie jest skrajnie zaburzony. Zbiory jednoklasowe mogą być użyteczne jako materiał pomocniczy, ale wymagają ostrożnego łączenia z innymi źródłami, aby nie stworzyć sztucznego zadania rozpoznawania źródła zamiast rozpoznawania spamu.

Wybór głównego zbioru do eksperymentów

Podstawą podziału danych w eksperymentach był wariant zbudowany z połączenia wszystkich źródeł poza **enron**, **fraudulent_email_corpus** oraz **spam_ham**. Zbiory celowo zostały wykluczone z treningów. **enron** pomijano ze względu na bardzo dużą licznosc i przyjęty jednoklasowy charakter. Jego włączenie mogłoby zdominować klasę negatywną oraz sprawić, że model uczyłby się specyfiki wewnętrznej korespondencji jednej organizacji zamiast ogólnych cech wiadomości pożądaných. Ponieważ **fraudulent_email_corpus** jest jednoklasowy, nie włączono go do treningu i wykorzystano do osobnej oceny wykrywania wiadomości oszukańczych. **spam_ham** był traktowany podobnie: jako prosty, zewnętrzny punkt odniesienia dla klasycznego zadania spam/ham.

Po zbalansowaniu najczęściej używany wariant treningowy zawierał 88 970 wiadomości: 44 485 przykładów klasy negatywnej i 44 485 przykładów klasy pozytywnej. Zbiór powstał po



Rysunek 4.2.: Rozkład klas w surowych zbiorach. Klasa pozytywna oznacza spam, phishing lub wiadomość oszukańczą.

Źródło: opracowanie własne.

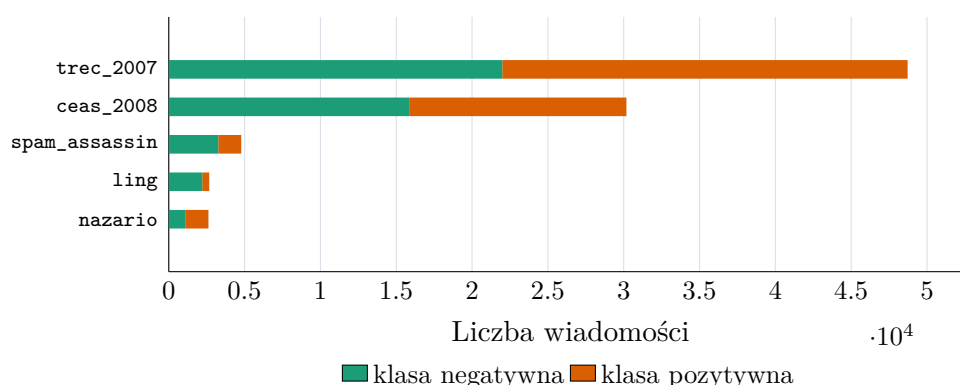
usunięciu pustych rekordów, zastosowaniu agresywnej deduplikacji treści oraz próbkowaniu do proporcji 50/50. W składzie źródłowym dominowały `trec_2007` i `ceas_2008`, uzupełnione o `spam_assassin`, `ling` oraz `nazario`.

Rysunek 4.3 przedstawia skład źródłowy głównego wariantu danych przed podziałem. Balans klas nie oznacza tu balansu źródeł: najwięcej rekordów pochodzi z `trec_2007`. Pole `source` zachowano w danych analitycznych, aby po treningu sprawdzać wyniki oddzielnie dla poszczególnych zbiorów. Duża różnica skuteczności między źródłami oznaczałaby ograniczoną odporność na zmianę dystrybucji danych.

Zbiór treningowy

Podział wykonano ze stratyfikacją względem etykiety i z ziarnem losowym równym 67. Zbiór treningowy służył do aktualizacji parametrów modelu w procesie fine-tuningu i stanowił 80% całego przygotowanego wariantu danych, czyli 71 176 wiadomości.

Podobnie jak pozostałe podzbiory, był wydzielany dopiero po normalizacji, oczyszczeniu, deduplikacji i balansowaniu wybranego wariantu danych. Każda konfiguracja eksperymentalna korzystała więc ze spójnego podziału, a ta sama lub bardzo podobna wiadomość nie mogła trafić jednocześnie do zbioru treningowego oraz testowego. Zbiór treningowy zawierał po 35 588 przykładów obu klas, ponieważ wcześniej zastosowano balansowanie do relacji



Rysunek 4.3.: Udział źródeł w najczęściej używanym zbiorze treningowym.

Źródło: opracowanie własne.

50/50.

Zbiór walidacyjny

Zbiór walidacyjny stanowił 10% danych, czyli 8 897 wiadomości: 4448 przykładów jednej klasy i 4449 drugiej. Wykorzystywano go do obserwowania przebiegu treningu i porównywania konfiguracji. Przy metodach generatywnych sprawdzano na nim również format odpowiedzi. Gdy model miał zwracać etykietę w określonej strukturze, na przykład jako tekst `spam/ham` albo obiekt JSON, na zbiorze walidacyjnym sprawdzano, czy odpowiedzi były poprawnie parsowalne. W dalszej części pracy określenie *część walidacyjna* odnosi się do tego samego 10-procentowego podzbioru.

Zbiór testowy

Zbiór testowy stanowił 10% danych. Nie był używany do aktualizacji parametrów ani do wyboru konfiguracji. W głównym porównaniu punktów kontrolnych posługiwano się jednak próbką treningową oraz trzema zbiorami zewnętrznymi. Z tego powodu rozdział nie przedstawia wspólnego wyniku końcowego na tym podziale dla wszystkich czterech metod dostrajania; jest to jedno z ograniczeń badania.

4.2.2. Przygotowanie danych

Normalizacja schematu

Pierwszym krokiem przygotowania danych było sprowadzenie różnych formatów źródłowych do jednego schematu. Niektóre pliki zawierają osobne kolumny `subject` i `body`; inne

przechowują całą wiadomość w jednym polu tekstowym; jeszcze inne mają format zbliżony do surowego e-maila z nagłówkami. Przyjęto zasadę zachowania możliwie dużej liczby przykładów. W przypadku wiadomości surowych wydzielano temat oraz treść. Jeżeli wiadomość była wieloczęściowa, analizowano część tekstową. Jeżeli nie udało się poprawnie sparsować struktury, całą dostępną treść traktowano jako ciało wiadomości.

Drugim elementem normalizacji było ujednolicenie etykiet. W różnych zbiorach klasa pozytywna bywa oznaczana jako `spam`, `1`, `Class=1` albo przez sam fakt pochodzenia ze zbioru wiadomości oszukańczych. W pracy przyjęto konwencję, że `label=0` oznacza wiadomość pożądaną (*ham*), natomiast `label=1` wiadomość niepożądaną, phishingową lub oszukańczą. Dla zbiorów CEAS, TREC, Ling, Nazario i SpamAssassin zachowano klasy zapisane w źródle. W `spam_ham` etykietę tekstową `spam` zamieniono na 1, a `ham` na 0. Wszystkim wiadomościom z `enron` przypisano 0, a rekordom z `fraudulent_email_corpus` wartość 1. Nie zmieniano ręcznie znaczenia pojedynczych etykiet; wiadomość reklamowa trafiała zatem do klasy pozytywnej tylko wtedy, gdy tak oznaczono ją w źródle.

W tabeli 4.3 zestawiono kolumny, z których pobierano temat, treść i etykietę przed sprowadzeniem danych do wspólnego schematu.

Tabela 4.3.: Dostępne kolumny w surowych zbiorach danych oraz sposób ich parsowania

Zbiór	Plik źródłowy	Dostępne kolumny	Sposób parsowania
enron	emails.csv	file, message	Wydzielenie tematu i treści z pola <code>message</code> .
trec_2007	email_origin.csv	label, origin	Wydzielenie tematu i treści z pola <code>origin</code> .
ceas_2008	CEAS_08.csv	sender, receiver, date, subject, body, label, urls	Bezpośrednie mapowanie pól <code>subject</code> , <code>body</code> , <code>label</code> .
spam_assassin	spam_assassin.csv	text, target	Wydzielenie tematu i treści z pola <code>text</code> .
spam_ham	spam_ham_dataset.csv	label, text, label_num	Wydzielenie tematu z prefiksu <code>Subject:</code> .

Zbiór	Plik źródłowy	Dostępne kolumny	Sposób parsowania
fraudulent_email_corpus	fradulent_emails.txt	wiadomości zapisane kolejno z nagłówkami	Wydzielenie tematu z nagłówków i treści z wiadomości.
nazario	Nazario.csv	sender, receiver, date, subject, body, label, urls	Bezpośrednie mapowanie pól subject, body, label.
ling	Ling.csv	subject, message, label	Przeniesienie message do pola body

Wykorzystane pliki pochodziły z kopii opublikowanych w serwisie Kaggle. Dla zbiorów Enron, TREC 2007, Ling-Spam, SpamAssassin i Nazario dostępna jest także dokumentacja źródeł, z których utworzono późniejsze kopie [54, 55, 30, 56, 57]. Nazwy plików podano w tabeli, a sumy SHA-256 użytych kopii zapisano w pliku `dataset/raw_datasets/SHA256SUMS`. Numery wersji zestawów Kaggle nie zostały zapisane podczas pobierania, dlatego odtworzenie ich pochodzenia opiera się na adresie źródła, nazwie pliku i sumie kontrolnej.

Usuwanie pustych rekordów

Po normalizacji odrzucano rekordy, w których puste były jednocześnie temat i treść. Przykład pozbawiony jakiegokolwiek tekstu wnosi niewiele do uczenia, a może wprowadzać artefakty techniczne. Jeżeli pewien zbiór zawierałby dużo pustych rekordów tylko w jednej klasie, model mógłby nauczyć się, że brak danych jest cechą klasy, co nie byłoby wiarygodną regułą w realnym wdrożeniu.

Deduplikacja

Deduplikacja ogranicza ryzyko przecieku informacji między zbiorem treningowym a testowym. Bez tego modele językowe o dużej pojemności mogłyby zapamiętywać powtarzające się wiadomości zamiast uczyć się ogólnych reguł klasyfikacji.

W dalszych eksperymentach użyto poziomu deduplikacji **high**. Klucz duplikatu tworzone z treści wiadomości: wykonywano normalizację Unicode NFKC, dekodowano encje HTML, zamieniano tekst na małe litery, usuwano znaczniki HTML, adresy URL, adresy e-mail i znaki niealfanumeryczne. Temat i etykieta nie były częścią klucza. Po złączeniu źródeł pozostawiano pierwszy rekord dla każdego klucza, a dopiero potem dzielono dane. Zapobiegało to rozdzielaniu kopii tej samej treści między podzbiory, ale sposób wyboru pierwszego re-

kordu oznaczał, że ewentualne konflikty etykiet zależały od kolejności danych. Ustawienia zastosowane podczas tworzenia klucza duplikatu zestawiono w tabeli 4.4.

Element procedury	Ustawienie
Klucz porównania	znormalizowana treść wiadomości
Normalizacja Unicode	NFKC
HTML	dekodowanie encji i usunięcie znaczników
Elementy techniczne	usunięcie adresów URL i adresów e-mail
Pozostałe znaki	małe litery i usunięcie znaków niealfanumerycznych
Temat i etykieta	niewzglądane w kluczu
Wybór rekordu	pierwszy rekord w kolejności połączenia źródeł

Tabela 4.4.: Parametry deduplikacji na poziomie **high**.

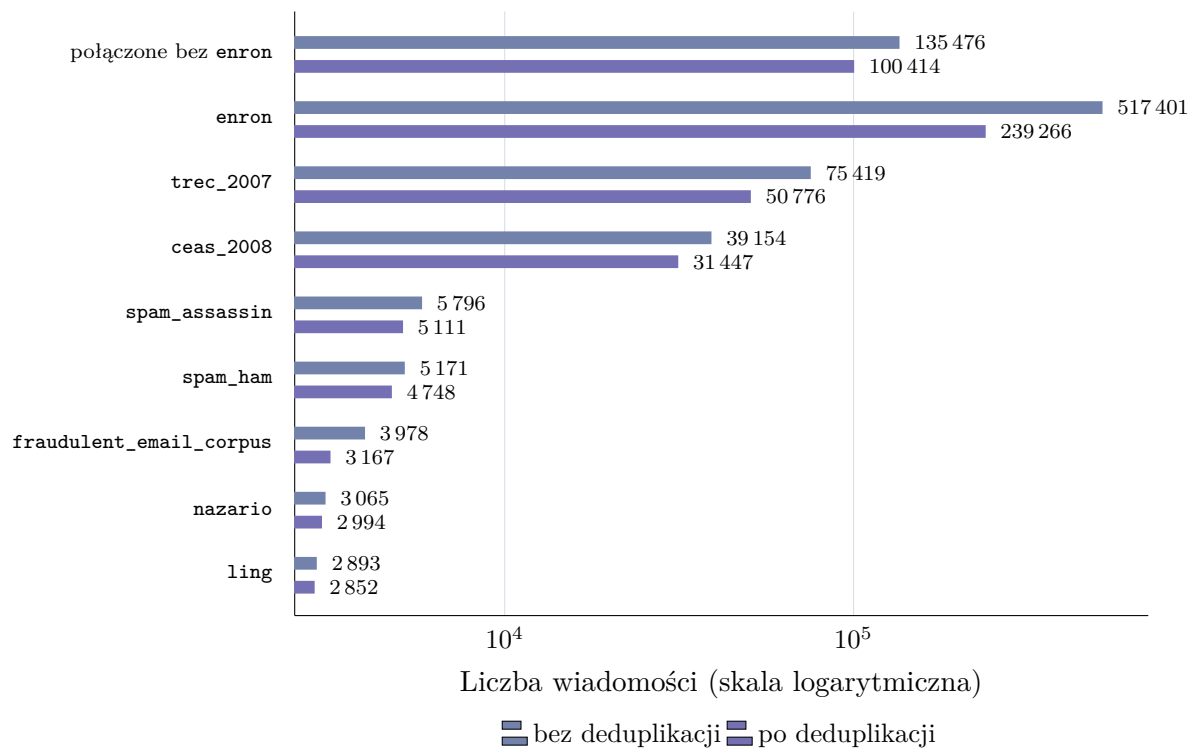
W tej części analizowano zbiór utworzony z połączenia wszystkich źródeł poza **enron**. Po normalizacji otrzymano 135 476 rekordów. Następnie odrzucono 106 wiadomości z pustym tematem i pustą treścią, pozostawiając 135 370 przykładów do deduplikacji. Deduplikacja na poziomie **high** wykryła 5344 grupy duplikatów i usunęła 34 956 nadmiarowych rekordów. Po tym etapie pozostało 100 414 wiadomości. Mediana rozmiaru grupy duplikatów wyniosła 2, a największa grupa zawierała 7832 wiadomości. Wszystkie rekordy w tej grupie miały pustą treść po normalizacji, ale różne tematy, między innymi **APEC security**, **GOODDAY SIR** oraz **Turn your views into cash**. Grupa obejmowała zarówno etykiety **ham**, jak i **spam**. Łącznie wykryto siedem grup z konfliktem etykiet. Procedura nie rozstrzygała ich osobno, lecz zachowywała pierwszy rekord zgodnie z kolejnością źródeł. Jest to ograniczenie przyjętej deduplikacji i mogło zmienić rozkład danych przed balansowaniem.

Rysunek 4.4 zestawia dwa poziomy analizy: osobne zbiory źródłowe oraz zbiór połączony ze wszystkich źródeł poza **enron**. Dzięki temu widać zarówno wpływ deduplikacji na dane używane do dalszego łączenia, jak i skalę powtórzeń w poszczególnych źródłach. Największą różnicę widać w osobno pokazanym zbiorze **enron**. Jest to archiwum poczty z jednej firmy, w którym ta sama wiadomość mogła trafić do wielu odbiorców, tworząc dużą liczbę powtórzeń.

Rysunek 4.5 pokazuje, że większość grup duplikatów jest niewielka. Jest to typowe dla zbiorów e-mailowych: część wiadomości występuje dokładnie lub prawie dokładnie dwa razy, natomiast bardzo duże grupy są rzadsze. Ze względu na długi ogon rozkładu większe grupy zostały połączone w przedziały. Nawet niewielkie grupy są jednak istotne, ponieważ przy losowym podziale danych mogłyby zostać rozdzielone między trening i test.

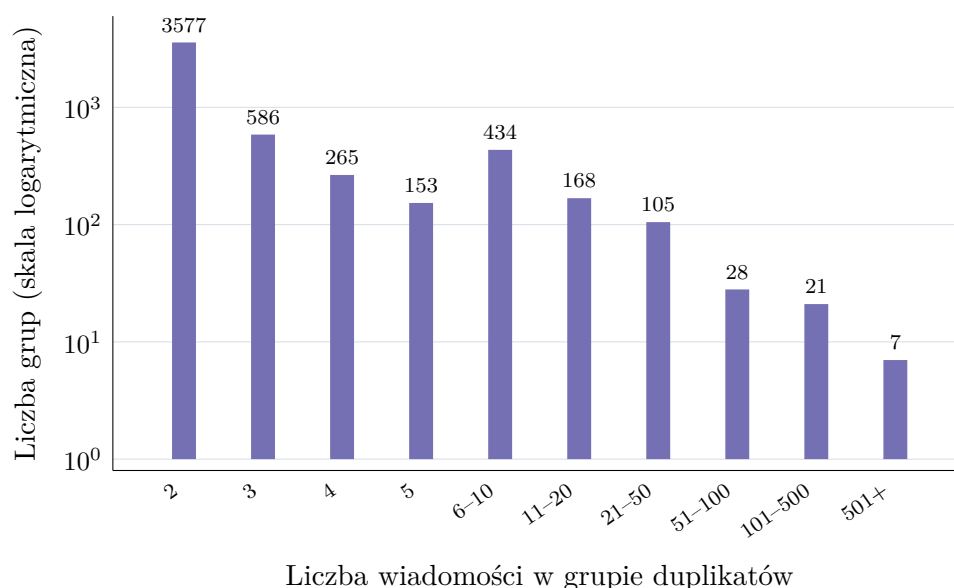
Balansowanie klas

Po oczyszczeniu i deduplikacji zastosowano balansowanie klas do proporcji 50/50. Po deduplikacji zbiór utworzony ze wszystkich źródeł poza **enron** zawierał 51 418 wiadomości



Rysunek 4.4.: Liczba rekordów przed deduplikacją oraz po deduplikacji.

Źródło: opracowanie własne.

Rysunek 4.5.: Rozmiary grup duplikatów po połączeniu zbiorów danych poza **enron**.

Źródło: opracowanie własne.

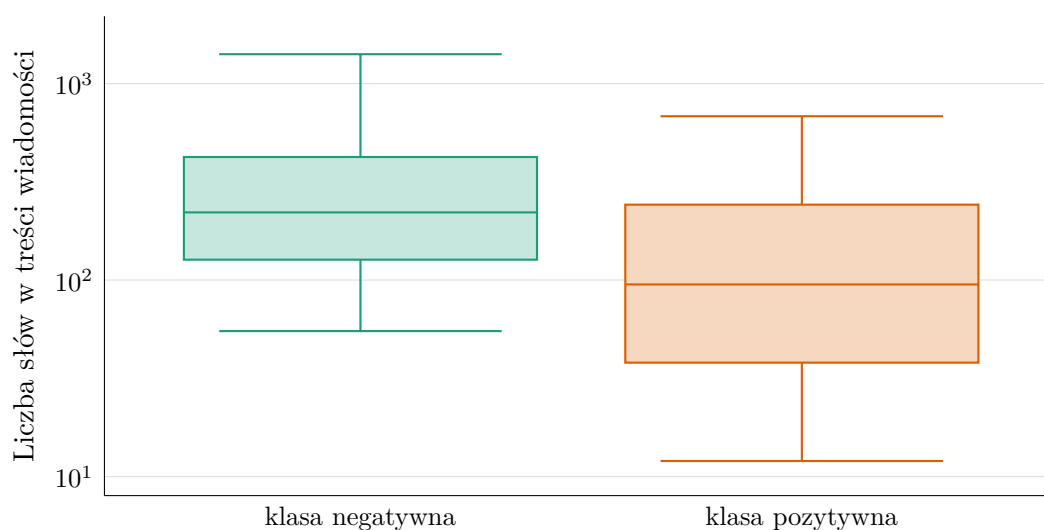
legalnych oraz 48 996 wiadomości niepożądanych. Balansowanie usunęło więc jedynie nadmiar przykładów z klasy negatywnej. Po balansowaniu zbiór ten zawierał 97 992 rekordy: 48 996 wiadomości pozytywnych i 48 996 negatywnych. Dzięki temu metryka dokładności staje się łatwiejsza do interpretacji, ponieważ klasy mają równy udział. Model nie jest też zachęcany do preferowania klasy większościowej. Porównanie wariantów eksperymentalnych jest stabilniejsze, ponieważ zmiany wyników nie wynikają z przypadkowego przesunięcia proporcji klas.

Balansowanie ma jednak również ograniczenia. Rzeczywisty strumień poczty użytkownika nie będzie zbalansowany. W większości środowisk spam może stanowić mały procent wiadomości, w innych znacznie większy. Dlatego wyniki na zbiorze zbalansowanym należy interpretować jako ocenę zdolności rozróżniania klas, a nie bezpośrednią symulację produkcyjnego rozkładu wiadomości.

Analiza długości wiadomości

Długość wiadomości wpływa na koszt przetwarzania przez model językowy, dobór maksymalnej liczby tokenów oraz potencjalne ucinanie wejścia. W analizowanym wariancie mediana długości treści wynosi 159 słów, natomiast 95. percentyl wynosi 1057 słów. Oznacza to, że większość wiadomości mieści się w umiarkowanym limicie kontekstu, ale istnieje istotna grupa długich wiadomości. Jeżeli model ma ograniczony maksymalny kontekst, zbyt krótkie

obcięcie treści mogłoby usuwać informacje ważne dla klasyfikacji.



Rysunek 4.6.: Rozkład długości treści wiadomości według etykiety w analizowanym wariancie danych.

Źródło: opracowanie własne.

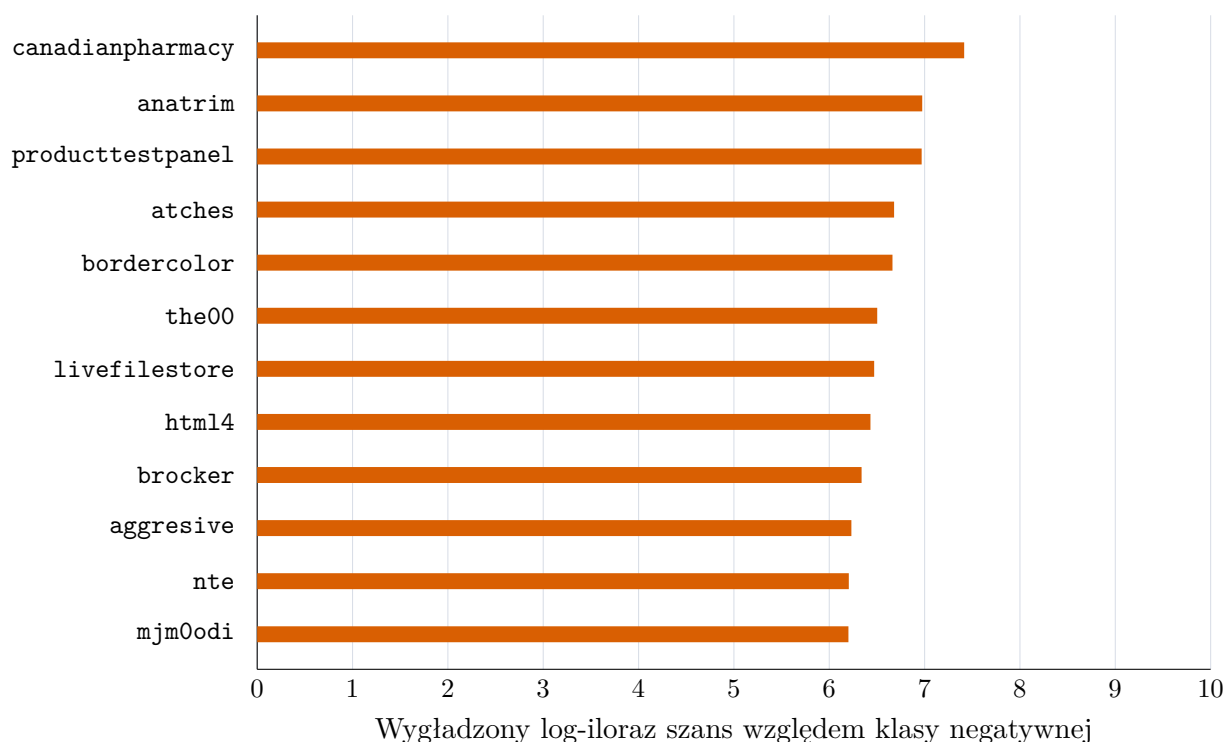
Rysunek 4.6 pokazuje różnice długości między klasami. Wiadomości pożądane mają medianę 221 słów, pierwszy kwartyl 127 słów, trzeci kwartyl 423 słowa i 95. percentyl 1411 słów. Wiadomości spamowe są krótsze: mediana wynosi 95 słów, pierwszy kwartyl 38 słów, trzeci kwartyl 242 słowa, a 95. percentyl 682 słowa. Krótsza treść może wynikać z charakteru analizowanych kampanii spamowych, ale w innych zbiorach spam może być dłuższy, na przykład w wiadomościach oszukańczych typu *advance-fee fraud* [5].

Analiza według źródeł ujawnia dodatkowe różnice. W `ceas_2008` wiadomości pożądane mają medianę 201 słów, a spam jedynie 40 słów. W `trec_2007` wiadomości pożądane mają medianę 224 słowa, natomiast spam 147 słów. W `spam_assassin` spam ma medianę 379 słów, a w `ling` spam ma medianę 260 słów. Oznacza to, że różnica długości między klasami nie jest stabilną właściwością samego problemu, lecz zależy od źródła danych. Model może wykorzystywać tę informację, co negatywnie wpłynie na jego realną skuteczność. Jeżeli klasyfikator opierałby się głównie na długości, generalizacja do nowych typów spamu mogłaby być ograniczona.

Analiza słownictwa

Przeprowadzono także prostą analizę słów najbardziej powiązanych z klasą pozytywną i negatywną. Ranking oparto na wygładzonym ilorazie szans liczonym dla tokenów występujących w temacie i treści wiadomości. Wśród terminów silnie związanych ze spamem po-

jawiają się między innymi `canadianpharmacy`, `anatrim`, `producttestpanel`, `atches` oraz `bordercolor`. Część z nich odpowiada klasycznym kampaniom reklamowym i farmaceutycznym, a część odzwierciedla konkretne źródła wiadomości phishingowych. Z kolei wśród terminów powiązanych z wiadomościami legalnymi pojawiają się między innymi nazwy serwisów pogodowych, takie jak `accuweather`, oraz tokeny `reproducible`, `speakup` i `cbsnews`.



Rysunek 4.7.: Terminy najsilniej powiązane z klasą pozytywną według wygładzonego ilorazu szans.

Źródło: opracowanie własne.

Rysunek 4.7 wskazuje, że wysoka pozycja słów farmaceutycznych potwierdza, że część spamu w zbiorze ma charakter reklamowy. Jednocześnie obecność bardzo specyficznych tokenów pokazuje ryzyko uczenia się artefaktów zbioru. Duży model językowy może poprawnie klasyfikować takie wiadomości, ale dobra skuteczność na zbiorze zawierającym powtarzalne słowa kampanii nie musi automatycznie oznaczać odporności na nowy, bardziej wyrafinowany phishing.

4.2.3. Testowane sposoby dostrajania modelu

W eksperymentach porównano cztery sposoby dostrojenia modelu do klasyfikacji wiadomości. Wszystkie warianty korzystały z tego samego modelu bazowego Qwen/Qwen3-0.6B oraz adapterów LoRA, ale inaczej zapisywały zadanie i inaczej przygotowywały przykłady treningowe.

1. **Klasyfikacja sekwencji** - Model ładowano przez `AutoModelForSequenceClassification` z biblioteki `transformers`. Treść wiadomości była tokenizowana bez szablonu czatu, a kolumna `label` trafiała do trenera jako etykieta. Taki wariant odpowiada klasycznej klasyfikacji tekstu.
2. **Generowanie odpowiedzi** - Model trenowano tak, aby po instrukcji klasyfikacyjnej wygenerował krótką odpowiedź tekstową: `ham` albo `spam`. Podczas ewaluacji odpowiedź miała ograniczoną liczbę tokenów, a wynik wyznaczał parser szukający etykiety w wygenerowanym tekście.
3. **Następny token** - Klasyfikację sprowadzono do przewidywania następnego tokenu. Dla każdej wiadomości budowano instrukcję w formacie czatu, w której model miał wybrać jedną z dwóch etykiet. W ewaluacji model nie generował tekstu. Porównywano prawdopodobieństwa tokenów odpowiadających klasom `ham` i `spam` w pozycji następnego tokenu po instrukcji.
4. **Odpowiedź strukturyzowana** - Ten wariant rozwijał poprzedni przez narzucenie formatu odpowiedzi. Model trenowano na krótkim uzupełnieniu w postaci obiektu JSON z polem `label` o wartości `spam` albo `ham`. Podczas ewaluacji parser w pierwszej kolejności szukał obiektu JSON, a dopiero potem prostszego wystąpienia pary `label: spam` lub `label: ham`.

We wszystkich wariantach liczba trenowanych parametrów była ograniczona przez adaptery LoRA. W dalszej części rozdziału najpierw przeanalizowano dobór parametrów dla metody następnego tokenu. Wybrane ustawienia wykorzystano później przy treningu pozostałych sposobów fine-tuningu.

W metodach korzystających z formatu czatu zastosowano szablon tokenizatora Qwen3 z wyłączonym trybem rozumowania, czyli `enable_thinking=False` [13]. W użytej wersji szablonu początek odpowiedzi asystenta kończył się pustym blokiem `<think></think>`; decyzję metody następnego tokenu odczytywano bezpośrednio po tym bloku. Etykiety były pojedynczymi tokenami bez poprzedzającej spacji: identyfikator `ham` wynosił 5604, a `spam` 75545. Dla ich logitów z_{ham} i z_{spam} obliczano softmax ograniczony do tych dwóch kandydatów:

$$P(c) = \frac{e^{z_c}}{e^{z_{ham}} + e^{z_{spam}}}, \quad c \in \{\text{ham}, \text{spam}\}.$$

Model przypisywał wiadomości klasę o większym prawdopodobieństwie:

$$\hat{c} = \arg \max_{c \in \{\text{ham}, \text{spam}\}} P(c).$$

Po podstawieniu treści wiadomości i zastosowaniu szablonu czatu wejście miało poniższą postać. Zapis `{email}` oznacza miejsce wstawienia badanej wiadomości.

Kod źródłowy 4.1: Prompt po zastosowaniu szablonu czatu Qwen3 w metodzie oceny następnego tokenu.

```
<|im_start|>user
You are an email spam classifier.
Classify the following email as spam or ham.
Return exactly one lowercase word: spam or ham.

Email:
{email}<|im_end|>
<|im_start|>assistant
<think>

</think>
```

4.2.4. Ocena modelu bazowego bez dostrajania

Przed rozpoczęciem właściwych eksperymentów sprawdzono, jak z zadaniem radzi sobie sam model Qwen/Qwen3-0.6B, bez aktualizacji wag i bez adapterów LoRA. Celem tego pomiaru było ustalenie, czy model bazowy ma już wystarczające właściwości zero-shot, czy dopiero dostrajanie pozwala wykorzystać go jako klasyfikator. Ewaluację wykonano na tych samych czterech zbiorach, które wykorzystano później do oceny punktów kontrolnych. Z każdego zbioru wybrano po 1000 przykładów.

Sprawdzono dwa sposoby odczytywania decyzji modelu. W pierwszym model generował krótką odpowiedź tekstową. Żeby nie zaniżyć wyniku modelu bazowego przez zbyt ogólną instrukcję, w wariancie generowania i parsowania zastosowano prompt przedstawiony na listingu 4.2.

Kod źródłowy 4.2: Prompt wykorzystany do oceny modelu bazowego w wariancie generowania i parsowania.

```
1 You are an email security classifier.
2 Classify the email as exactly one of two labels.
3
4 Definitions:
5 - spam: unsolicited advertising, scam, phishing, fraud, suspicious offer,
6   malware, or otherwise unwanted email.
7 - ham: legitimate non-spam email.
```

```

7
8 Return only one lowercase word: spam or ham.
9
10 Email:
11 {email}

```

W tym wariancie zastosowano możliwie tolerancyjny parser. Oprócz odpowiedzi **ham** i **spam** rozpoznawał on także obiekt JSON, zapis typu `label: spam`, krótkie odpowiedzi opisowe oraz zaprzeczenia, na przykład **not spam**. Generowanie było deterministyczne, ograniczone do 24 nowych tokenów i kończone po tokenie `<|im_end|>` lub tokenie końca sekwencji. W przypadku niepowodzenia parsera domyślną etykietą było **ham**, ale w pomiarze 1000 wiadomości na zbiór nie wystąpił ani jeden taki przypadek. Ręczna kontrola dodatkowych 50 odpowiedzi z każdego zbioru pokazała zawsze postać **ham<|im_end|>**.

W drugim wariancie zastosowano ten sam szablon czatu z `enable_thinking=False` i porównywano logity pojedynczych tokenów **ham** oraz **spam**. Softmax liczone wyłącznie dla tych dwóch kandydatów. Pozwoliło to oddzielić problem formatu odpowiedzi od samej preferencji modelu przy wyborze etykiety.

Zbiór	Wariant	Accuracy	F1	Recall	Specificity	Balanced acc.	Spam pred.
train_subset	generowanie	50,0	0,0	0,0	100,0	50,0	0,0
train_subset	następny token	50,0	0,0	0,0	100,0	50,0	0,0
enron	generowanie	100,0	0,0	0,0	100,0	50,0	0,0
enron	następny token	100,0	0,0	0,0	100,0	50,0	0,0
fraudulent	generowanie	0,0	0,0	0,0	0,0	0,0	0,0
fraudulent	następny token	0,0	0,0	0,0	0,0	0,0	0,0
spam_ham	generowanie	71,6	0,0	0,0	100,0	50,0	0,0
spam_ham	następny token	71,6	0,0	0,0	100,0	50,0	0,0

Tabela 4.5.: Wyniki modelu bazowego bez dostrajania. Wartości podano w procentach.

Zera w kolumnie *Spam pred.* oznaczają, że model bazowy nie wybrał klasy **spam** ani razu. Dlatego F1 i *recall* dla klasy pozytywnej pozostają równe 0%, mimo że część zbiorów zawiera wiadomości niepożądane.

Wyniki w tabeli 4.5 pokazują, że oba warianty dały ten sam rezultat. Nie wynikało to z błędu parsera, ponieważ w wariancie generacyjnym wszystkie odpowiedzi zostały poprawnie odczytane.

Na zbiorze **enron** model uzyskał 100%, ponieważ wszystkie wiadomości w tej próbce należały do klasy **ham**. Podobnie wynik 71,6% na zbiorze **spam_ham** odpowiadał udziałowi wiadomości poprawnych w próbce, a nie rzeczywistej zdolności wykrywania spamu. Znacznie lepiej pokazują to metryki związane z klasą pozytywną: *recall* oraz F1 dla spamu wyniosły 0% na każdym zbiorze zawierającym wiadomości niepożądane. Na zbiorze wiadomości oszukańczych **fraudulent_email_corpus** model nie wskazał poprawnie ani jednego przykładu.

Wyniki modelu bazowego pokazują więc, że samo sformułowanie zadania w postaci instrukcji nie wystarczało w tej konfiguracji.

4.3. Dobór parametrów treningu

Po przygotowaniu danych i przetestowaniu modelu bazowego kolejnym etapem był dobór parametrów fine-tuningu. Eksperymenty przeprowadzono dla metody opartej na przewidywaniu następnego tokenu. Model porównywał prawdopodobieństwa dwóch tokenów odpowiedzi: `ham` oraz `spam`, a klasę końcową wybierał na podstawie większego prawdopodobieństwa.

Eksperymenty przeprowadzono dla modelu `Qwen/Qwen3-0.6B`. Do dostrajania zastosowano adapter LoRA. We wszystkich konfiguracjach ustawiono ziarno losowe na 67, aby zachować powtarzalność podziału danych i próbkowania.

4.3.1. Przestrzeń hiperparametrów

Przeanalizowano 16 konfiguracji. Zmieniane parametry obejmowały maksymalną długość kontekstu, współczynnik uczenia¹, pojemność adaptera LoRA oraz dropout. Zmiany parametrów można podzielić na trzy grupy. Pierwsza grupa eksperymentów badała wpływ długości kontekstu i współczynnika uczenia. Druga grupa porównywała różne parametry LoRA, a trzecia zmieniała współczynnik dropout. Większej liczby kombinacji parametrów nie przetestowano ze względu na ograniczone zasoby. Tabela 4.6 przedstawia zakres konfiguracji objętych analizą punktów kontrolnych.

4.3.2. Ewaluacja pośrednich punktów kontrolnych

Oceniono zarówno końcowe adaptory, jak i pośrednie punkty kontrolne². Celem było znalezienie momentu, w którym model jest już dobrze dopasowany do danych treningowych, ale nie traci skuteczności na zbiorach zewnętrznych.

Dla każdej konfiguracji analizowano maksymalnie 10 punktów treningu: pierwszy punkt kontrolny, ostatni punkt kontrolny oraz punkty pośrednie rozłożone możliwie równomiernie. Ograniczało to koszt ewaluacji, ale nadal dawało obraz przebiegu uczenia. Łącznie oceniono 160 punktów treningu, po cztery zbiory danych dla każdego punktu. Daje to 640 zakończonych pomiarów.

¹ang. *learning rate*

²ang. *checkpoint*

Konf.	Kontekst	LR	LoRA r	LoRA alpha	Dropout
K01	512	1e-06	16	32	0.05
K02	512	5e-05	16	32	0.05
K03	512	1e-04	16	32	0.05
K04	1024	1e-06	16	32	0.05
K05	1024	5e-05	16	32	0.05
K06	1024	1e-04	16	32	0.05
K07	512	1e-04	8	16	0.05
K08	512	1e-04	32	64	0.05
K09	512	1e-04	64	128	0.05
K10	1024	1e-04	8	16	0.05
K11	1024	1e-04	32	64	0.05
K12	1024	1e-04	64	128	0.05
K13	512	1e-04	16	32	0.00
K14	512	1e-04	16	32	0.40
K15	1024	1e-04	16	32	0.00
K16	1024	1e-04	16	32	0.40

Tabela 4.6.: Konfiguracje treningu objęte analizą punktów kontrolnych.

4.3.3. Zbiory użyte do ewaluacji

Ewaluację przeprowadzono oddzielnie dla czterech zbiorów. Pierwszy zbiór, oznaczony jako `train_subset`, wydzielono z części treningowej `training_all`. Jego zadaniem było sprawdzenie, jak dobrze model radzi sobie z przykładami pochodzącymi z tej samej dystrybucji co dane treningowe. Nie jest to jednak całkowicie niezależny test generalizacji, ponieważ dane te są blisko powiązane ze zbiorem używanym do aktualizacji wag adaptera.

Poza podzbiorem `train_subset` wykorzystano również trzy kolejne, niezależne zbiory. `enron` zawiera wyłącznie wiadomości klasy `ham`, dlatego wykorzystano go do oceny błędów typu *false positive*. Zbiór `fraudulent_email_corpus` zawiera wyłącznie wiadomości oszukańcze, dlatego służy do oceny *recall* względem spamu o charakterze odmiennym od wiadomości treningowych. `spam_ham` jest natomiast zbiorem mieszanym, dlatego pozwala ocenić metryki zadania binarnego, w szczególności F1 dla klasy pozytywnej. Każdy z tych zbiorów został ograniczony do 1000 przykładów.

4.3.4. Metryki oceny

Wyniki analizowano za pomocą metryk *accuracy*, *precision*, *recall*, F1, *specificity*, *balanced accuracy*, *false positive rate* oraz *false negative rate*. Ponieważ część zbiorów zewnętrznych jest jednoklasowa, nie wszystkie metryki są jednakowo informacyjne. Dla `enron` kluczowa jest *specificity*, a dla `fraudulent_email_corpus` *recall*.

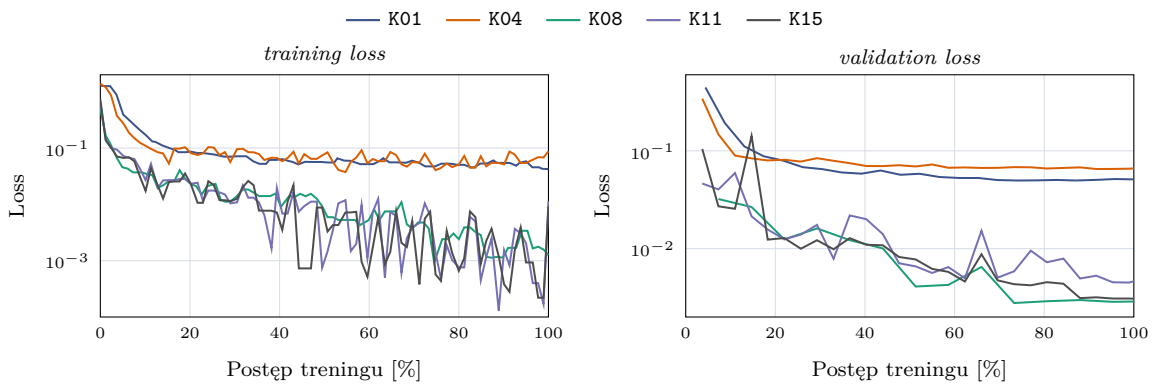
Do porównania konfiguracji wykorzystano następujący wzór:

$$S_{\text{ext}} = \frac{\text{specificity}_{\text{enron}} + \text{recall}_{\text{fraudulent}} + \text{F1}_{\text{spam_ham}}}{3}.$$

W ten sposób zewnętrzna walidacja uwzględniała trzy różne typy danych, a nie tylko jedną metrykę z jednego zbioru. Symbol S_{ext} oznacza dalej wynik zewnętrznej walidacji, nie wynik niezależnego testu końcowego.

4.3.5. Statystyki przebiegu treningu

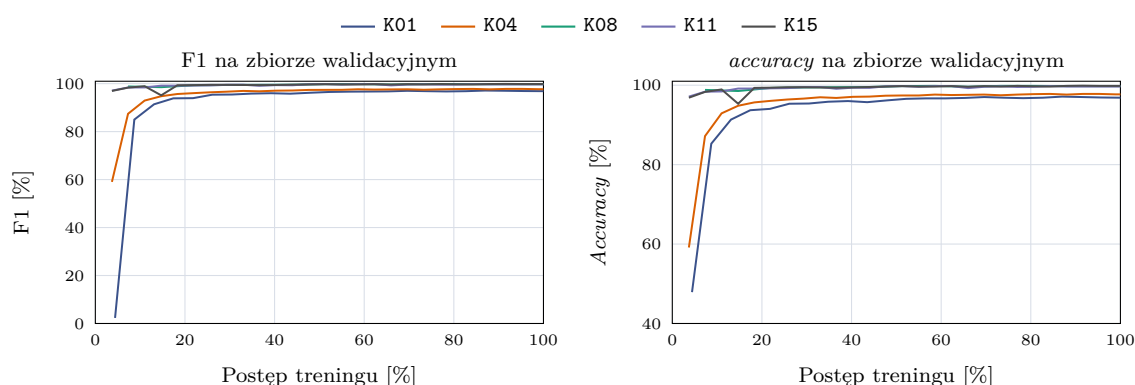
Przeanalizowano również metryki zapisywane podczas treningu. Nie były one głównym kryterium wyboru modelu, ale pomagały sprawdzić, jak szybko model dostosowuje się do danych treningowych i jak zmienia się koszt różnych konfiguracji. Do porównania wybrano konfiguracje K01, K04, K08, K11 oraz K15. K01 i K04 reprezentują warianty z niskim współczynnikiem uczenia, K11 ma podobną pojemność adaptera przy kontekście 1024 tokenów, a K15 odpowiada wariantowi bez dropout przy dłuższym kontekście.



Rysunek 4.8.: Przebieg funkcji straty dla wybranych konfiguracji treningu. Wykres *training loss* został wygładzony medianą kroczącą, aby ograniczyć szum wynikający z pomiaru na kolejnych batchach.

Źródło: opracowanie własne.

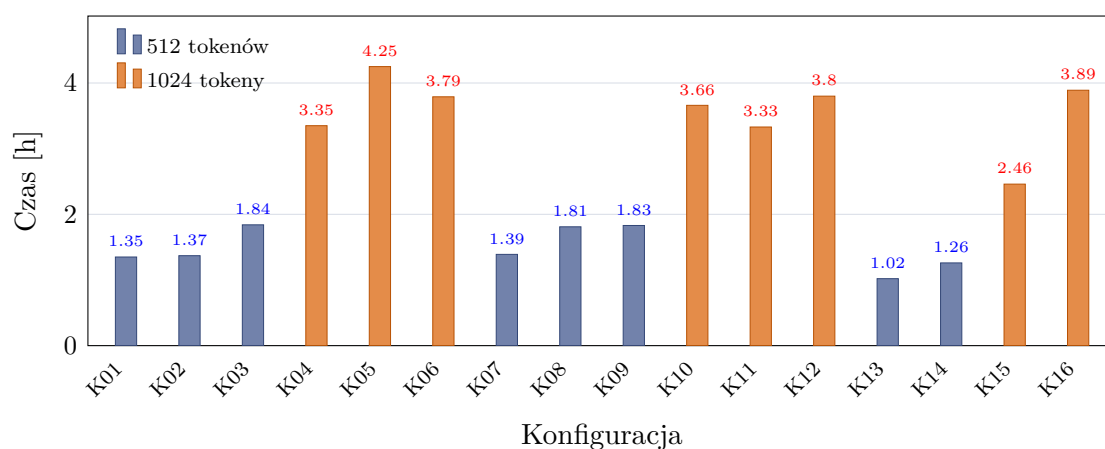
Na rysunku 4.8 widać, że konfiguracje z niskim współczynnikiem uczenia wolniej zmniejszały wartość funkcji straty. Dotyczy to przede wszystkim K01 i K04, czyli tych samych wariantów, które później uzyskały słabsze wyniki na zbiorach zewnętrznych. Dla K08, K11 i K15 strata walidacyjna szybko spadała do niskiego poziomu. Sam przebieg straty nie wystarczał jednak do wyboru konfiguracji, ponieważ po początkowej poprawie dalszy trening nie przekładał się jednoznacznie na lepszy wynik zewnętrzny.



Rysunek 4.9.: Przebieg F1 oraz *accuracy* na zbiorze walidacyjnym dla wybranych konfiguracji treningu.

Źródło: opracowanie własne.

Rysunek 4.9 pokazuje podobne zjawisko. F1 i *accuracy* na zbiorze walidacyjnym bardzo szybko osiągały wartości bliskie 100%. Model dobrze dopasowywał się więc do danych walidacyjnych, ale same metryki walidacyjne słabo rozdzielały najlepsze konfiguracje. O wyborze parametrów decydowały przede wszystkim wyniki na *enron*, *fraudulent_email_corpus* i *spam_ham*.



Rysunek 4.10.: Czas treningu poszczególnych konfiguracji.

Źródło: opracowanie własne.

Rysunek 4.10 pokazuje, że największy wpływ na czas treningu miała maksymalna długość kontekstu. Kolor słupka odpowiada maksymalnej długości kontekstu; wykres obejmuje 16 treningów, dla których zapisano metrykę *train_runtime*. Konfiguracje z limitem 512

tokenów trenowały się od około 1,02 do 1,84 godziny, natomiast warianty z limitem 1024 tokenów od około 2,46 do 4,25 godziny. Średnio oznaczało to wzrost z 1,48 do 3,57 godziny oraz spadek przepustowości z około 16,0 do 6,5 próbek na sekundę. Pozostałe parametry także wpływały na czas wykonania, ale w mniejszym i mniej regularnym stopniu. Większy rząd LoRA nie powodował jednoznacznego wzrostu czasu treningu w każdej grupie, a różnice między konfiguracjami o tym samym kontekście były mniejsze niż różnica między samymi długościami kontekstu. Dłuższy kontekst był więc wyraźnie droższy, a jak pokazano w dalszej części pracy, nie dawał przewagi uzasadniającej taki koszt.

4.3.6. Ranking konfiguracji

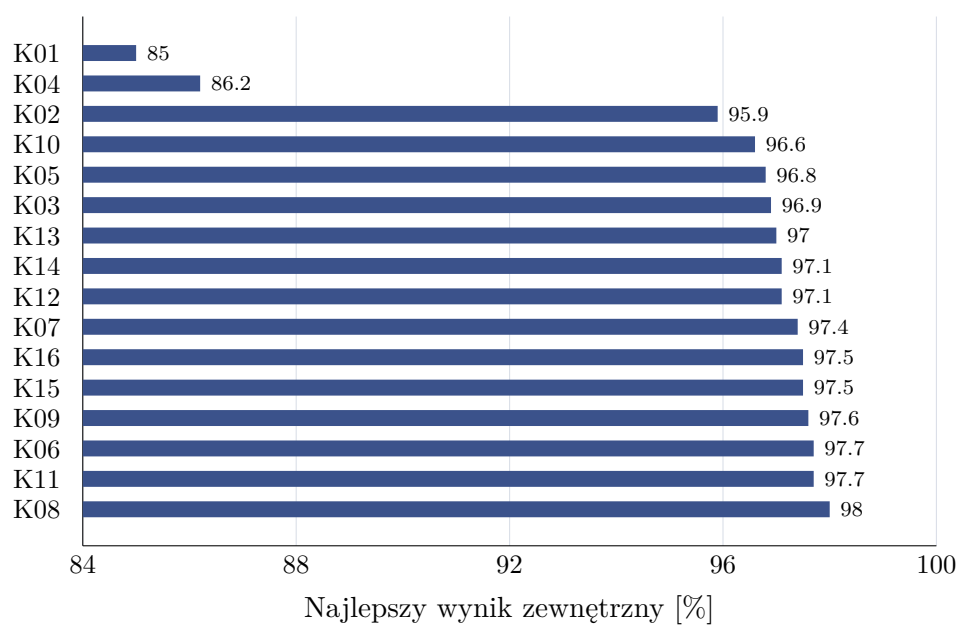
Tabela 4.7 przedstawia ranking najlepszych punktów kontrolnych dla poszczególnych konfiguracji. Dla każdej konfiguracji wybrano punkt kontrolny o najwyższym wyniku zewnętrznym. W tabeli uwzględniono pozycję w rankingu, identyfikator konfiguracji, numer wybranego punktu kontrolnego, wynik zewnętrzny S_{ext} , F1 na `train_subset`, F1 na `spam_ham` oraz różnicę między F1 na `train_subset` a wynikiem zewnętrznym.

Najlepszy rezultat uzyskała konfiguracja K08, czyli wariant z kontekstem 512 tokenów, współczynnikiem uczenia 10^{-4} , rzędem LoRA równym 32, parametrem alpha równym 64 oraz dropout równym 0,05. Najlepszy punkt kontrolny tej konfiguracji miał numer 7 i osiągnął wynik zewnętrzny 98,0%.

Miejsce	Konf.	Nr punktu kontrolnego	Wynik zewn. [%]	<code>train_subset</code> F1 [%]	<code>spam_ham</code> F1 [%]	Różnica [p.p.]
1	K08	7	98.0	99.9	97.2	1.9
2	K11	7	97.7	99.9	97.0	2.2
3	K06	8	97.7	99.9	96.7	2.2
4	K09	7	97.6	99.9	96.3	2.3
5	K15	8	97.5	99.8	96.2	2.3
6	K16	8	97.5	99.9	95.5	2.4
7	K07	7	97.4	99.8	96.2	2.4
8	K12	8	97.1	99.7	94.3	2.6
9	K14	5	97.1	100.0	96.6	2.9
10	K13	7	97.0	99.8	95.4	2.8

Tabela 4.7.: Ranking najlepszych punktów kontrolnych dla konfiguracji metody klasyfikacji następnego tokenu. Wynik zewnętrzny jest średnią z *specificity* na `enron`, *recall* na `fraudulent_email_corpus` oraz F1 na `spam_ham`.

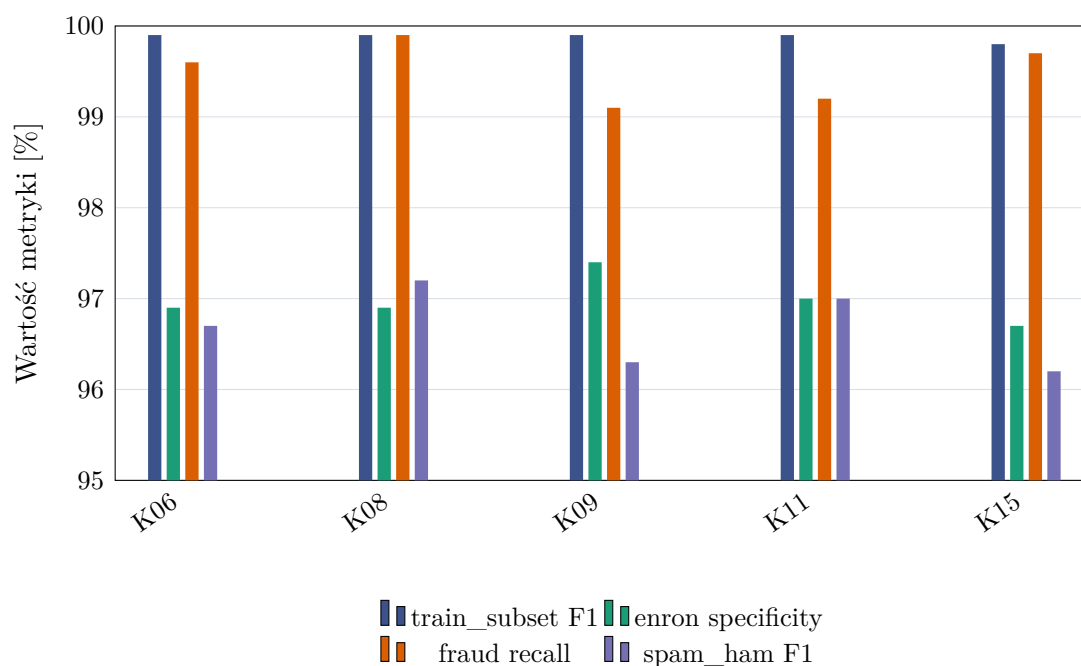
Rysunek 4.11 pokazuje, że większość konfiguracji ze współczynnikiem uczenia 10^{-4} osiąga bardzo zbliżone wyniki zewnętrzne. Różnice między najlepszymi wariantami mieszczą się w kilku dziesiątych punktu procentowego. Wyraźnie słabsze są natomiast warianty K01 i K04, które wykorzystywały współczynnik uczenia 10^{-6} . Osiągnęły one wyniki zewnętrzne odpowiednio 85,0% i 86,2%, a więc znacząco niższe od pozostałych konfiguracji. Sugeruje to, że przy przyjętej liczbie epok i rozmiarze danych tak niski współczynnik uczenia nie pozwalał dostatecznie zaadaptować modelu do zadania.



Rysunek 4.11.: Najlepszy wynik zewnętrzny uzyskany przez każdą analizowaną konfigurację metody klasyfikacji następnego tokenu.

Źródło: opracowanie własne.

Rysunek 4.12 przedstawia szczegóły dla pięciu najlepszych konfiguracji. Wszystkie osiągały bardzo wysokie F1 na `train_subset`, bliskie 100%. Najważniejsze różnice widoczne są jednak na zbiorach zewnętrznych. Konfiguracja K08 zachowuje wysokie *specificity* na `enron`, bardzo wysoki *recall* na `fraudulent_email_corpus` oraz najwyższy F1 na `spam_ham` wśród porównywanych wariantów. Wynik ten wskazuje na stabilność tej konfiguracji przy zmianie źródła danych.



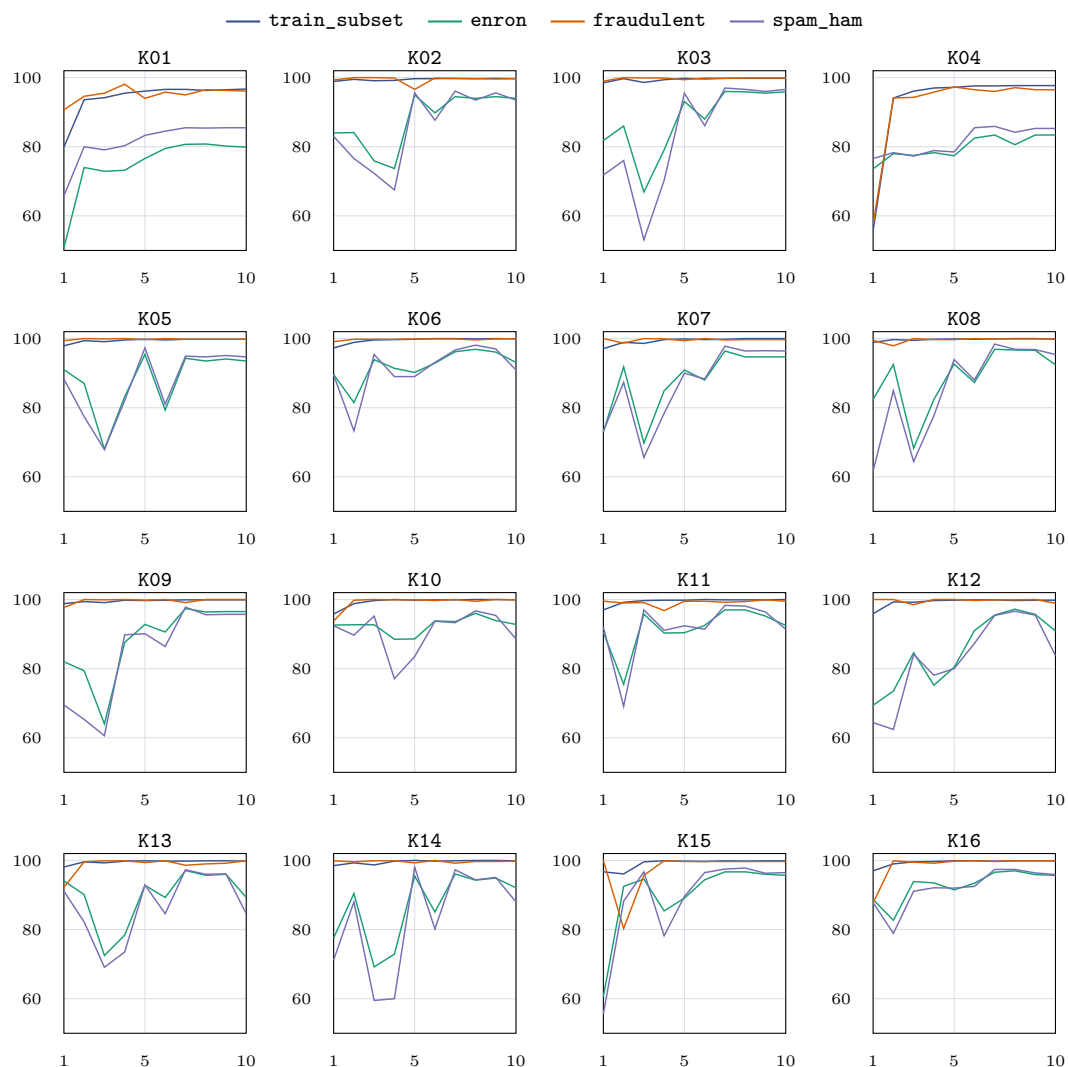
Rysunek 4.12.: Porównanie metryk dla pięciu najlepszych konfiguracji według wyniku zewnętrznego.

Źródło: opracowanie własne.

Uzupełniając przeanalizowano również przebieg *accuracy* dla wszystkich analizowanych konfiguracji oraz wszystkich zbiorów ewaluacyjnych. Rysunek 4.13 ma charakter informacyjny i pokazuje, jak zmieniała się skuteczność w kolejnych ocenianych punktach kontrolnych. Wnioski dotyczące wyboru parametrów oparto jednak na opisanym wcześniej wyniku zewnętrznej walidacji oraz porównaniu z próbką treningową.

4.3.7. Analiza przeuczenia

Porównanie wyników na `train_subset` i zewnętrznych zbiorach walidacyjnych pozwala ocenić ryzyko przeuczenia. Sam wynik na próbce treningowej jest niewystarczający, ponieważ większość konfiguracji osiąga tam F1 bliskie 100%. Gdyby wybór opierał się tylko na

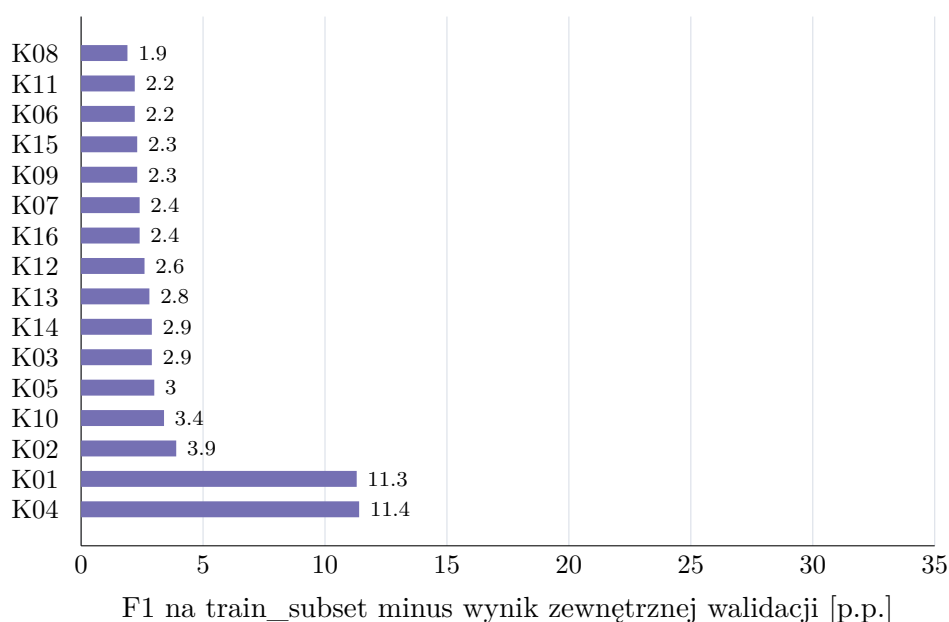


Rysunek 4.13.: Przebieg *accuracy* dla analizowanych konfiguracji. Każdy panel odpowiada jednej konfiguracji, a linie przedstawiają osobne zbiory ewaluacyjne.

Źródło: opracowanie własne.

tej metryce, nie dałoby się rozróżnić konfiguracji podobnie dopasowanych do danych treningowych, ale inaczej zachowujących się po zmianie źródła.

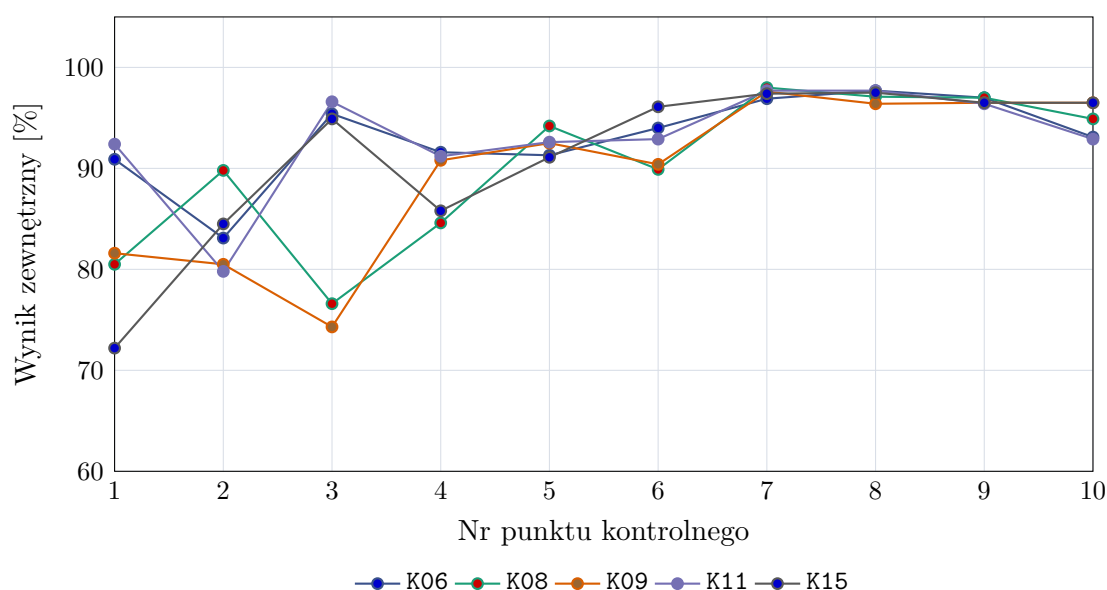
Rysunek 4.14 pokazuje różnicę między F1 na `train_subset` a wynikiem zewnętrznej walidacji. Najmniejszą różnicę spośród analizowanych konfiguracji ma K08. Różnica wynosi około 1,9 punktu procentowego. Dla większości dobrych konfiguracji różnica mieści się w zakresie od około 2 do 3 punktów procentowych. Wyjątkami są K01 i K04, dla których różnica wynosi ponad 11 punktów procentowych. Oznacza to, że konfiguracje z najniższym współczynnikiem uczenia znacznie gorzej przenoszą skuteczność ze zbioru podobnego do treningowego na dane o innej charakterystyce.



Rysunek 4.14.: Różnica między F1 na próbce treningowej `train_subset` a wynikiem zewnętrznej walidacji dla najlepszych punktów kontrolnych poszczególnych konfiguracji.

Źródło: opracowanie własne.

Analiza przebiegu punktów kontrolnych wskazuje również, że wybór końcowego adaptera nie zawsze jest optymalny. Rysunek 4.15 przedstawia zmianę wyniku zewnętrznego w kolejnych ocenianych punktach kontrolnych dla pięciu najlepszych konfiguracji. We wszystkich przypadkach najlepszy wynik pojawia się przed końcem treningu. Dla K08 najlepszy wynik uzyskano dla punktu kontrolnego nr 7, mimo że dalsze punkty kontrolne nadal osiągają wysokie wartości. Sugeruje to, że po tym etapie dalsze dopasowywanie do zbioru treningowego zaczyna pogarszać wynik na danych zewnętrznych. Podobny przebieg widać w pozostałych czterech najlepszych konfiguracjach.



Rysunek 4.15.: Przebieg wyniku zewnętrznego dla pięciu najlepszych konfiguracji w kolejnych ocenianych punktach kontrolnych.

Źródło: opracowanie własne.

4.3.8. Wybór najlepszej konfiguracji

Na podstawie wyników jako najlepszą konfigurację metody klasyfikacji następnego tokenu wybrano K08. Konfiguracja ta osiągnęła najwyższy wynik zewnętrzny spośród ocenianych wariantów. Miała również najmniejszą różnicę między wynikiem na próbie treningowej i zewnętrznej walidacji, co ogranicza ryzyko wyboru modelu nadmiernie dopasowanego do danych treningowych. Wyniki na zbiorach zewnętrznych wskazują również, że model zachowywał wysoką wykrywalność spamu, a jednocześnie utrzymywał niski odsetek fałszywych pozytywów na wiadomościach typu *ham*.

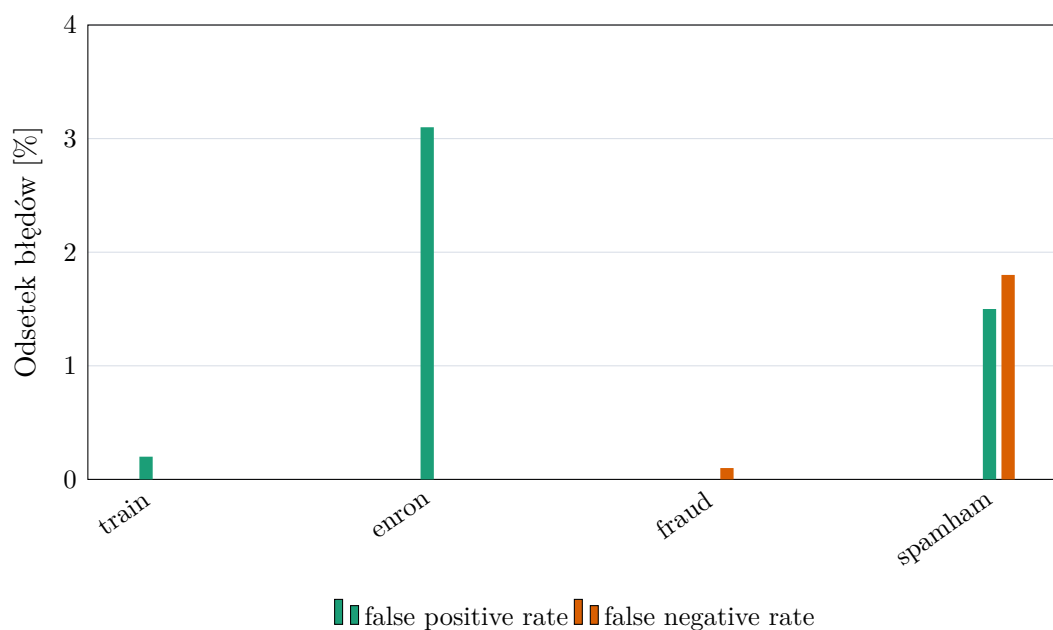
Wybrane parametry to maksymalna długość kontekstu 512 tokenów, współczynnik uczenia 10^{-4} , LoRA $r = 32$, LoRA $\alpha = 64$ oraz dropout równy 0,05. Najlepszy oceniony punkt kontrolny miał numer 7. Długość kontekstu 512 tokenów ogranicza także koszt treningu i inferencji względem wariantów 1024-tokenowych. Wyniki wskazują, że w tej serii eksperymentów dłuższy kontekst nie był konieczny do uzyskania najlepszego wyniku w tej walidacji.

W obrębie analizowanej metody konfiguracja K08 stanowi najlepszy kompromis między skutecznością na danych podobnych do treningowych, odpornością na zmianę źródła danych oraz kosztem obliczeniowym. Wysoki wynik na *fraudulent_email_corpus* wskazuje, że model poprawnie rozpoznaje wiadomości oszukańcze o charakterystyce innej niż część wiadomości treningowych. Wynik na *enron* pokazuje natomiast, że model nie osiąga wysokiej

wykrywalności spamu kosztem nadmiernej liczby fałszywych alarmów. Z tego powodu konfiguracja K08 została przyjęta jako reprezentatywny wariant metody klasyfikacji następnego tokenu.

4.3.9. Profil błędów wybranej konfiguracji

Dla wybranej konfiguracji K08 przeanalizowano również profil błędów. Rysunek 4.16 zestawia odsetek decyzji typu *false positive* i *false negative* na poszczególnych zbiorach. Na **enron** FPR wyniósł 3,1%; przy dużej liczbie wiadomości może to prowadzić do znacznej liczby fałszywych alarmów, dlatego konieczna jest analiza progów i kosztów błędów. Odpowiada to średnio około 31 zatrzymanym wiadomościom na każde 1000 wiadomości pożądanых. Na **fraudulent_email_corpus** odsetek decyzji typu *false negative* wynosi około 0,1%, a więc model prawie wszystkie wiadomości oszukańcze rozpoznaje jako klasę pozytywną. Na mieszanym zbiorze **spam_ham** *false positive* i *false negative* są zbliżone i nie przekraczają 2%. Model nie osiąga więc wysokiej skuteczności kosztem skrajnego przesunięcia w stronę jednej klasy.



Rysunek 4.16.: Odsetki błędów typu *false positive* i *false negative* dla wybranego punktu kontrolnego konfiguracji K08.

Źródło: opracowanie własne.

4.3.10. Wnioski z eksperymentu

Analiza pokazała, że wynik na `train_subset` nie wystarczał do wyboru najlepszej konfiguracji. Najlepsze warianty miały tam bardzo wysokie F1, więc różnice między nimi było widać dopiero na zbiorach zewnętrznych: `enron`, `fraudulent_email_corpus` i `spam_ham`.

Najgorzej wypadły warianty K01 i K04, trenowane ze współczynnikiem uczenia 10^{-6} . Używały niższe wyniki na zbiorach zewnętrznych i miały największe różnice względem próbki treningowej. Lepsze okazały się konfiguracje z 10^{-4} . Wśród nich najlepiej wypadł wariant z $r = 32$ i $\alpha = 64$. W badanym przebiegu wariant $r = 8$ uzyskał niższy wynik, ale jego zwiększanie nie prowadziło do wyraźnej poprawy.

Nie było też potrzeby stosowania kontekstu 1024-tokenowego. Najlepszy wariant korzystał z limitu 512 tokenów, więc był tańszy obliczeniowo, a jednocześnie zachowywał wystarczająco dużo informacji do klasyfikacji.

Wyniki punktów kontrolnych pokazały, że ostatni zapisany adapter nie zawsze był najlepszy. W kilku konfiguracjach lepsze wyniki na zbiorach zewnętrznych pojawiły się wcześniej. Dlatego przy wyborze modelu warto sprawdzać nie tylko metryki walidacyjne, ale też okresowo oceniać model na danych z innych źródeł.

Klasyfikacja przez przewidywanie następnego tokenu sprawdziła się w tym zadaniu. Po dostrojeniu model dobrze wykrywał wiadomości oszukańcze i utrzymywał niski odsetek fałszywych alarmów. Najlepszym wariantem był K08; jego parametry wykorzystano później przy porównaniu metod zwracających etykietę `ham/spam`.

4.4. Porównanie metod dostrajania

Po wybraniu najlepszej konfiguracji dla wariantu przewidywania następnego tokenu porównano pozostałe sposoby dostrajania modelu. W tym etapie nie powtarzano pełnego przeszukiwania hiperparametrów. Sprawdzano raczej, czy przy zbliżonych ustawieniach treningu znaczenie ma sama postać zadania: klasyfikacja sekwencji, generowanie etykiety, ocena następnego tokenu albo generowanie odpowiedzi strukturyzowanej. Dla metod 01, 02 i 04 oceniono cztery najlepsze konfiguracje wybrane na podstawie wcześniejszego eksperymentu. Metoda 03 obejmuje pełny ranking 16 konfiguracji opisany wcześniej.

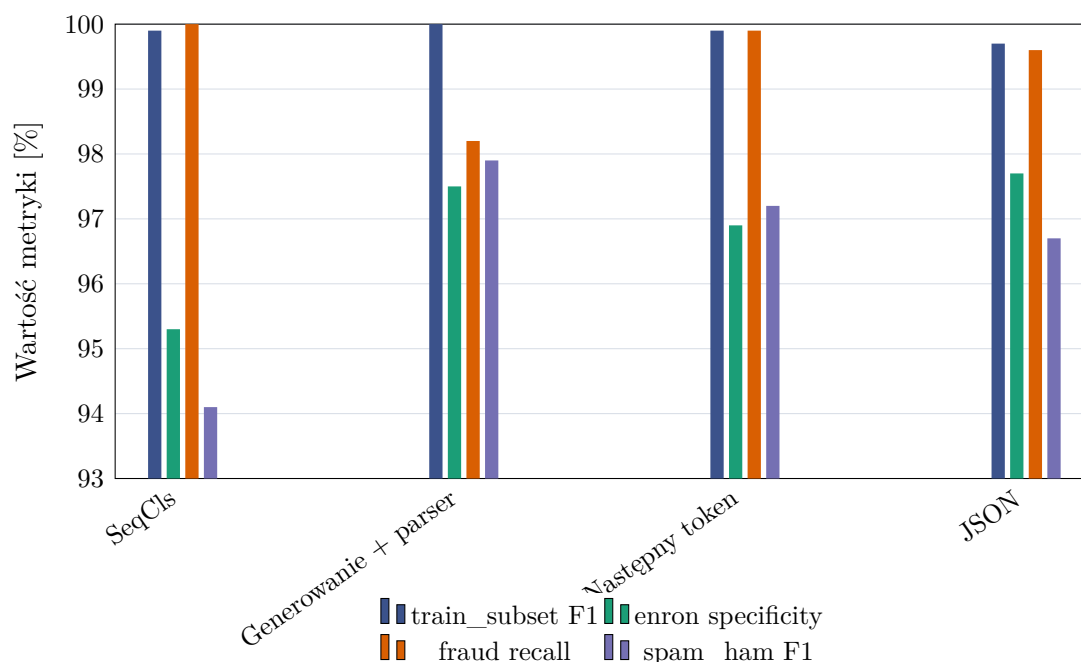
Przy wyborze metody liczył się nie tylko wynik zbiorczy, ale też sposób działania modelu podczas inferencji. Klasyfikacja sekwencji zwraca decyzję bez generowania tekstu i bez parsera, ale wymaga osobnej głowicy klasyfikacyjnej. Generowanie odpowiedzi jest proste koncepcyjnie, jednak wynik zależy od tego, czy parser poprawnie odczyta tekst zwrócony przez model. Metoda następnego tokenu porównuje bezpośrednio prawdopodobieństwa etykiet `ham` i `spam`, więc nie wprowadza osobnego problemu parsowania. Odpowiedź strukturyzowana ogranicza swobodę formatu przez wymuszenie pola `label`, ale nadal pozostaje wariantem opartym na generowaniu tekstu.

W tabeli 4.8 zestawiono najlepszy punkt kontrolny każdej metody oraz metryki, na których oparto porównanie.

Metoda	Konf.	Nr punktu	train_subset F1 [%]	enron spec. [%]	fraudulent recall [%]	spam_ham F1 [%]	S_{ext} [%]
SeqCls (01)	K06	10	99.9	95.3	100.0	94.1	96.5
Generowanie + parser (02)	K08	7	100.0	97.5	98.2	97.9	97.9
Następny token (03)	K08	7	99.9	96.9	99.9	97.2	98.0
Strukturyzowany JSON (04)	K08	7	99.7	97.7	99.6	96.7	98.0

Tabela 4.8.: Najlepsze punkty kontrolne uzyskane dla porównywanych metod.

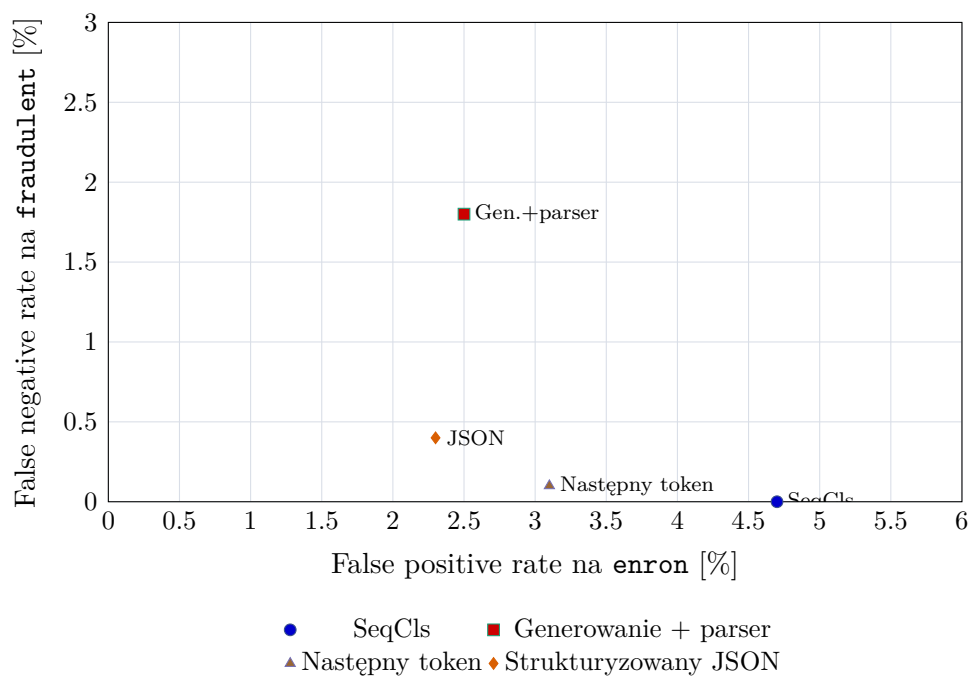
Najlepsze warianty metod opartych na modelu językowym uzyskały bardzo zbliżone wyniki zewnętrzne. Najwyższą wartość S_{ext} osiągnęła metoda 03, ale po zaokrągleniu metoda 04 ma ten sam wynik, a metoda 02 jest niższa o około 0,1 punktu procentowego. Różnice nie wskazują więc na jednoznaczną dominację jednej metody we wszystkich warunkach ewaluacji.



Rysunek 4.17.: Porównanie najlepszych wariantów metod na czterech zbiorach ewaluacyjnych. Dla `train_subset` i `spam_ham` pokazano F1, dla `enron specificity`, a dla `fraudulent_email_corpus recall`.

Źródło: opracowanie własne.

Rysunek 4.17 pokazuje, że różnice między metodami widać wyraźniej dopiero po rozbi-
ciu wyniku na poszczególne zbiory ewaluacyjne. Klasyfikacja sekwencji, czyli metoda 01,



Rysunek 4.18.: Porównanie błędów typu *false positive* na zbiorze **enron** oraz *false negative* na zbiorze **fraudulent_email_corpus** dla najlepszych wariantów metod. Punkty położone bliżej lewego dolnego rogu mają korzystniejszy profil błędów.

Źródło: opracowanie własne.

osiąga bardzo wysoki wynik na `train_subset` i pełny *recall* na `fraudulent_email_corpus`, ale wypada słabiej na `enron` oraz `spam_ham`. Taki profil oznacza, że metoda dobrze uczy się rozpoznawania wiadomości oszukańczych, lecz częściej niż pozostałe warianty oznacza poprawną korespondencję jako spam i gorzej radzi sobie na zbiorze mieszanym.

Metody 02, 03 i 04 są znacznie bliżej siebie. Wariant generowania i parsowania odpowiedzi ma najwyższy F1 na `spam_ham`, ale słabszy *recall* na zbiorze wiadomości oszukańczych. Metoda 04 lokuje się blisko najlepszych wariantów: osiąga najwyższą *specificity* na `enron` i bardzo wysoki *recall*, lecz jej F1 na `spam_ham` jest niższe niż w metodach 02 i 03.

Rysunek 4.18 pokazuje tę różnicę od strony profilu błędów. Metoda 01 nie przepuszcza wiadomości oszukańczych w zbiorze `fraudulent_email_corpus`, ale robi to kosztem większego odsetka fałszywych alarmów na `enron`. Metoda 02 ma odwrotny profil: rzadziej oznacza poprawne wiadomości jako spam, lecz częściej pomija wiadomości oszukańcze. Metody 03 i 04 znajdują się bliżej lewego dolnego rogu, co oznacza bardziej zrównoważony kompromis między tymi dwoma rodzajami błędów.

Najbardziej wyrównany profil ma metoda 03. Nie jest najlepsza na każdym pojedyńczym zbiorze, ale uzyskuje najwyższy wynik zbiorczy w wartościach niezaokrąglonych, prawie pełny *recall* na zbiorze wiadomości oszukańczych oraz wysokie F1 na `spam_ham`. Ważna jest też prostota inferencji: model nie musi generować odpowiedzi ani utrzymywać formatu tekstowego, ponieważ decyzja wynika z porównania dwóch prawdopodobieństw.

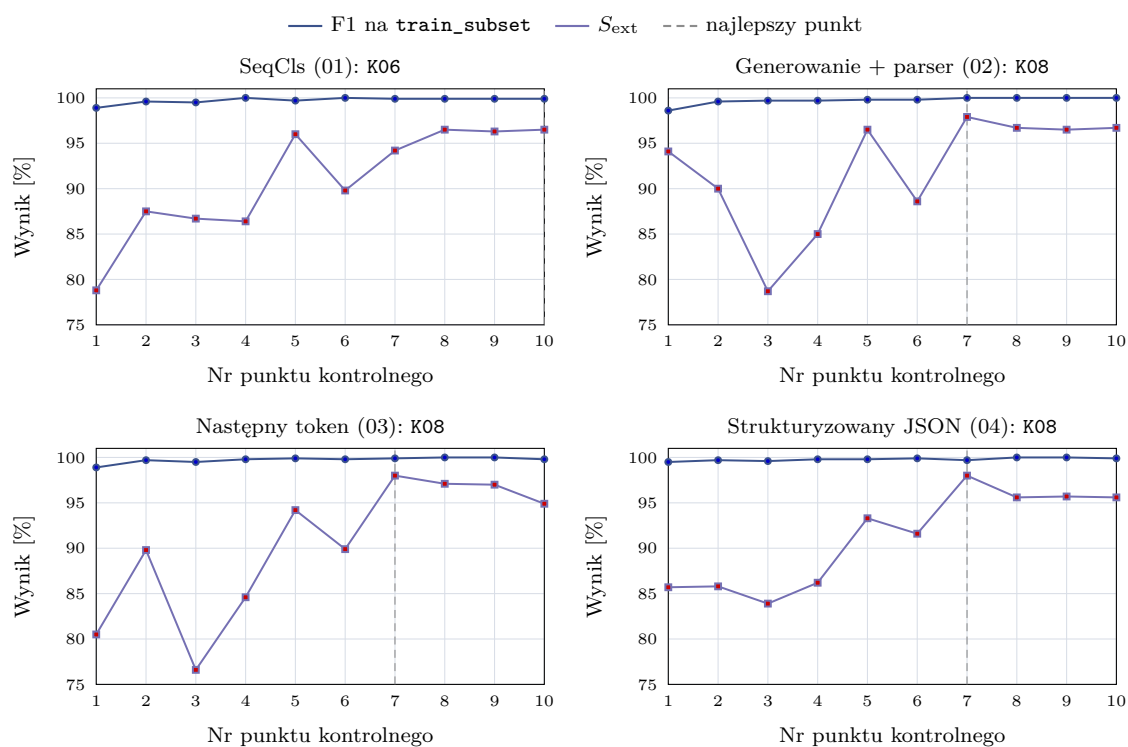
Moment osiągnięcia najwyższego S_{ext} różnił się między metodami. W tabeli 4.9 podano odpowiadający mu punkt kontrolny. Wcześniejsza stabilizacja wyniku zewnętrznego ułatwia wybór momentu zatrzymania treningu, choć nie oznacza krótszego czasu pojedynczej epoki.

Metoda	Konf.	Najlepszy nr punktu	Krok	S_{ext} [%]	Punkty konf.
SeqCls (01)	K06	10	final	96.5	10
Generowanie + parser (02)	K08	7	3000	97.9	10
Następny token (03)	K08	7	3000	98.0	10
Strukturyzowany JSON (04)	K08	7	3000	98.0	10

Tabela 4.9.: Moment uzyskania najlepszego wyniku zewnętrznego dla najlepszej konfiguracji każdej metody.

Rysunek 4.19 pokazuje, że we wszystkich metodach F1 na `train_subset` rośnie już w pierwszych ocenianych punktach kontrolnych i szybko zbliżało się do wartości bliskich 1. Wynik na danych podobnych do treningowych stabilizował się więc wcześniej niż wynik zewnętrzny. Krzywa S_{ext} była mniej regularna, zwłaszcza w pierwszej części treningu. Sam wynik na `train_subset` nie był więc wystarczającym kryterium wyboru modelu.

Metody 02, 03 i 04 osiągnęły najlepszy wynik zewnętrzny w siódmym ocenianym punkcie kontrolnym, odpowiadającym krokowi 3000. Dalszy trening nie poprawiał już wyniku zewnętrznej walidacji, mimo że F1 na próbce treningowej pozostawało bardzo wysokie. Metoda 01 dochodziła do najlepszego wyniku później, w końcowej części treningu. Pod wzglę-



Rysunek 4.19.: Przebieg F1 na `train_subset` oraz wyniku zewnętrznego S_{ext} dla najlepszej konfiguracji każdej metody. Linia przerywana oznacza punkt kontrolny o najwyższym wyniku zewnętrznym.

Źródło: opracowanie własne.

dem liczby punktów kontrolnych potrzebnych do uzyskania najlepszego wyniku warianty generatywne i wariant następnego tokenu zbiegały więc szybciej niż klasyfikacja sekwencji.

Najwyższy S_{ext} uzyskała metoda 03. Jej przewaga liczbowa nad metodami 02 i 04 jest mała, dlatego przy wyborze uwzględniono również brak generowania tekstu i parsera odpowiedzi. Metodę 03 wybrano do porównania z pozostałymi klasyfikatorami spamu.

4.5. Porównanie z metodami bazowymi

Porównanie obejmowało metody klasyfikujące treść wiadomości jako **ham** albo **spam**, a więc korzystające z tego samego wejścia co model językowy. Pełne filtry pocztowe wykorzystują również reputację nadawcy, wyniki SPF/DKIM/DMARC, listy reputacyjne i nagłówki wiadomości, których nie zawierały badane zbiory danych.

Metody bazowe trenowano na tym samym podziale danych co modele dostrajane. Do oceny wykorzystano `train_subset`, `enron`, `fraudulent_email_corpus` oraz `spam_ham`. Zastosowano ten sam dobór metryk co wcześniej: F1 dla `train_subset` i `spam_ham`, *specificity* dla `enron` oraz *recall* dla `fraudulent_email_corpus`. Taki dobór metryk ogranicza wpływ zbiorów jednoklasowych, dla których accuracy łatwo prowadziłoby do mylącej interpretacji.

4.5.1. Dobór parametrów metod bazowych

Parametry metod bazowych dobierano na 10-procentowej części walidacyjnej wydzielonej w opisanym wcześniej podziale 80/10/10. Trzy zbiory zewnętrzne wykorzystano dopiero po wyborze konfiguracji. Dla każdej sprawdzanej konfiguracji dobierano próg decyzyjny maksymalizujący F1 na zbiorze walidacyjnym. Regresję logistyczną, SVM i `fastText` porządkowano według F1 obliczonego dla całej części walidacyjnej, a kolejnym kryterium była *balanced accuracy*. W przypadku MiniLM pierwszym kryterium była średnia wyników F1 obliczonych osobno dla poszczególnych źródeł danych. Ograniczało to wpływ najliczniejszego lub najłatwiejszego źródła na wybór konfiguracji. Następnie uwzględniano F1 dla całej części walidacyjnej i *balanced accuracy*. W tabeli 4.10 podano F1 dla całej części walidacyjnej.

Zakres uczenia zależał od metody. Dla klasyfikatorów korzystających z TF-IDF na danych treningowych dopasowywano wektoryzator i właściwy klasyfikator. `fastText` uczył reprezentacje n-gramów oraz klasyfikator. Wagi MiniLM nie były aktualizowane: model wyznaczał wektory wiadomości, na których uczono regresję logistyczną. W DistilBERT dostrajano cały enkoder wraz z głowicą klasyfikacyjną.

Dla regresji logistycznej sprawdzono cztery wartości parametru C : 0,01; 0,1; 1 i 10. Dla SVM tę samą listę połączono z dwiema reprezentacjami TF-IDF: n-gramami słów długości 1–2 oraz n-gramami znaków długości 3–5. Strojenie `fastText` wykonano w dwóch etapach. Najpierw wybrano współczynnik uczenia i liczbę epok, a następnie wymiar wektorów oraz zakres n-gramów słów i znaków. Dla MiniLM porównano reprezentacje całej wiadomości,

fragmentów tekstu oraz osobno zakodowanych tematu i treści. Sprawdzono też normalizację wektorów, ważenie klas i parametr C regresji logistycznej. Dla regresji logistycznej oceniono 4 konfiguracje, dla SVM 8, dla **fastText** 26, a dla MiniLM 64. Naive Bayes nie podlegał strojeniu. Użyto n -gramów słów długości 1–2, słownika ograniczonego do 200 tys. cech i parametru wygładzania $\alpha = 1$.

Metoda	Reprezentacja	Parametry	Próg	Wal. F1 [%]
TF-IDF + Naive Bayes	słowa 1–2	$\alpha = 1$, maks. 200 tys. cech	0.500	–
TF-IDF + regresja logistyczna	słowa 1–2	$C = 10$	0.358	99.4
TF-IDF + SVM	znaki 3–5	$C = 10$	0.103	99.8
fastText	znaki 3–6, słowa 1	$lr = 0,25$, 20 epok, $d = 100$	0.252	99.6
MiniLM + regresja logistyczna	temat i treść osobno	po 256 tokenów, norm., $C = 10$	0.581	96.3
DistilBERT	klasyfikacja sekwencji	1 epoka, 512 tokenów, $lr = 2 \cdot 10^{-5}$	0.462	99.6

Tabela 4.10.: Konfiguracje metod bazowych wybrane na zbiorze walidacyjnym.

DistilBERT trenowano przez jedną epokę z kontekstem 512 tokenów, współczynnikiem uczenia $2 \cdot 10^{-5}$, rozmiarem partii 16 i liniowym harmonogramem współczynnika uczenia. Parametr *weight decay* wynosił 0,01, a rozgrzewka obejmowała pierwsze 6% kroków. Dla DistilBERT nie przeszukiwano hiperparametrów; na zbiorze walidacyjnym dobrano jedynie próg klasyfikacji. Trening jednej epoki zajął 7650 sekund, czyli około 2 godzin i 8 minut.

4.5.2. Skuteczność klasyfikacji

Wyniki metod bazowych i wybranego wariantu Qwen3-0.6B zestawiono w tabeli 4.11.

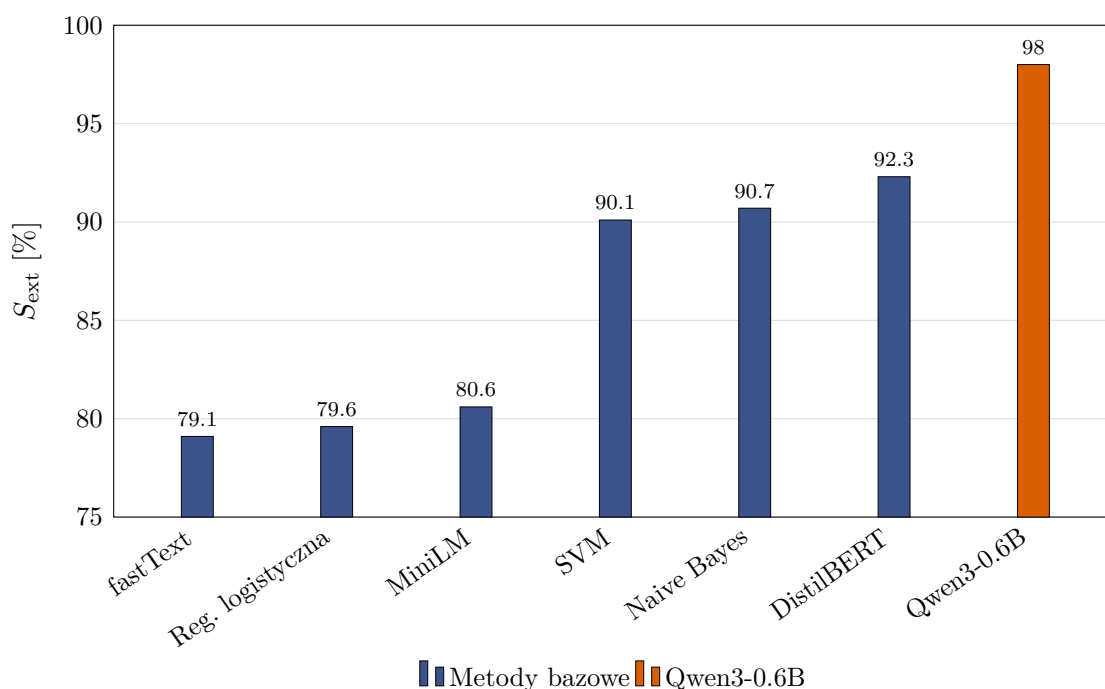
Metoda	Train F1 [%]	Enron spec. [%]	Fraud recall [%]	SpamHam F1 [%]	S_{ext} [%]
TF-IDF + Naive Bayes	96.8	87.5	99.4	85.1	90.7
TF-IDF + regresja logistyczna	100.0	70.5	100.0	68.4	79.6
TF-IDF + SVM	100.0	85.0	99.7	85.6	90.1
fastText	99.9	70.9	99.9	66.5	79.1
MiniLM + regresja logistyczna	96.8	75.0	98.5	68.4	80.6
DistilBERT	99.9	89.3	99.9	87.8	92.3
Qwen3 + LoRA, 03	99.9	96.9	99.9	97.2	98.0

Tabela 4.11.: Porównanie metod bazowych z najlepszym wariantem modelu Qwen3-0.6B dostrojonym metodą LoRA. Wartość S_{ext} obliczono na podstawie metryk ze zbiorów zewnętrznych: **enron**, **fraudulent_email_corpus** i **spam_ham**.

Najwyższy wynik wśród metod bazowych uzyskał DistilBERT: $S_{\text{ext}} = 92,3\%$. Wartość *specificity* na **enron** wyniosła 89,3%, *recall* na **fraudulent_email_corpus** 99,9%, a F1 na **spam_ham** 87,8%. TF-IDF z Naive Bayes osiągnął 90,7%, a SVM 90,1%. Różnica między tymi klasyfikatorami wyniosła 0,6 punktu procentowego. Naive Bayes uzyskał wyższą *specificity* na **enron**, natomiast SVM miał nieco wyższe F1 na **spam_ham**.

Regresja logistyczna, **fastText** i MiniLM uzyskały od 79,1% do 80,6%. Wyniki na podzbiórze danych treningowych i F1 walidacyjne pozostawały wysokie, lecz na **enron** wartości *specificity* wynosiły tylko 70,5–75,0%. Na **spam_ham** również dominowały fałszywe alarmy. Regresja logistyczna i **fastText** przepuściły odpowiednio 4 i 19 wiadomości spamowych, ale błędnie zatrzymały 518 i 534 wiadomości pożądane. MiniLM popełnił 451 takich błędów. Wysokie F1 na części walidacyjnej nie przełożyło się zatem na równie dobre wyniki dla wiadomości z innych źródeł.

Różnice między wynikami zewnętrznymi porównano również na rysunku 4.20.



Rysunek 4.20.: Wynik zewnętrzny S_{ext} dla metod bazowych oraz najlepszego wariantu Qwen3-0.6B + LoRA.

Źródło: opracowanie własne.

Qwen3-0.6B + LoRA uzyskał $S_{\text{ext}} = 98,0\%$, o 5,7 punktu procentowego więcej niż DistilBERT. Różnica wynikała głównie z *specificity* na **enron**, równego 96,9%, oraz F1 na **spam_ham**, równego 97,2%. Na jednoklasowym zbiorze wiadomości oszukańczych oba modele osiągnęły *recall* 99,9%. Wynik Qwen3 trzeba jednak interpretować z uwzględnieniem sposobu wyboru modelu: te same zbiory zewnętrzne służyły wcześniej do wskazania jego konfiguracji i punktu kontrolnego, podczas gdy parametry metod bazowych dobierano bez ich użycia.

4.5.3. Koszt inferencji

Koszt działania porównano na instancji z systemem Linux, kartą NVIDIA A100-SXM4 80 GB, procesorem Intel Xeon Platinum 8470 i 96 GiB pamięci operacyjnej. Procesom udostępniono 12 wątków CPU. Metody transformerowe korzystały z GPU, natomiast TF-IDF i **fastText** działały na CPU.

Do pomiaru wszystkich klasyfikatorów użyto tej samej, deterministycznie wybranej próbki 1000 wiadomości ze zbioru **spam_ham**. Próba obejmowała 716 wiadomości **ham** oraz 284 wiadomości **spam**. Do czasu inferencji wliczano również przygotowanie wejścia. W zależności od metody była to wektoryzacja TF-IDF, wyznaczenie reprezentacji przez MiniLM albo tokenizacja dla DistilBERT i Qwen3. Nie wliczano ładowania modelu ani pojedynczego przebiegu rozgrzewającego.

Czas ładowania obejmował odczyt zapisanego modelu i elementów potrzebnych do przetwarzania wejścia, na przykład wektoryzatora lub tokenizatora, do momentu gotowości klasyfikatora. Wartość tę zmierzono osobno i nie doliczano jej do czasu inferencji.

Rozmiar partii metod korzystających z GPU dobrano w krótkich testach poprzedzających pomiar. Wybrano 256 dla MiniLM, 512 dla DistilBERT i 64 dla Qwen3. Metody działające na CPU przetwarzały całą próbkę jednocześnie. Każdy pomiar wykonano trzykrotnie, a w tabeli 4.12 podano medianę. Wartość VRAM oznacza szczytową ilość pamięci przydzielonej przez PyTorch, a nie całą pamięć zarezerwowaną przez proces.

Porównanie ma charakter orientacyjny z powodu różnego budżetu strojenia i ustawień rozmiaru partii danych; nie stanowi pełnej analizy kosztu wdrożenia. Pomiar nie obejmował opóźnień pojedynczego żądania, wielu równoczesnych klientów ani kosztu utrzymania usługi.

Metoda	Partia	Ładowanie [s]	Inferencja [s]	Wiad./s	RAM [MiB]	VRAM [MiB]	Model [MiB]
TF-IDF + Naive Bayes	1000	1.430	0.156	6390.2	286.6	0.0	13.0
TF-IDF + regresja logistyczna	1000	1.515	0.160	6252.9	291.7	0.0	9.2
TF-IDF + SVM	1000	1.505	0.780	1282.7	289.5	0.0	8.1
fastText	1000	1.230	0.351	2850.0	1282.2	0.0	980.3
MiniLM + regresja logistyczna	256	0.971	1.393	717.9	1030.3	1266.2	87.3
DistilBERT	512	1.130	3.527	283.5	1145.6	8846.6	256.1
Qwen3 + LoRA, 03	64	4.164	7.992	125.1	2595.0	2787.2	1525.9

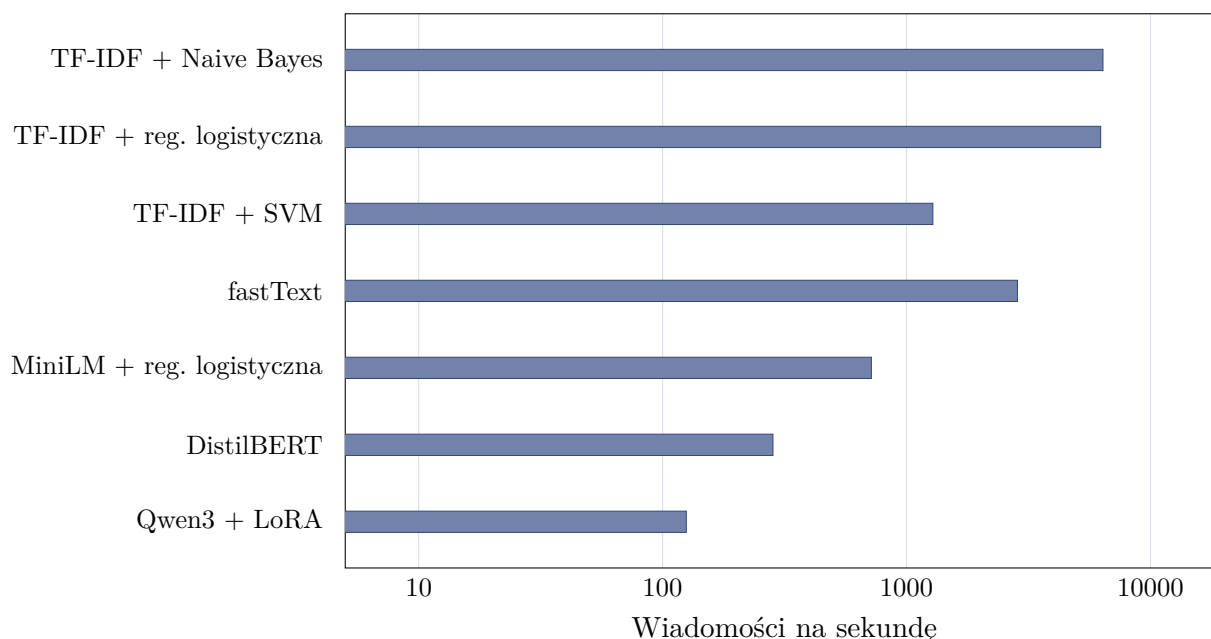
Tabela 4.12.: Mediana kosztu inferencji na wspólnej próbce 1000 wiadomości.

Naive Bayes i regresja logistyczna osiągnęły przepustowość około 6,3 tys. wiadomości na sekundę, a SVM 1,3 tys. Ich zapisane modele z wektoryzatorami zajmowały 8,1–13,0 MiB. **fastText** przetwarzał 2,9 tys. wiadomości na sekundę. Jego model zajmował 980 MiB, a szczytowe użycie pamięci operacyjnej wyniosło około 1,25 GiB.

MiniLM przetwarzał 717,9 wiadomości na sekundę. Szczytowe użycie pamięci wyniosło około 1,03 GiB RAM i 1266 MiB VRAM. DistilBERT osiągnął 283,5 wiadomości na sekundę, a inferencja na całej próbce trwała 3,53 sekundy. Jego model zajmował 256 MiB. Proces wykorzystał szczytowo około 1,12 GiB RAM i 8847 MiB VRAM. Tak duże użycie pamięci GPU wynikało między innymi z rozmiaru partii równego 512. DistilBERT był około 2,5 raza

wolniejszy od MiniLM i 2,3 raza szybszy od Qwen3.

Przepustowość wszystkich metod przedstawiono na rysunku 4.21.



Rysunek 4.21.: Przepustowość metod na wspólnej próbce 1000 wiadomości. Oś pozioma ma skalę logarytmiczną.

Źródło: opracowanie własne.

Qwen3 przetwarzał 125,1 wiadomości na sekundę. Inferencja trwała 7,99 sekundy, czyli około 51 razy dłużej niż dla Naive Bayes. Szczytowe użycie pamięci operacyjnej wyniosło około 2,53 GiB, a VRAM 2787 MiB. Model bazowy z adapterem zajmował 1526 MiB. Sam adapter LoRA miał 77,1 MiB i wymagał osobnego załadowania wag Qwen3-0.6B.

Qwen3 uzyskał najwyższy S_{ext} , ale miał najniższą przepustowość i największy rozmiar modelu. DistilBERT był około 2,3 raza szybszy, lecz przy zastosowanym rozmiarze partii zużywał więcej VRAM, a jego wynik był niższy o 5,7 punktu procentowego. Naive Bayes i SVM osiągnęły co najmniej 1,3 tys. wiadomości na sekundę, a ich modele zajmowały nie więcej niż 13 MiB. Ich wyniki zewnętrzne były jednak niższe od wyniku Qwen3 odpowiednio o 7,3 i 7,9 punktu procentowego.

5. Podsumowanie i wnioski

5.1. Synteza przeprowadzonych badań

Praca dotyczyła wykorzystania modelu Qwen3-0.6B do wykrywania niechcianej poczty na podstawie tematu i treści wiadomości. Publiczne korpusy sprowadzono do wspólnego schematu, usunięto puste rekordy i duplikaty, a dane treningowe zbalansowano. Trzy źródła wyłączono z treningu i wykorzystano podczas wyboru konfiguracji do oceny wyników na wiadomościach o innej charakterystyce.

Model bez dostrajania przypisywał wszystkie oceniane wiadomości do klasy `ham`. Na zbalansowanym `train_subset` oznaczało to 50,0% *accuracy* i F1 równe 0% dla klasy pozytywnej. Po dostrojeniu najlepszy wynik uzyskała konfiguracja K08: kontekst 512 tokenów, współczynnik uczenia 10^{-4} , LoRA $r = 32$, $\alpha = 64$ oraz dropout 0,05. Siódmy oceniany punkt kontrolny osiągnął $S_{\text{ext}} = 98,0\%$, F1 równe 97,2% na `spam_ham`, *specificity* 96,9% na `enron` i *recall* 99,9% na `fraudulent_email_corpus`.

Najlepsze warianty generowania etykiety, oceny następnego tokenu i odpowiedzi ustrukturyzowanej różniły się wynikiem zbiorczym o około 0,1 punktu procentowego. Najlepszą metodą bazową był DistilBERT z wynikiem $S_{\text{ext}} = 92,3\%$. TF-IDF z Naive Bayes i SVM osiągnęły odpowiednio 90,7% i 90,1%.

5.2. Odpowiedzi na pytania badawcze

1. Jakie wyniki osiąga mały model językowy dostrojony za pomocą LoRA na próbce treningowej i w zewnętrznej walidacji obejmującej źródła niewykorzystane do aktualizacji wag?

W badanych warunkach dostrojenie za pomocą LoRA poprawiło klasyfikację zarówno na próbce treningowej, jak i na trzech korpusach zewnętrznych. Najlepszy wariant osiągnął F1 równe 99,9% na `train_subset` oraz $S_{\text{ext}} = 98,0\%$ w zewnętrznej walidacji. Druga wartość nie jest wynikiem niezależnego testu końcowego, ponieważ służyła do wyboru konfiguracji.

2. Który z badanych sposobów użycia modelu językowego daje najkorzystniejsze połączenie skuteczności, niezależności decyzji od formatu generowanej odpowiedzi i prostoty inferencji?

Wybrano metodę oceny następnego tokenu, chociaż jej przewaga liczbowa nad generowaniem etykiety i odpowiedzi ustrukturyzowanej była niewielka. Model porównywał prawdopodobieństwa etykiet `spam` i `ham`, bez generowania pełnej odpowiedzi wymagającej późniejszego parsowania. Klasyfikacja sekwencji również nie wymagała parsera, lecz uzyskała niższy wynik zewnętrzny, równy 96,5%.

3. Jak parametry treningu oraz jego czas trwania wpływają na wynik modelu na korpusach niewykorzystanych w treningu?

Metryki na danych podobnych do treningowych szybko zbliżały się do 100%, dlatego o wyborze konfiguracji decydowały wyniki na pozostałych korpusach. Warianty ze współczynnikiem uczenia 10^{-6} były na nich słabsze. Wśród wariantów ze współczynnikiem 10^{-4} najlepiej wypadł adapter z $r = 32$ i $\alpha = 64$. Mniejszy adapter uzyskał niższy wynik, a dalsze zwiększanie jego pojemności nie dawało regularnej poprawy. Wpływu dropout nie udało się określić jednoznacznie.

Kontekst 1024 tokenów wydłużał trening, ale nie poprawił najlepszego rezultatu. Najwyższe S_{ext} dla najlepszych konfiguracji występowało jeszcze przed końcem treningu. Wybór ostatniego adaptera albo kierowanie się wyłącznie metrykami treningowymi prowadziło więc do wyboru słabszego wariantu.

4. Jaki kompromis między skutecznością a kosztem inferencji wynika z porównania dostrojonego modelu językowego z prostszymi klasyfikatorami?

Qwen3 uzyskał wynik S_{ext} wyższy o 5,7 punktu procentowego od DistilBERT, ale przetwarzał wiadomości około 2,3 raza wolniej. Metody TF-IDF działały jeszcze szybciej, przy niższych wynikach zewnętrznych. Pomiar na karcie NVIDIA A100 dotyczył przetwarzania w partiach, nie określa więc opóźnienia pojedynczego żądania ani kosztu utrzymania serwera. Porównanie ma charakter orientacyjny, ponieważ metody różniły się budżetem strojenia oraz rozmiarem partii użytym podczas inferencji.

5.3. Ograniczenia pracy

Trzy zbiory zewnętrzne nie były używane do treningu Qwen3, ale ich wyniki służyły do wyboru konfiguracji i punktu kontrolnego. W metodach bazowych parametry dobierano wyłącznie na części walidacyjnej, a wymienione zbiory wykorzystano dopiero do oceny wybranego wariantu.

Eksperymenty przeprowadzono na publicznych korpusach, z których część powstała wiele lat temu. Wiadomości z poszczególnych źródeł różnią się formatowaniem, słownictwem i sposobem nadawania etykiet. Deduplikacja ograniczyła liczbę powtórzeń, lecz nie usunęła wszystkich cech pozwalających rozpoznać źródło wiadomości. Wyników nie potwierdzono na bieżącym strumieniu poczty.

Każdy trening wykonano z jednym ziarnem losowym. Nie obliczano przedziałów ufności i nie powtarzano tych samych konfiguracji, dlatego różnice rzędu 0,1–0,3 punktu procentowego mogą wynikać częściowo z losowości treningu lub próbkowania. Dotyczy to zwłaszcza kolejności najlepszych sposobów dostrajania.

Zakres strojenia różnił się między metodami. Szesnaście konfiguracji i pośrednie punkty kontrolne oceniono dla metody następnego tokenu. Dla klasyfikacji sekwencji, generowania etykiety i odpowiedzi ustrukturyzowanej sprawdzono po cztery konfiguracje wybrane na podstawie tego eksperymentu. Metody bazowe oceniono dla ograniczonych zbiorów parametrów i jednego podziału walidacyjnego. Naive Bayes nie podlegał strojeniu, a DistilBERT wytrenowano tylko z jednym zestawem parametrów. Przedstawione wyniki dotyczą więc wyłącznie sprawdzonych konfiguracji.

Badanie objęło jeden model bazowy i jeden jego rozmiar. Pomiar wydajności wykonano w jednym środowisku sprzętowym, na próbce obejmującej 1000 wiadomości. Nie sprawdzono zachowania przy wielu równoczesnych żądaniach, opóźnień krańcowych, zużycia energii ani kosztu pracy serwera. Zmierzone wartości pokazują różnice między metodami w badanym środowisku, ale nie opisują infrastruktury produkcyjnej.

Klasyfikacja binarna łączy spam reklamowy, phishing i wiadomości oszukańcze w jedną klasę. Model nie określa rodzaju zagrożenia ani nie wyjaśnia, który fragment wiadomości wpłynął na decyzję. Do modelu przekazywano tylko temat i treść, bez reputacji nadawcy, pełnych nagłówków i mechanizmów uwierzytelniania domeny.

5.4. Możliwość wdrożenia

Koszt inferencji porównano na wspólnej próbce 1000 wiadomości. Qwen3 z adapterem LoRA przetwarzał 125,1 wiadomości na sekundę, a przetworzenie całej próbki trwało 7,99 sekundy. Proces wykorzystał szczytowo około 2,53 GiB pamięci operacyjnej i 2787 MiB VRAM, natomiast model bazowy i adapter zajmowały razem około 1526 MiB na dysku. DistilBERT przetwarzał 283,5 wiadomości na sekundę, wykorzystywał około 1,12 GiB pamięci operacyjnej i zajmował 256 MiB na dysku. Dla TF-IDF z Naive Bayes przepustowość wyniosła około 6,4 tys. wiadomości na sekundę, szczytowe użycie pamięci 287 MiB, a rozmiar modelu z wektoryzatorem 13 MiB.

W układzie kaskadowym uwierzytelnianie nadawcy, reguły, reputacja i tańszy klasyfikator tekstu mogłyby obsługiwać przypadki jednoznaczne. Model językowy analizowałby wtedy tylko pozostałe wiadomości. Eksperymenty nie sprawdzały jednak, czy Qwen3 poprawia wyniki właśnie dla przykładów, co do których prostszy klasyfikator jest niepewny.

Model Qwen3-0.6B z adapterem LoRA klasyfikuje wyłącznie temat i treść wiadomości. W systemie produkcyjnym jego wynik byłby jednym z kilku sygnałów używanych do podjęcia decyzji. Samodzielne odrzucanie wiadomości wymagałoby dokładniejszej kontroli fałszywych alarmów i testów na danych produkcyjnych.

5.5. Kierunki dalszych badań

Kolejny eksperyment powinien oddzielić wybór konfiguracji Qwen3 od testu na zbiorach zewnętrznych. Walidacja *leave-one-source-out* umożliwiłaby kolejne wyłączanie z treningu każdego źródła. Drugą możliwością jest podział czasowy, w którym do testu trafiają nowsze wiadomości. Te same konfiguracje należałoby również wytrenować z kilkoma ziarnami losowymi, aby oszacować zmienność wyników. Szersze strojenie Naive Bayes i DistilBERT wyrównałoby natomiast zakres poszukiwania parametrów między porównywanymi metodami.

Pomiar na niezbalansowanym strumieniu pokazałby liczbę fałszywych alarmów przy proporcjach klas zbliżonych do ruchu produkcyjnego. Osobne etykiety spamu reklamowego, phishingu i oszustw pozwoliłyby natomiast ocenić rozpoznawanie rodzaju zagrożenia. Progi klasyfikacji powinny wtedy odpowiadać przyjętemu kosztowi *false positive* i *false negative*.

Ocena wdrożeniowa wymaga porównania wariantów kwantyzacji, silników inferencji i ustawień przetwarzania partiami na sprzęcie serwerowym. Oprócz średniej przepustowości należy zmierzyć opóźnienie, użycie pamięci przy równoczesnych żądaniach oraz koszt przetworzenia określonej liczby wiadomości. Osobnego testu wymaga też układ kaskadowy z tanim klasyfikatorem pierwszego etapu.

Qwen3 pozostaje kandydatem do roli klasyfikatora drugiego etapu. Skuteczności takiego układu nie sprawdzono w tej pracy.

Bibliografia

- [1] National Institute of Standards and Technology. *spam*. URL: <https://csrc.nist.gov/glossary/term/spam> (term. wiz. 15.06.2026).
- [2] F. Janez-Martino i in. „Classifying spam emails using agglomerative hierarchical clustering and a topic-based approach”. W: *arXiv preprint arXiv:2402.05296* (2024). URL: <https://arxiv.org/abs/2402.05296> (term. wiz. 15.06.2026).
- [3] National Institute of Standards and Technology. *phishing*. URL: <https://csrc.nist.gov/glossary/term/phishing> (term. wiz. 15.06.2026).
- [4] Bitdefender. *Nowa kampania phishingowa podszywająca się pod InPost*. URL: <https://bitdefender.pl/nowa-kampania-phishingowa-podszywajaca-sie-pod-inpost/> (term. wiz. 16.06.2026).
- [5] U.S. Securities and Exchange Commission. *Advance Fee Fraud*. URL: <https://www.investor.gov/protect-your-investments/fraud/types-fraud/advance-fee-fraud> (term. wiz. 08.07.2026).
- [6] Kristoffer Torngaard Pedersen i in. „Deepfake-Driven Social Engineering: Threats, Detection Techniques, and Defensive Strategies in Corporate Environments”. W: *Journal of Cybersecurity and Privacy* 5.2 (2025), s. 18. DOI: [10.3390/jcp5020018](https://doi.org/10.3390/jcp5020018). URL: <http://www.mdpi.com/2624-800X/5/2/18> (term. wiz. 18.06.2026).
- [7] Rico Sennrich, Barry Haddow i Alexandra Birch. „Neural Machine Translation of Rare Words with Subword Units”. W: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. 2016, s. 1715–1725. DOI: [10.18653/v1/P16-1162](https://doi.org/10.18653/v1/P16-1162). URL: <https://arxiv.org/abs/1508.07909> (term. wiz. 19.06.2026).
- [8] Alec Radford i in. *Language Models are Unsupervised Multitask Learners*. Spraw. tech. OpenAI, 2019. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf (term. wiz. 19.06.2026).
- [9] OpenAI. *GPT-2 encoder.py*. URL: <https://github.com/openai/gpt-2/blob/master/src/encoder.py> (term. wiz. 19.06.2026).
- [10] Yoshua Bengio i in. „A Neural Probabilistic Language Model”. W: *Journal of Machine Learning Research* 3 (2003), s. 1137–1155. URL: <https://www.jmlr.org/papers/v3/bengio03a.html> (term. wiz. 23.06.2026).

- [11] Tom B. Brown i in. „Language Models are Few-Shot Learners”. W: *Advances in Neural Information Processing Systems*. T. 33. 2020, s. 1877–1901. URL: <https://arxiv.org/abs/2005.14165> (term. wiz. 23.06.2026).
- [12] Long Ouyang i in. „Training Language Models to Follow Instructions with Human Feedback”. W: *Advances in Neural Information Processing Systems*. T. 35. 2022, s. 27730–27744. URL: <https://arxiv.org/abs/2203.02155> (term. wiz. 23.06.2026).
- [13] Qwen Team. *Qwen3-0.6B Model Card*. URL: <https://huggingface.co/Qwen/Qwen3-0.6B> (term. wiz. 07.06.2026).
- [14] Ashish Vaswani i in. „Attention Is All You Need”. W: *Advances in Neural Information Processing Systems*. T. 30. 2017. URL: <https://arxiv.org/abs/1706.03762> (term. wiz. 24.06.2026).
- [15] Jay Alammar. *The Illustrated Transformer*. URL: <https://jalammar.github.io/illustrated-transformer/> (term. wiz. 30.06.2026).
- [16] Yutao Sun i in. „Efficient Attention Mechanisms for Large Language Models: A Survey”. W: *arXiv preprint arXiv:2507.19595* (2025). URL: <https://arxiv.org/abs/2507.19595> (term. wiz. 30.06.2026).
- [17] Ian Goodfellow, Yoshua Bengio i Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <https://www.deeplearningbook.org/> (term. wiz. 02.07.2026).
- [18] Mengnan Du i in. „Shortcut Learning of Large Language Models in Natural Language Understanding”. W: *arXiv preprint arXiv:2208.11857* (2022). URL: <https://arxiv.org/abs/2208.11857> (term. wiz. 08.07.2026).
- [19] Jason Wei i in. „Finetuned Language Models Are Zero-Shot Learners”. W: *International Conference on Learning Representations*. 2022. URL: <https://arxiv.org/abs/2109.01652> (term. wiz. 02.07.2026).
- [20] Neil Houlsby i in. „Parameter-Efficient Transfer Learning for NLP”. W: *Proceedings of the 36th International Conference on Machine Learning*. 2019, s. 2790–2799. URL: <https://arxiv.org/abs/1902.00751> (term. wiz. 02.07.2026).
- [21] Vladislav Lialin i in. „Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning”. W: *arXiv preprint arXiv:2303.15647* (2023). URL: <https://arxiv.org/abs/2303.15647> (term. wiz. 02.07.2026).
- [22] Edward J. Hu i in. „LoRA: Low-Rank Adaptation of Large Language Models”. W: *International Conference on Learning Representations*. 2022. URL: <https://arxiv.org/abs/2106.09685> (term. wiz. 02.07.2026).
- [23] Paulius Micikevicius i in. „Mixed Precision Training”. W: *International Conference on Learning Representations*. 2018. URL: <https://arxiv.org/abs/1710.03740> (term. wiz. 08.07.2026).

- [24] Samyam Rajbhandari i in. „ZeRO: Memory Optimizations Toward Training Trillion Parameter Models”. W: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2020. URL: <https://arxiv.org/abs/1910.02054> (term. wiz. 08.07.2026).
- [25] Google. *Email sender guidelines*. URL: <https://support.google.com/mail/answer/81126> (term. wiz. 09.07.2026).
- [26] Microsoft. *Recommendations for Microsoft 365 security settings*. URL: <https://learn.microsoft.com/en-us/defender-office-365/recommended-settings-for-eop-and-office365> (term. wiz. 09.07.2026).
- [27] Apache SpamAssassin Project. *Mail::SpamAssassin::Conf - SpamAssassin configuration file*. URL: https://spamassassin.apache.org/full/4.0.x/doc/Mail_SpamAssassin_Conf.html (term. wiz. 09.07.2026).
- [28] Rspamd Project. *Rspamd modules*. URL: <https://docs.rspamd.com/modules/> (term. wiz. 09.07.2026).
- [29] Trevor Wood i in. „Systematic Literature Review: Anti-Phishing Defences and Their Application to Before-the-click Phishing Email Detection”. W: *arXiv preprint arXiv:2204.13054* (2022). URL: <https://arxiv.org/abs/2204.13054> (term. wiz. 09.07.2026).
- [30] Ion Androustopoulos i in. „Learning to Filter Spam E-Mail: A Comparison of a Naive Bayesian and a Memory-Based Approach”. W: *arXiv preprint cs/0009009* (2000). URL: <https://arxiv.org/abs/cs/0009009> (term. wiz. 09.07.2026).
- [31] Francisco Janez-Martino i in. „Classification of Spam Emails through Hierarchical Clustering and Supervised Learning”. W: *arXiv preprint arXiv:2005.08773* (2020). URL: <https://arxiv.org/abs/2005.08773> (term. wiz. 09.07.2026).
- [32] Quoc V. Le i Tomas Mikolov. „Distributed Representations of Sentences and Documents”. W: *Proceedings of the 31st International Conference on Machine Learning*. T. 32. Proceedings of Machine Learning Research. PMLR, 2014, s. 1188–1196. URL: <https://proceedings.mlr.press/v32/le14.html> (term. wiz. 09.07.2026).
- [33] Zhixiang Xu i in. „An alternative text representation to TF-IDF and Bag-of-Words”. W: *arXiv preprint arXiv:1301.6770* (2013). DOI: [10.48550/arXiv.1301.6770](https://doi.org/10.48550/arXiv.1301.6770). URL: <https://arxiv.org/abs/1301.6770> (term. wiz. 09.07.2026).
- [34] Armand Joulin i in. „Bag of Tricks for Efficient Text Classification”. W: *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*. 2017, s. 427–431. URL: <https://arxiv.org/abs/1607.01759> (term. wiz. 09.07.2026).

- [35] Nils Reimers i Iryna Gurevych. „Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. W: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. 2019, s. 3982–3992. URL: <https://arxiv.org/abs/1908.10084> (term. wiz. 09.07.2026).
- [36] Victor Sanh i in. „DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter”. W: *arXiv preprint arXiv:1910.01108* (2019). URL: <https://arxiv.org/abs/1910.01108> (term. wiz. 09.07.2026).
- [37] Shijing Si i in. „Evaluating the Performance of ChatGPT for Spam Email Detection”. W: *arXiv preprint arXiv:2402.15537* (2024). DOI: [10.48550/arXiv.2402.15537](https://doi.org/10.48550/arXiv.2402.15537). URL: <https://arxiv.org/abs/2402.15537> (term. wiz. 15.08.2026).
- [38] Sergio Rojas-Galeano. „Zero-Shot Spam Email Classification Using Pre-trained Large Language Models”. W: *arXiv preprint arXiv:2405.15936* (2024). DOI: [10.48550/arXiv.2405.15936](https://doi.org/10.48550/arXiv.2405.15936). URL: <https://arxiv.org/abs/2405.15936> (term. wiz. 15.08.2026).
- [39] An Yang i in. „Qwen3 Technical Report”. W: *arXiv preprint arXiv:2505.09388* (2025). URL: <https://arxiv.org/abs/2505.09388> (term. wiz. 07.06.2026).
- [40] Google DeepMind. *Gemma 3 Model Card*. URL: https://ai.google.dev/gemma/docs/core/model_card_3 (term. wiz. 07.06.2026).
- [41] Meta. *Llama 3.2 Model Card*. URL: https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md (term. wiz. 07.06.2026).
- [42] Hugging Face TB. *SmolLM2-1.7B Model Card*. URL: <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B> (term. wiz. 07.06.2026).
- [43] Marah Abdin i in. „Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone”. W: *arXiv preprint arXiv:2404.14219* (2024). URL: <https://arxiv.org/abs/2404.14219> (term. wiz. 07.06.2026).
- [44] Microsoft. *Phi-3-mini-4k-instruct Model Card*. URL: <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct> (term. wiz. 07.06.2026).
- [45] *Enron Email Dataset*. URL: <https://www.kaggle.com/datasets/wcukierski/enron-email-dataset> (term. wiz. 27.05.2026).
- [46] *Emails for Spam or Ham Classification TREC 2007*. URL: <https://www.kaggle.com/datasets/bayes2003/emails-for-spam-or-ham-classification-trec-2007> (term. wiz. 27.05.2026).
- [47] *CEAS 08*. URL: <https://www.kaggle.com/datasets/doryanay/ceas-08> (term. wiz. 27.05.2026).

- [48] *Spam Assassin Email Classification Dataset*. URL: <https://www.kaggle.com/datasets/ganiyuolalekan/spam-assassin-email-classification-dataset> (term. wiz. 27.05.2026).
- [49] *Spam Mails Dataset*. URL: <https://www.kaggle.com/datasets/venky73/spam-mails-dataset> (term. wiz. 27.05.2026).
- [50] *Fraudulent Email Corpus*. URL: <https://www.kaggle.com/datasets/rtatman/fraudulent-email-corpus> (term. wiz. 27.05.2026).
- [51] O. B. Longe i in. „Seeing Beyond the Surface, Understanding and Tracking Fraudulent Cyber Activities”. W: *International Journal of Computer Science and Information Security* 6.3 (2009), s. 124–135. URL: <https://arxiv.org/abs/1001.1993> (term. wiz. 08.07.2026).
- [52] *Nazario 5 Datatest*. URL: <https://www.kaggle.com/datasets/zeyadkarimfcai/nazario-5-datatest> (term. wiz. 27.05.2026).
- [53] *Ling-Spam Dataset*. URL: <https://www.kaggle.com/datasets/mandygu/lingspam-dataset> (term. wiz. 27.05.2026).
- [54] William W. Cohen. *Enron Email Dataset*. Carnegie Mellon University. URL: <https://www.cs.cmu.edu/~enron/> (term. wiz. 15.08.2026).
- [55] Gordon V. Cormack. „TREC 2007 Spam Track Overview”. W: *Proceedings of the Sixteenth Text Retrieval Conference (TREC 2007)*. 2007. URL: <https://plg.uwaterloo.ca/~gvcormac/spamtrack07.pdf> (term. wiz. 15.08.2026).
- [56] Apache SpamAssassin Project. *SpamAssassin Public Mail Corpus*. URL: <https://spamassassin.apache.org/old/publiccorpus/readme.html> (term. wiz. 15.08.2026).
- [57] Jose Nazario. *Phishing Corpus*. URL: <https://monkey.org/~jose/phishing/> (term. wiz. 15.08.2026).

Summary

This thesis investigates whether a small open-weight language model can be adapted to distinguish unwanted email from legitimate correspondence. Qwen3-0.6B was fine-tuned with Low-Rank Adaptation (LoRA). The classifier used only the message subject and body. The positive class included advertising spam, phishing, and fraudulent messages. Attachments, transport headers, sender reputation, and email authentication results were outside the scope of the experiments.

Public email datasets were converted to a common schema, cleaned, deduplicated, and split into training, validation, and test subsets in an 80/10/10 ratio. Three datasets were excluded from training and evaluated separately. Sixteen configurations were compared for classification based on the probability of the next token. They differed in context length, learning rate, LoRA rank, scaling factor, and dropout. Intermediate training checkpoints were evaluated. Three alternative task formulations were also tested: sequence classification, label generation with parsing, and structured response generation.

The unmodified model assigned every evaluated message to the legitimate class. After fine-tuning, the best configuration used a context of 512 tokens, a learning rate of 10^{-4} , LoRA rank $r = 32$, scaling factor $\alpha = 64$, and dropout of 0.05. It achieved an aggregate external score of 98.0%, including an F1 score of 97.2% on the `spam_ham` dataset, specificity of 96.9% on `enron`, and recall of 99.9% on `fraudulent_email_corpus`. The best variants of label generation, next-token scoring, and structured generation differed by approximately 0.1 percentage point. Next-token scoring was selected because it compares the probabilities of the `spam` and `ham` labels directly and requires no parser for generated text.

The three external datasets were used to select the Qwen3 configuration and checkpoint. The aggregate score is therefore an external validation result, not an independent final test estimate.

The selected model was compared with classifiers based on TF-IDF, fastText, MiniLM sentence representations, and DistilBERT. DistilBERT obtained the best result among these methods, with an aggregate external score of 92.3%. Qwen3 exceeded this result by 5.7 percentage points. On a common sample of 1,000 messages and an NVIDIA A100 GPU, Qwen3 processed 125.1 messages per second and used approximately 2,595 MiB of RAM and 2,787 MiB of GPU memory. DistilBERT reached 283.5 messages per second, while the TF-IDF classifiers processed between 1.3 and 6.4 thousand messages per second.

These results are insufficient to justify deploying the model as a complete production email filter. The experiments covered one base model, one random seed, public datasets, and a single inference environment. In a production system, the model could instead act as a second-stage classifier for messages not resolved by authentication checks, sender reputation, rules, or a less expensive text classifier.

Keywords: unwanted email, spam detection, phishing, language models, Qwen3, LoRA, text classification

Spis rysunków

2.1. Przykład tokenizacji byte-level BPE na podstawie GPT-2	8
2.2. Przepływ informacji w modelu opartym na architekturze Transformer	14
2.3. Schemat działania mechanizmu multi-head self-attention.	15
2.4. Uproszczony schemat sieci feed-forward typu MLP w bloku Transformera. . .	17
2.5. Uproszczony schemat adaptera LoRA	19
2.6. Porównanie kosztu inferencji i dostrajania	22
4.1. Porównanie liczności dostępnych surowych zbiorów danych. Skala pozioma jest logarytmiczna.	33
4.2. Rozkład klas w surowych zbiorach. Klasa pozytywna oznacza spam, phishing lub wiadomość oszukańczą.	34
4.3. Udział źródeł w najczęściej używanym zbiorze treningowym.	35
4.4. Liczba rekordów przed deduplikacją oraz po deduplikacji.	39
4.5. Rozmiary grup duplikatów po połączeniu zbiorów danych poza enron	40
4.6. Rozkład długości treści wiadomości według etykiety w analizowanym wariacie danych.	41
4.7. Terminy najsilniej powiązane z klasą pozytywną według wygładzonego ilorazu szans.	42
4.8. Przebieg funkcji straty dla wybranych konfiguracji treningu. Wykres <i>training loss</i> został wygładzony medianą kroczącą, aby ograniczyć szum wynikający z pomiaru na kolejnych batchach.	48
4.9. Przebieg F1 oraz <i>accuracy</i> na zbiorze walidacyjnym dla wybranych konfiguracji treningu.	49
4.10. Czas treningu poszczególnych konfiguracji.	49
4.11. Najlepszy wynik zewnętrzny uzyskany przez każdą analizowaną konfigurację metody klasyfikacji następnego tokenu.	51
4.12. Porównanie metryk dla pięciu najlepszych konfiguracji według wyniku zewnętrznego.	52
4.13. Przebieg <i>accuracy</i> dla analizowanych konfiguracji. Każdy panel odpowiada jednej konfiguracji, a linie przedstawiają osobne zbiory ewaluacyjne. . . .	53
4.14. Różnica między F1 na próbce treningowej train_subset a wynikiem zewnętrznej walidacji dla najlepszych punktów kontrolnych poszczególnych konfiguracji.	54

4.15. Przebieg wyniku zewnętrznego dla pięciu najlepszych konfiguracji w kolejnych ocenianych punktach kontrolnych.	55
4.16. Odsetki błędów typu <i>false positive</i> i <i>false negative</i> dla wybranego punktu kontrolnego konfiguracji K08.	56
4.17. Porównanie najlepszych wariantów metod na czterech zbiorach ewaluacyjnych. Dla <code>train_subset</code> i <code>spam_ham</code> pokazano F1, dla <code>enron</code> <i>specificity</i> , a dla <code>fraudulent_email_corpus</code> <i>recall</i>	58
4.18. Porównanie błędów typu <i>false positive</i> na zbiorze <code>enron</code> oraz <i>false negative</i> na zbiorze <code>fraudulent_email_corpus</code> dla najlepszych wariantów metod. Punkty położone bliżej lewego dolnego rogu mają korzystniejszy profil błędów.	59
4.19. Przebieg F1 na <code>train_subset</code> oraz wyniku zewnętrznego S_{ext} dla najlepszej konfiguracji każdej metody. Linia przerywana oznacza punkt kontrolny o najwyższym wyniku zewnętrznym.	61
4.20. Wynik zewnętrzny S_{ext} dla metod bazowych oraz najlepszego wariantu Qwen3-0.6B + LoRA.	64
4.21. Przepustowość metod na wspólnej próbce 1000 wiadomości. Oś pozioma ma skalę logarytmiczną.	66

Spis tabel

2.1. Macierz pomyłek dla binarnej klasyfikacji wiadomości	9
3.1. Zakres metod omawianych w przeglądzie oraz ich wykorzystanie w części eksperymentalnej.	27
4.1. Porównanie małych rodzin modeli językowych rozważanych jako baza dla klasyfikatora.	29
4.2. Charakterystyka zbiorów danych	32
4.3. Dostępne kolumny w surowych zbiorach danych oraz sposób ich parsowania .	36
4.4. Parametry deduplikacji na poziomie high	38
4.5. Wyniki modelu bazowego bez dostrajania. Wartości podano w procentach. .	45
4.6. Konfiguracje treningu objęte analizą punktów kontrolnych.	47
4.7. Ranking najlepszych punktów kontrolnych dla konfiguracji metody klasyfikacji następnego tokenu. Wynik zewnętrzny jest średnią z <i>specificity</i> na enron , <i>recall</i> na fraudulent_email_corpus oraz F1 na spam_ham	50
4.8. Najlepsze punkty kontrolne uzyskane dla porównywanych metod.	58
4.9. Moment uzyskania najlepszego wyniku zewnętrznego dla najlepszej konfiguracji każdej metody.	60
4.10. Konfiguracje metod bazowych wybrane na zbiorze walidacyjnym.	63
4.11. Porównanie metod bazowych z najlepszym wariantem modelu Qwen3-0.6B dostrojonym metodą LoRA. Wartość S_{ext} obliczono na podstawie metryk ze zbiorów zewnętrznych: enron , fraudulent_email_corpus i spam_ham	63
4.12. Mediana kosztu inferencji na wspólnej próbce 1000 wiadomości.	65

Lista kodów źródłowych

2.1. Schemat poglądowy rozmowy w stylu ChatML z blokiem analizy i wywołaniem narzędzia.	11
4.1. Prompt po zastosowaniu szablonu czatu Qwen3 w metodzie oceny następnego tokena.	44
4.2. Prompt wykorzystany do oceny modelu bazowego w wariancie generowania i parsowania.	44